Containers & OpenShift Workshop

Alexander Mamani - IBM
Winnie Imafidon - IBM
Christian Zaccaria - Red Hat



containers are created from container images

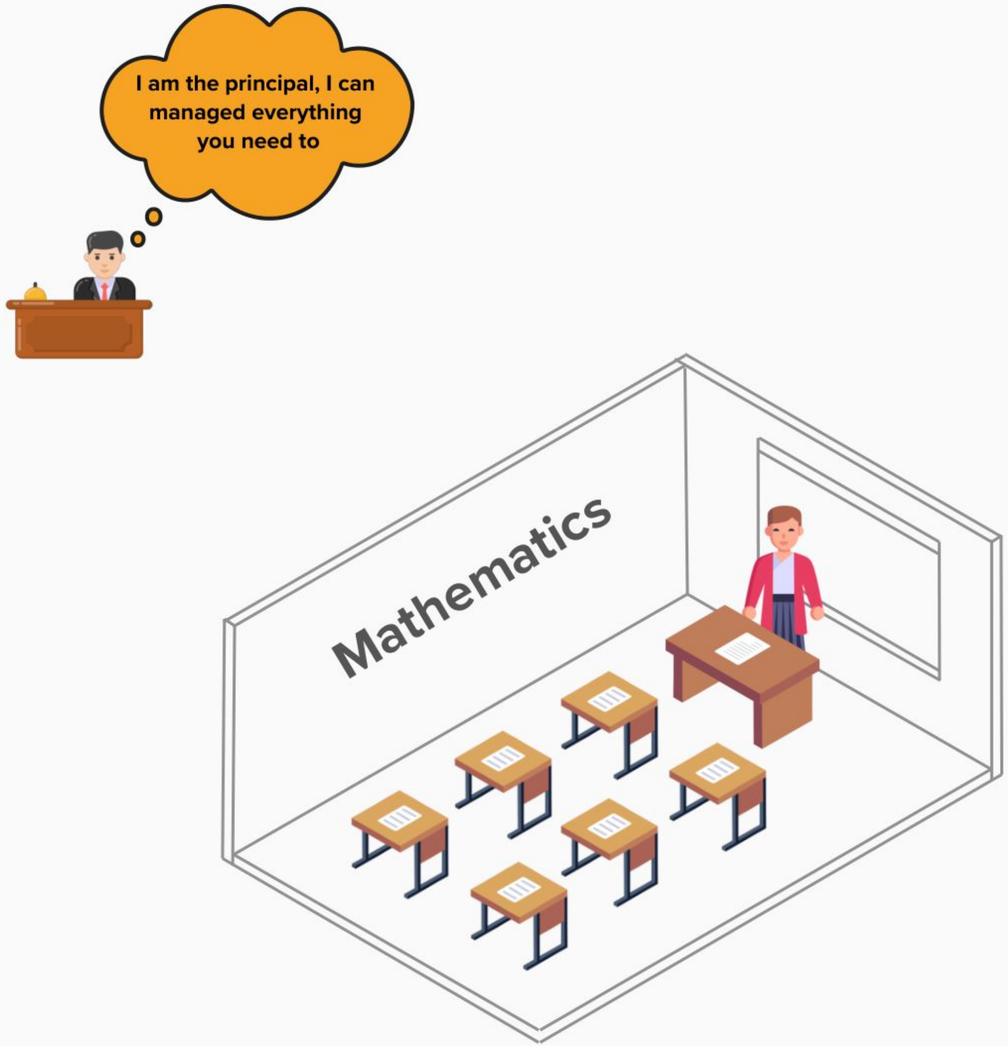
What is kubernetes?

containers are created from container images

Kubernetes is a portable, extensible, open source platform for managing containerized workloads and services that facilitate both declarative configuration and automation. It has a large, rapidly growing ecosystem. Kubernetes services, support, and tools are widely available.

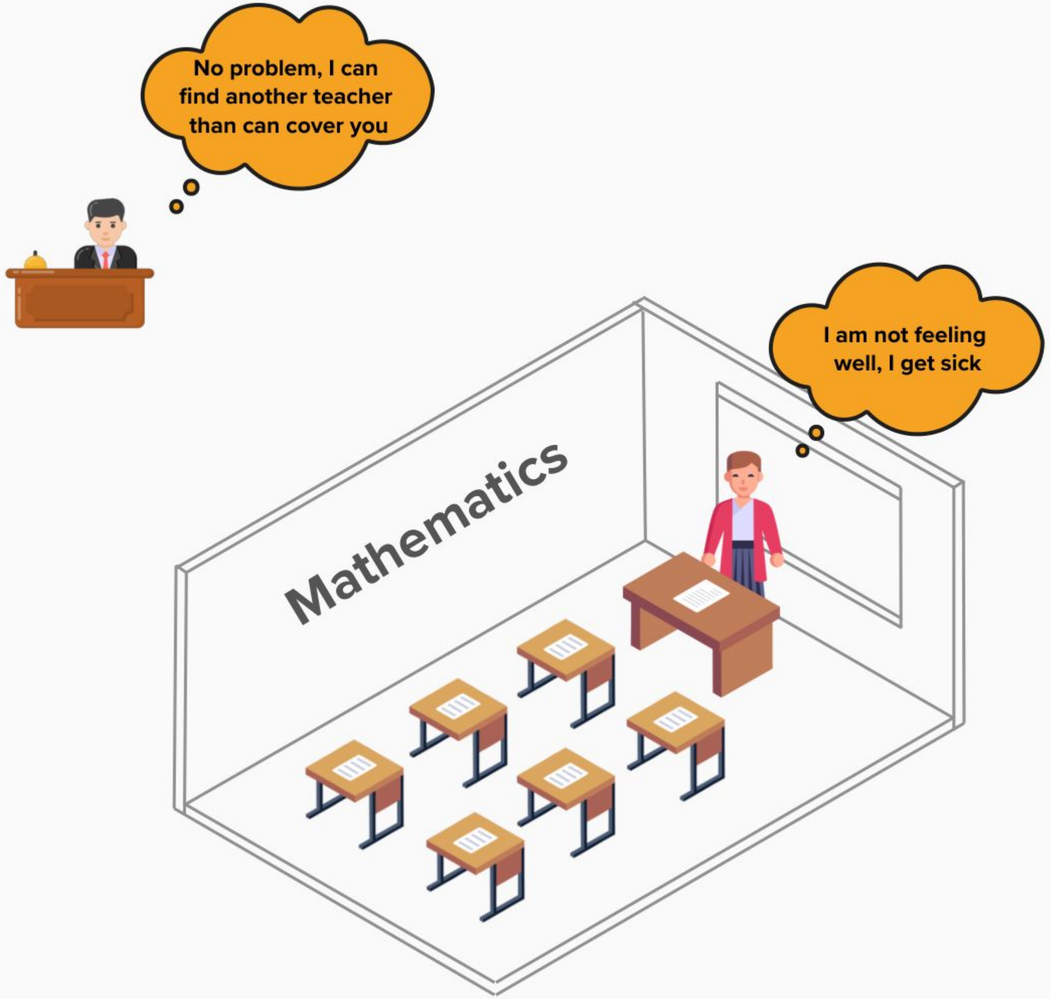
Let’s compare it with a real-life example

1. KUBERNETES



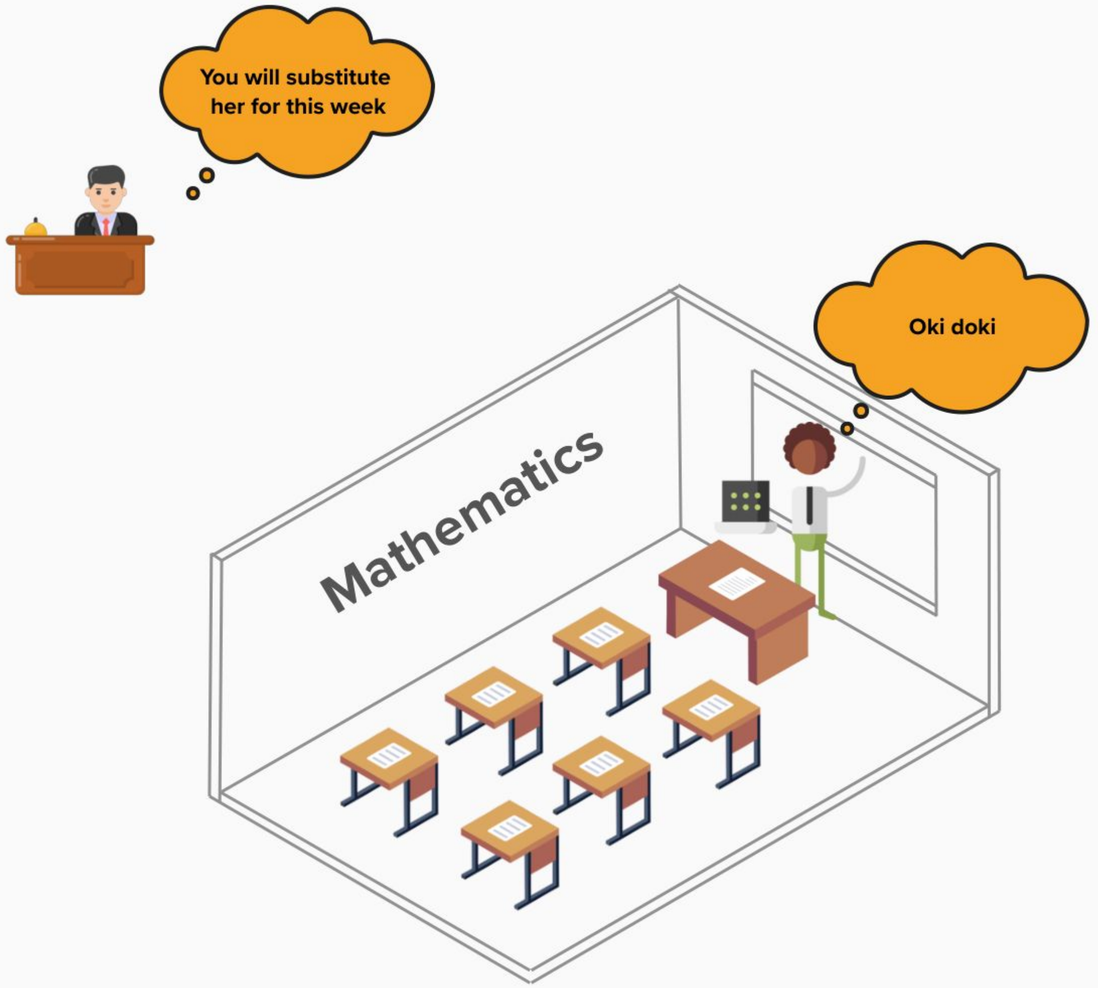
IBM Red Hat

1. KUBERNETES



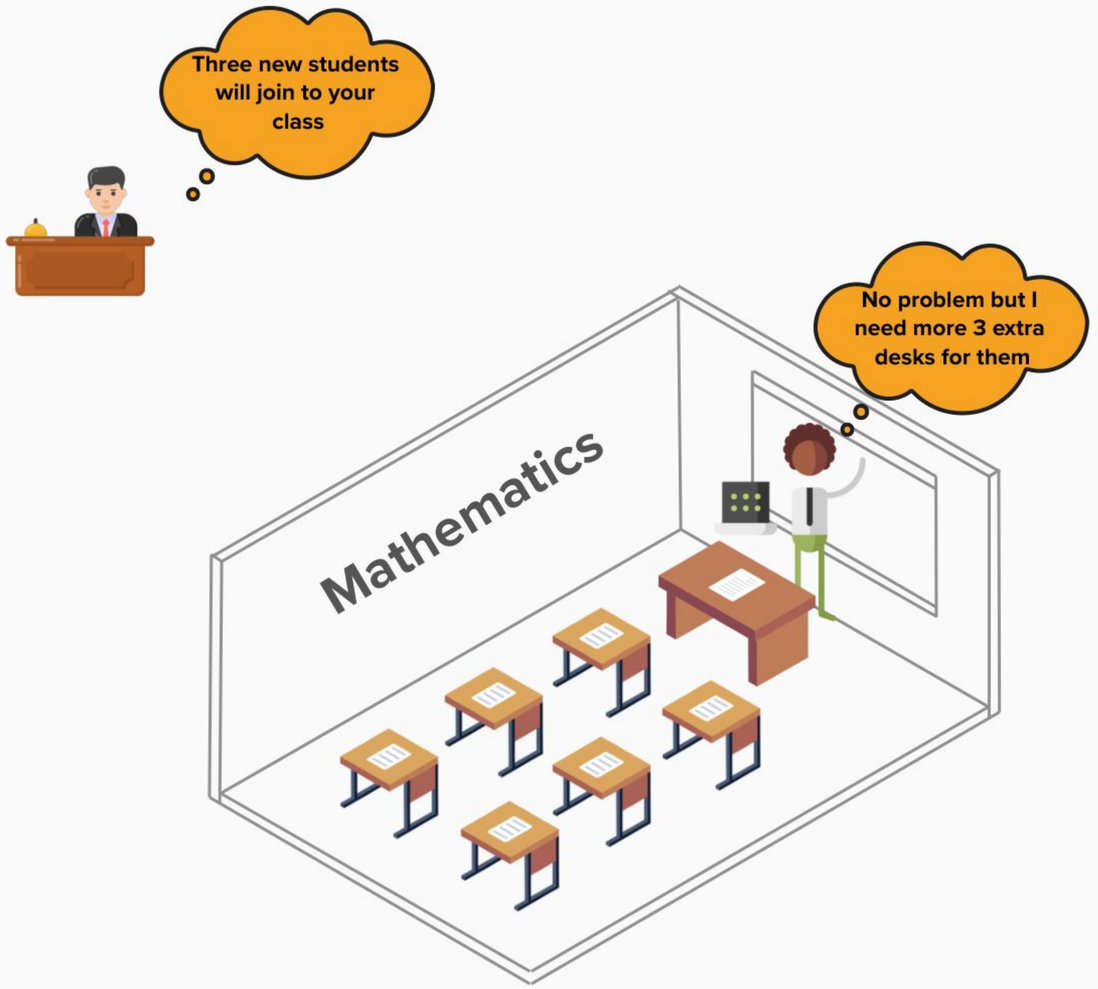
IBM Red Hat

1. KUBERNETES



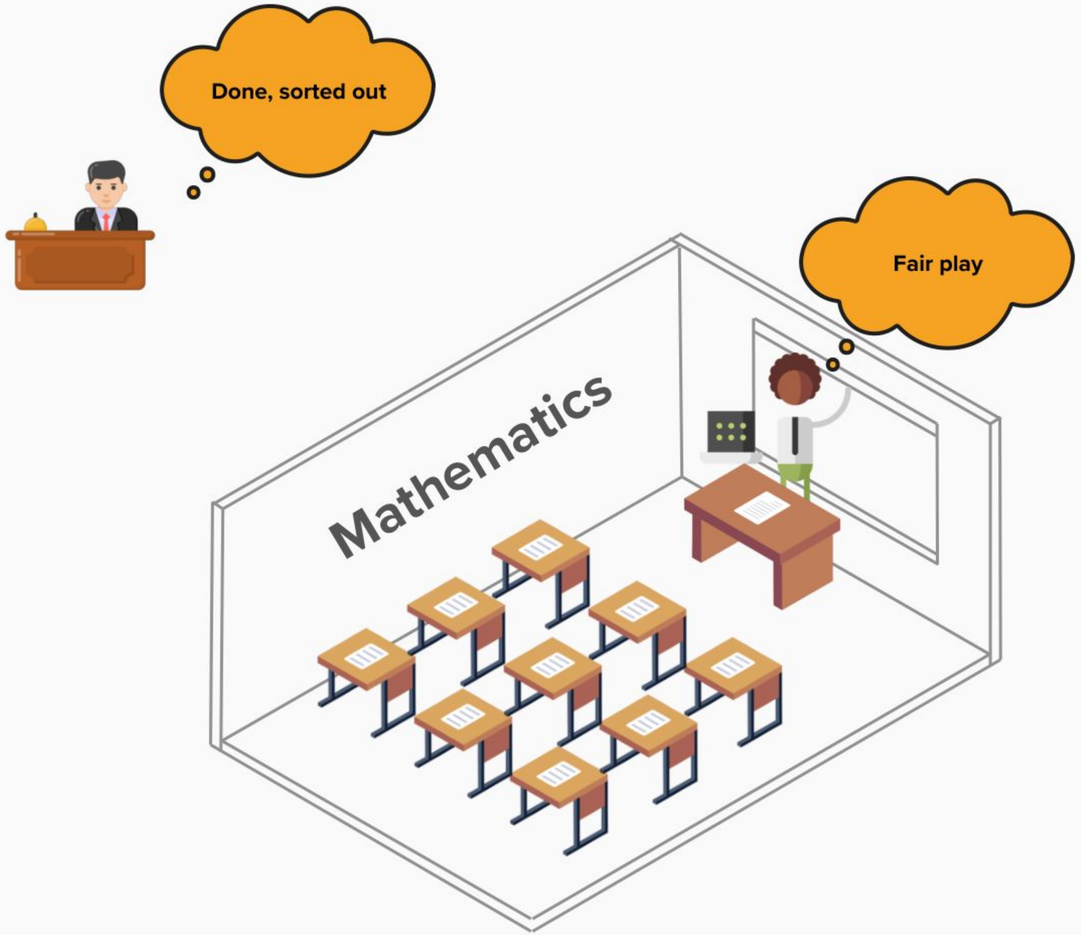
IBM Red Hat

1. KUBERNETES



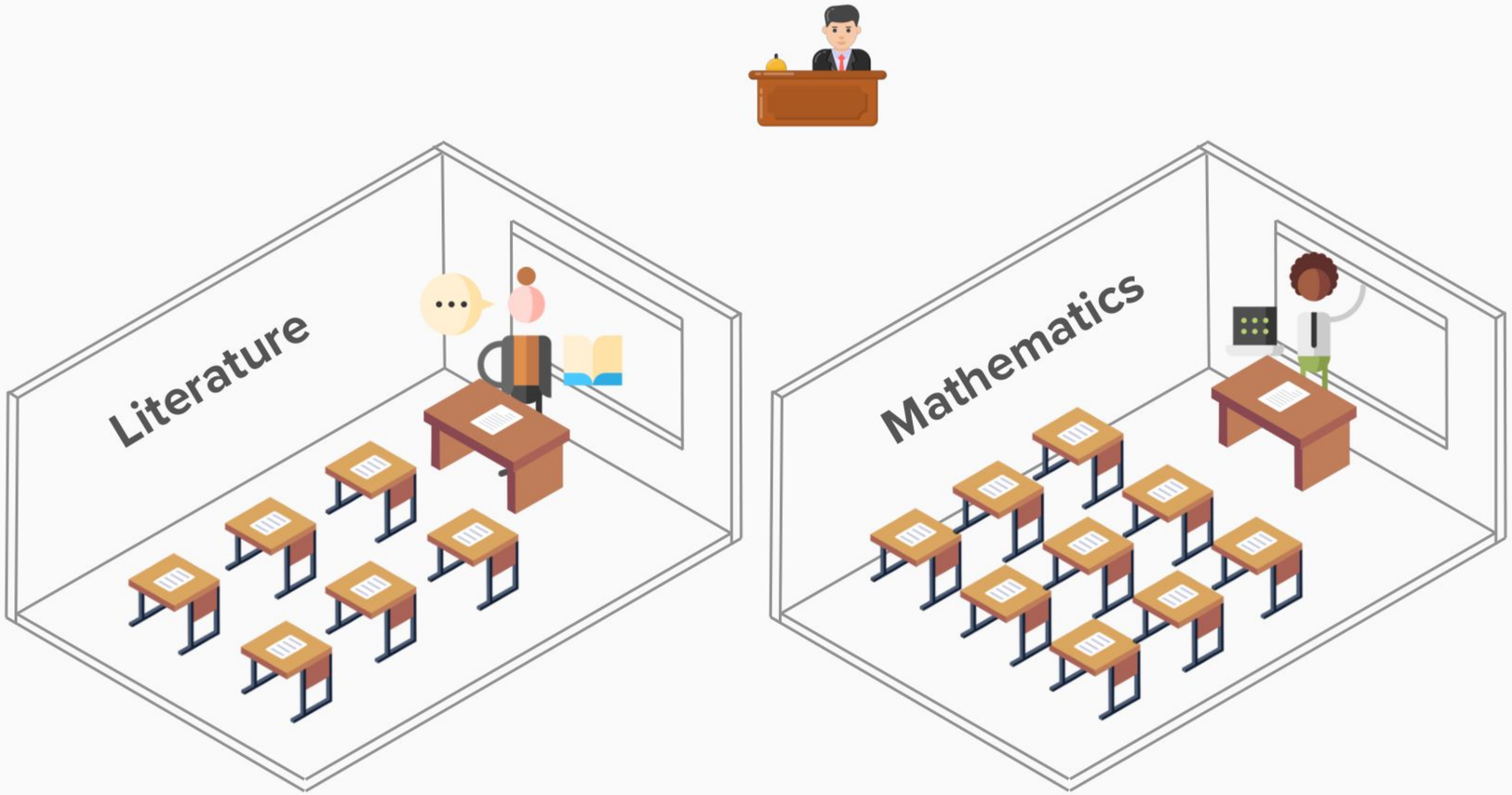
IBM Red Hat

1. KUBERNETES



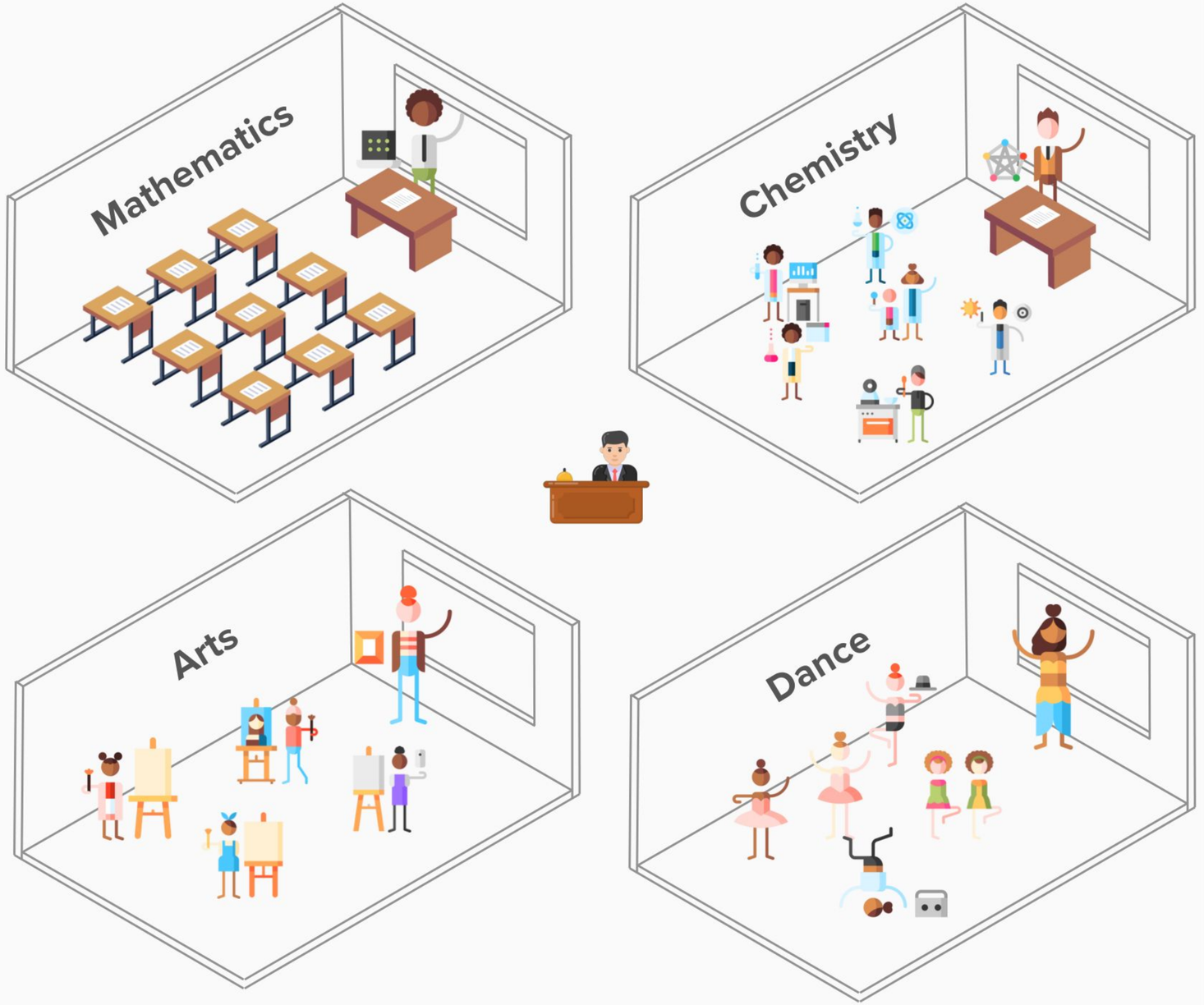
IBM Red Hat

1. KUBERNETES



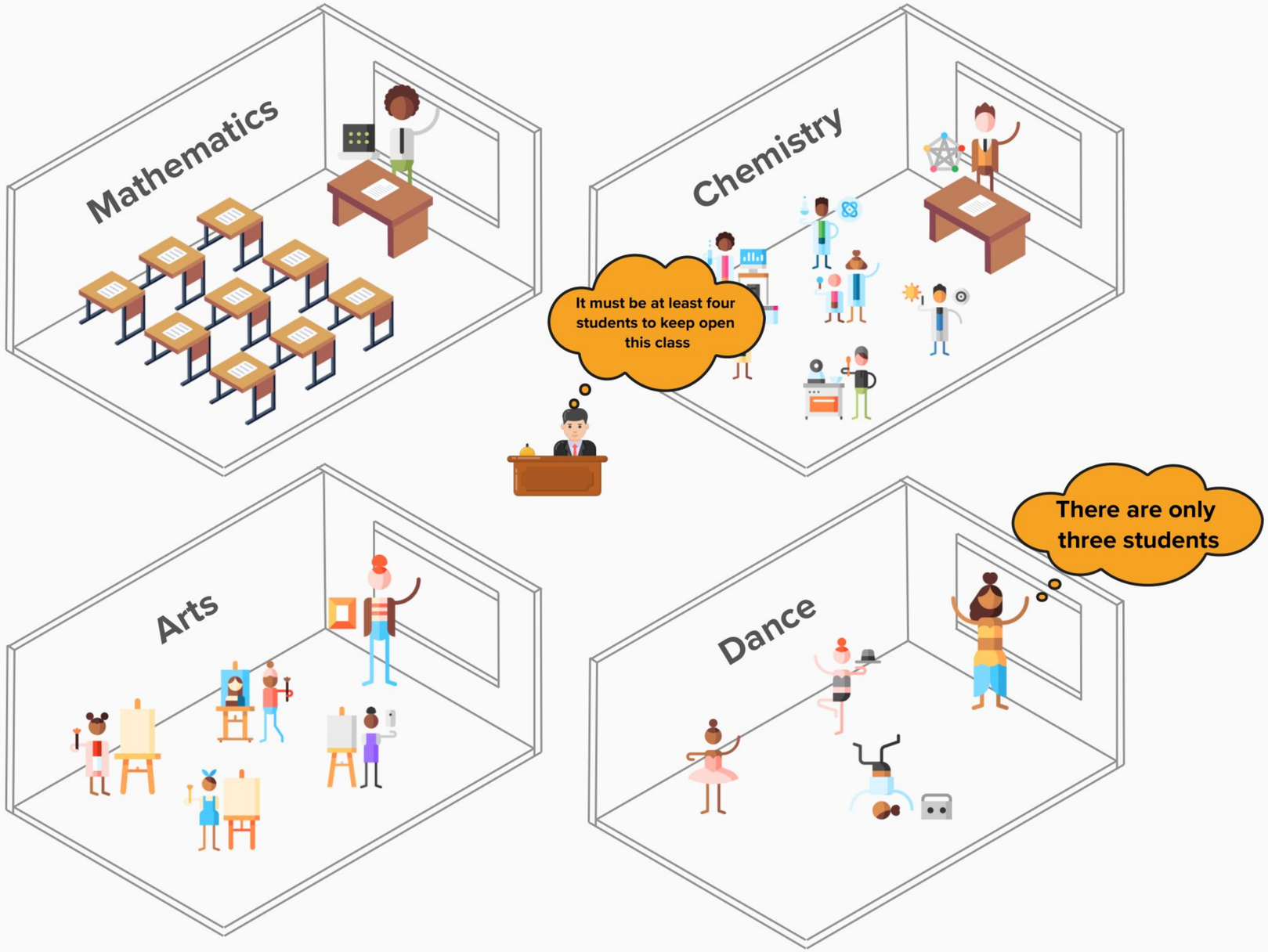
IBM Red Hat

1. KUBERNETES



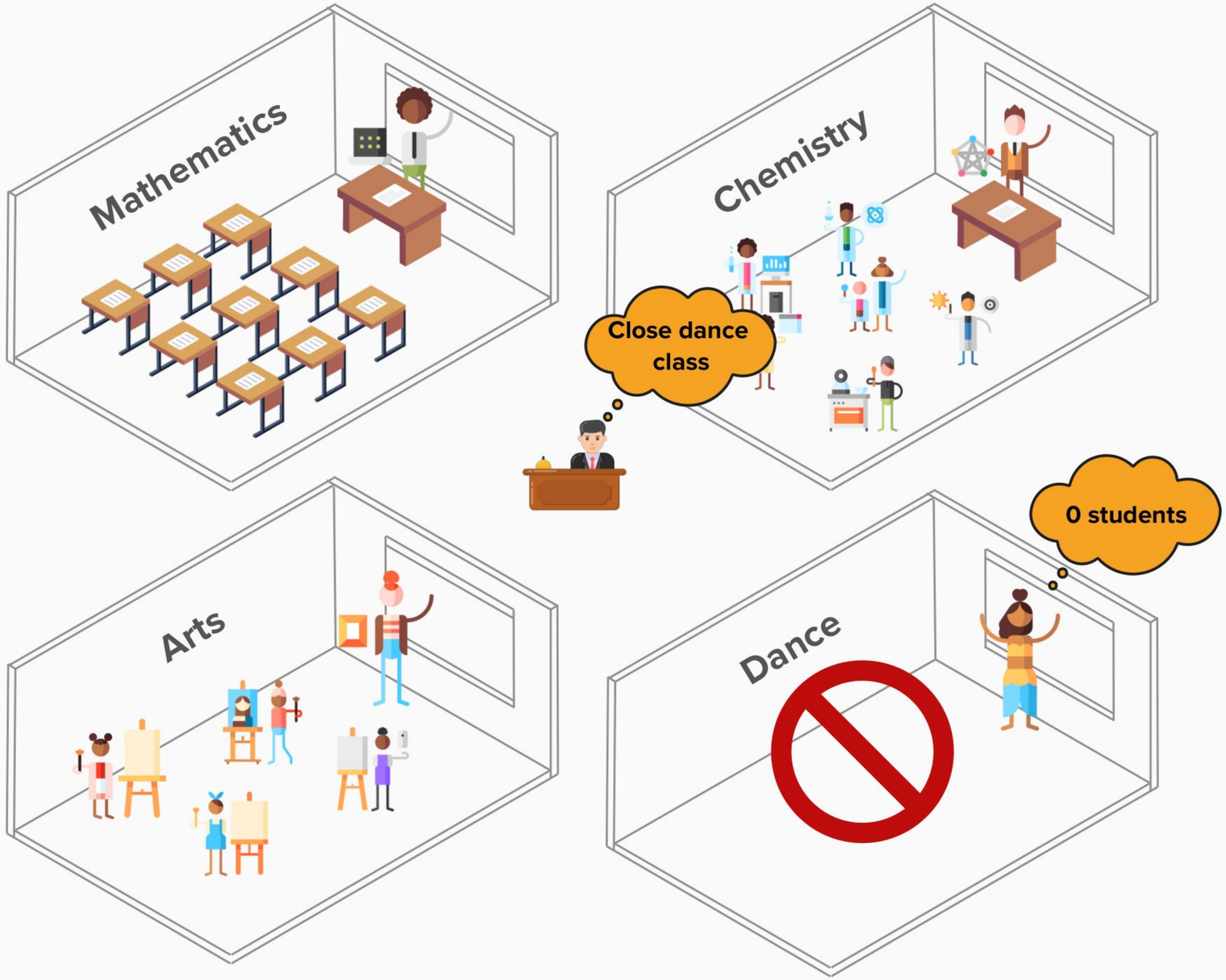
IBM Red Hat

1. KUBERNETES



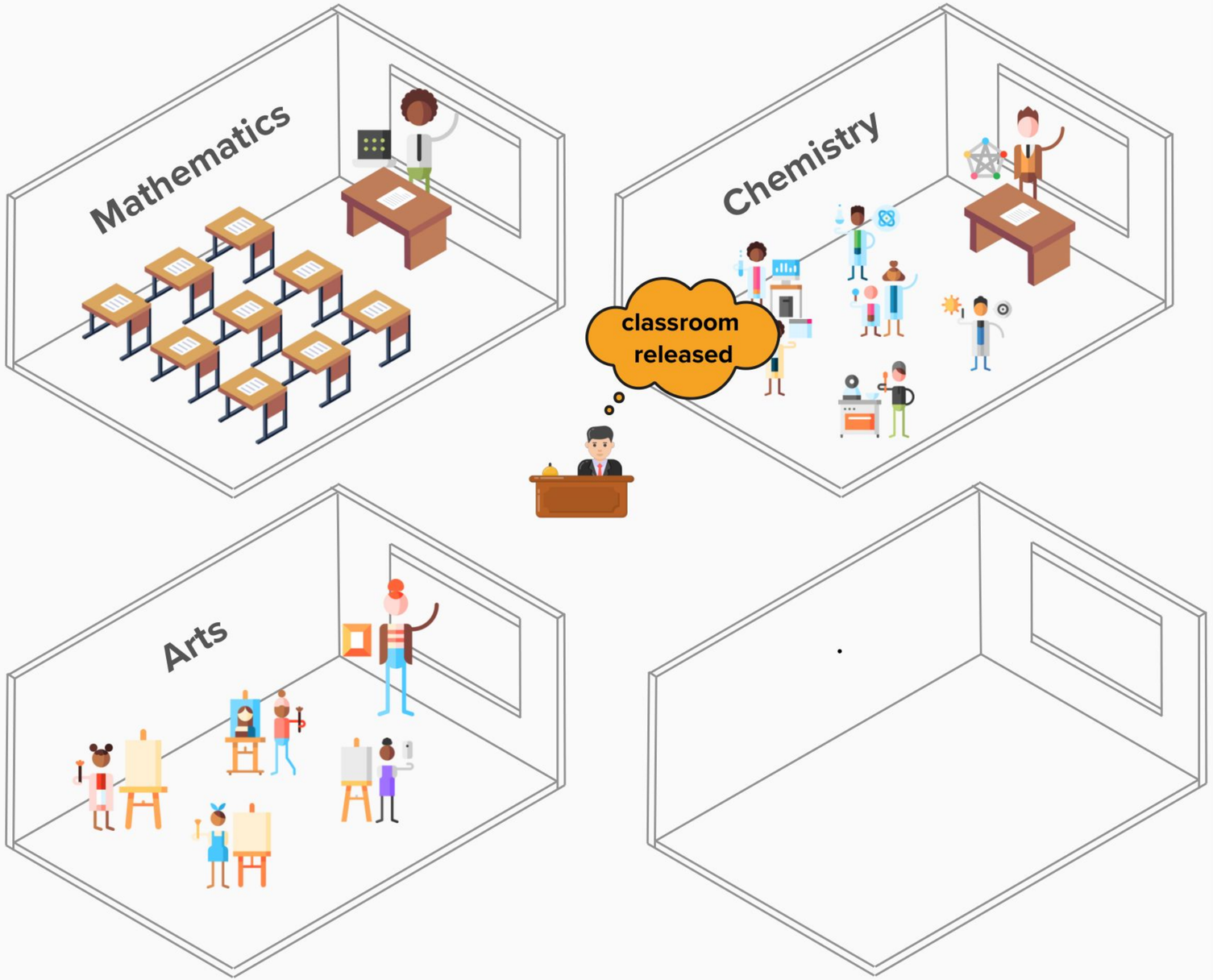
IBM Red Hat

1. KUBERNETES



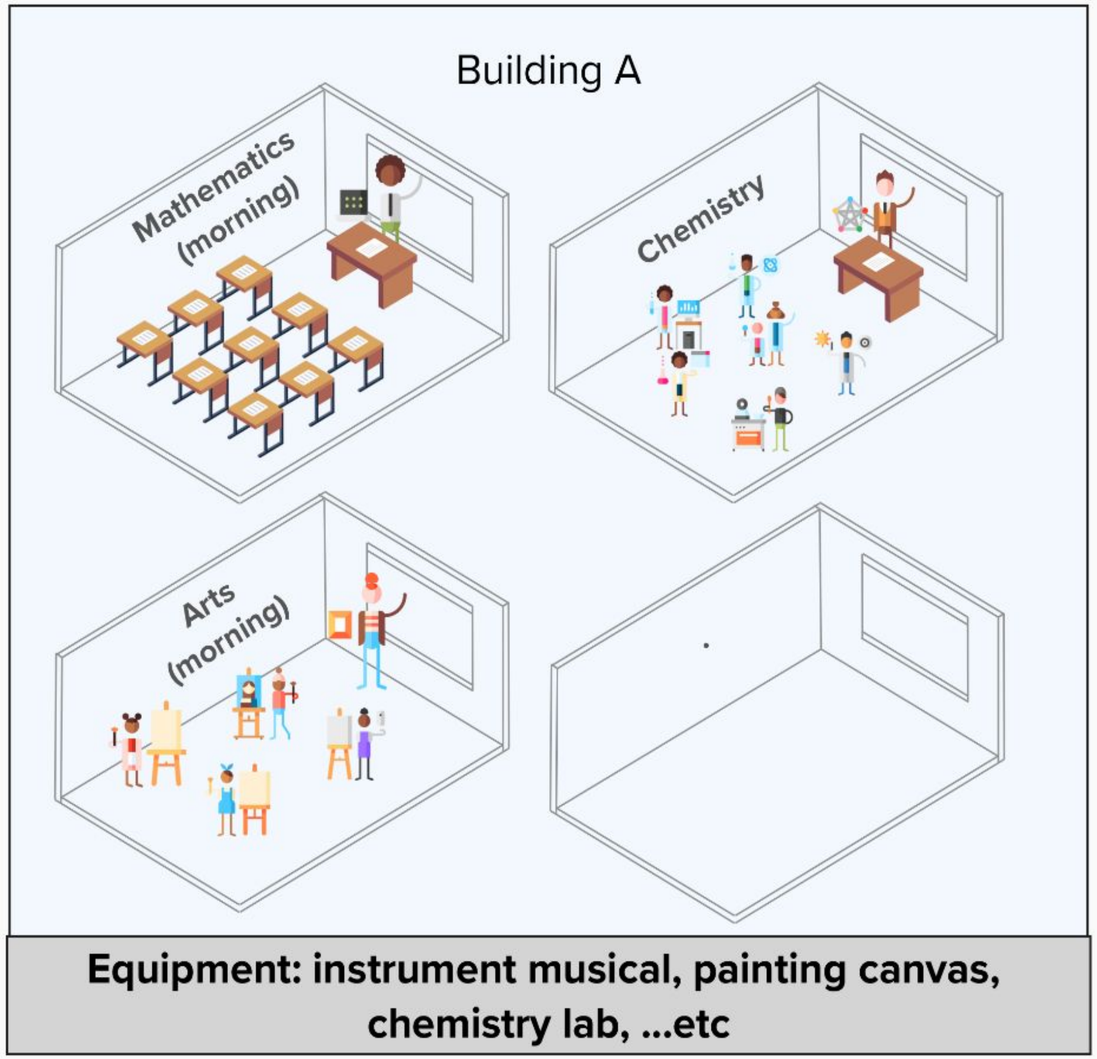
IBM Red Hat

1. KUBERNETES



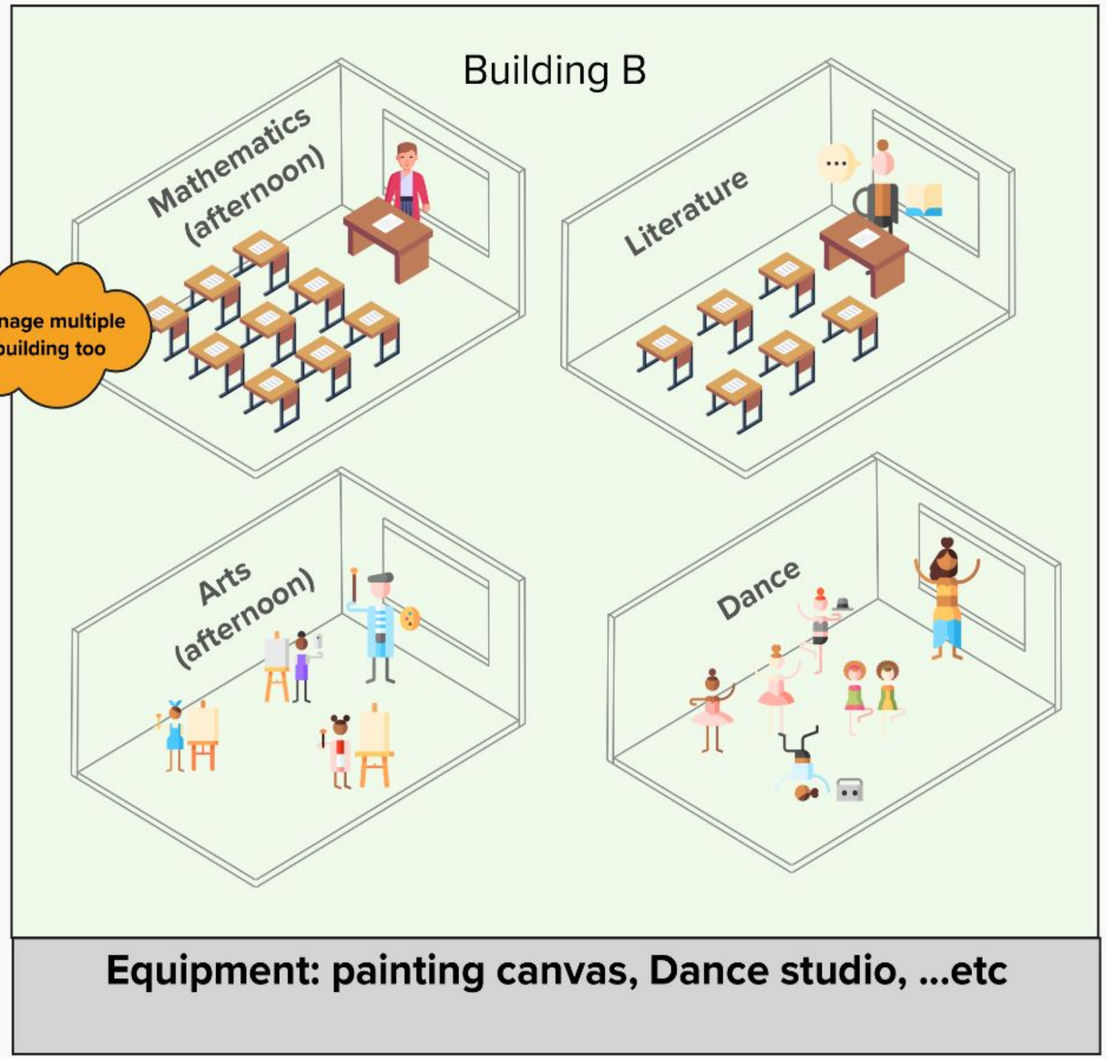
IBM Red Hat

1. KUBERNETES



Equipment: instrument musical, painting canvas, chemistry lab, ...etc

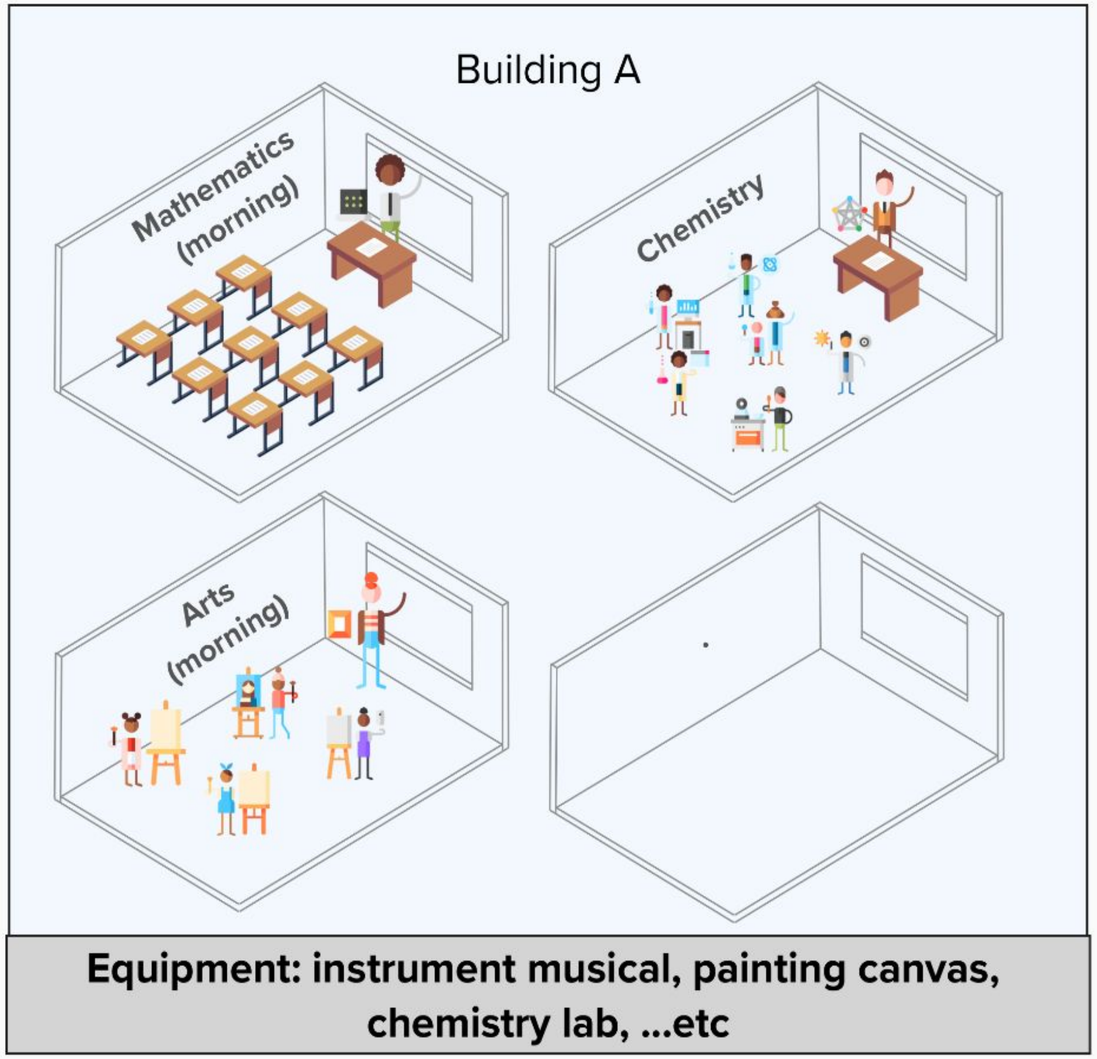




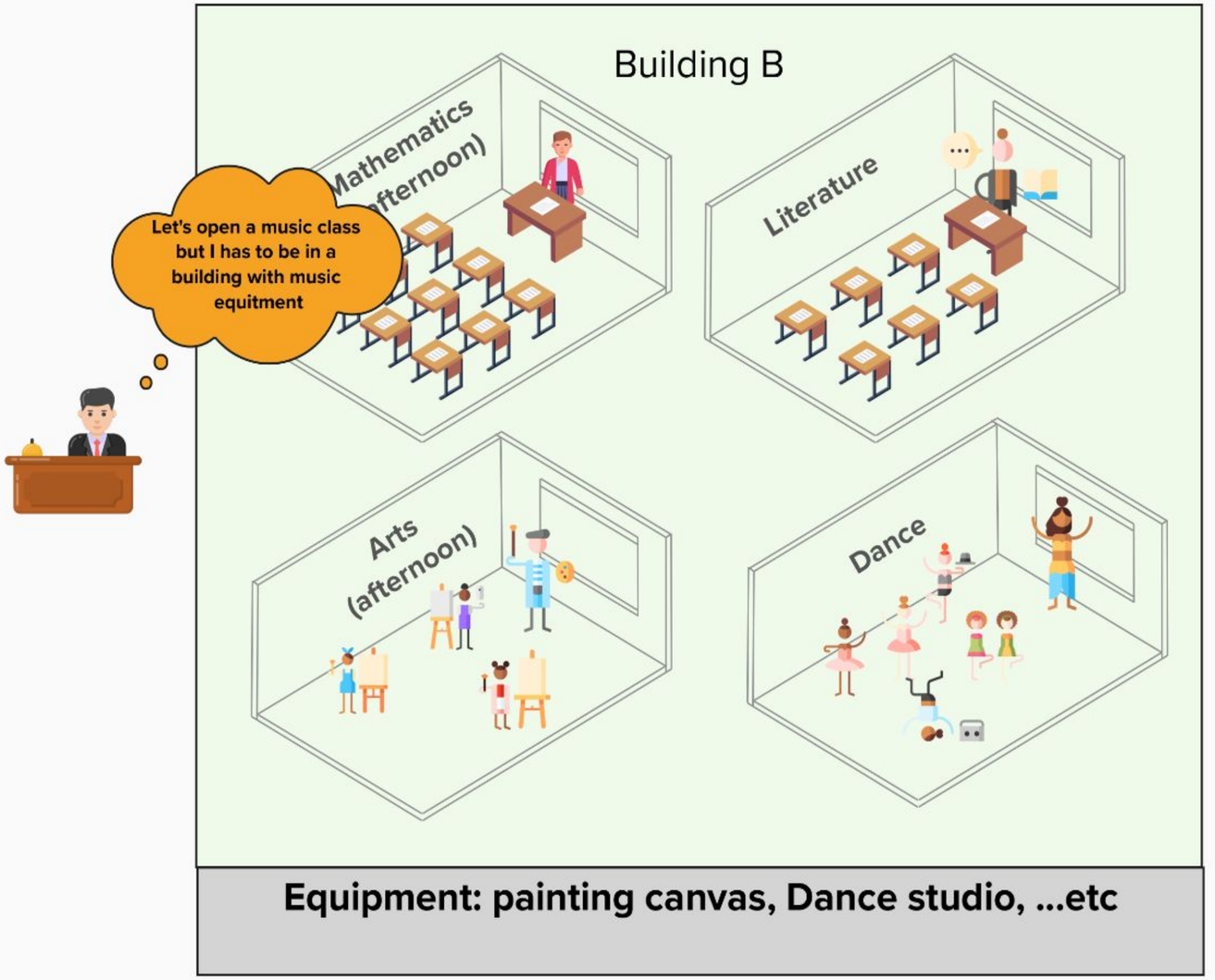
Equipment: painting canvas, Dance studio, ...etc

IBM Red Hat

1. KUBERNETES



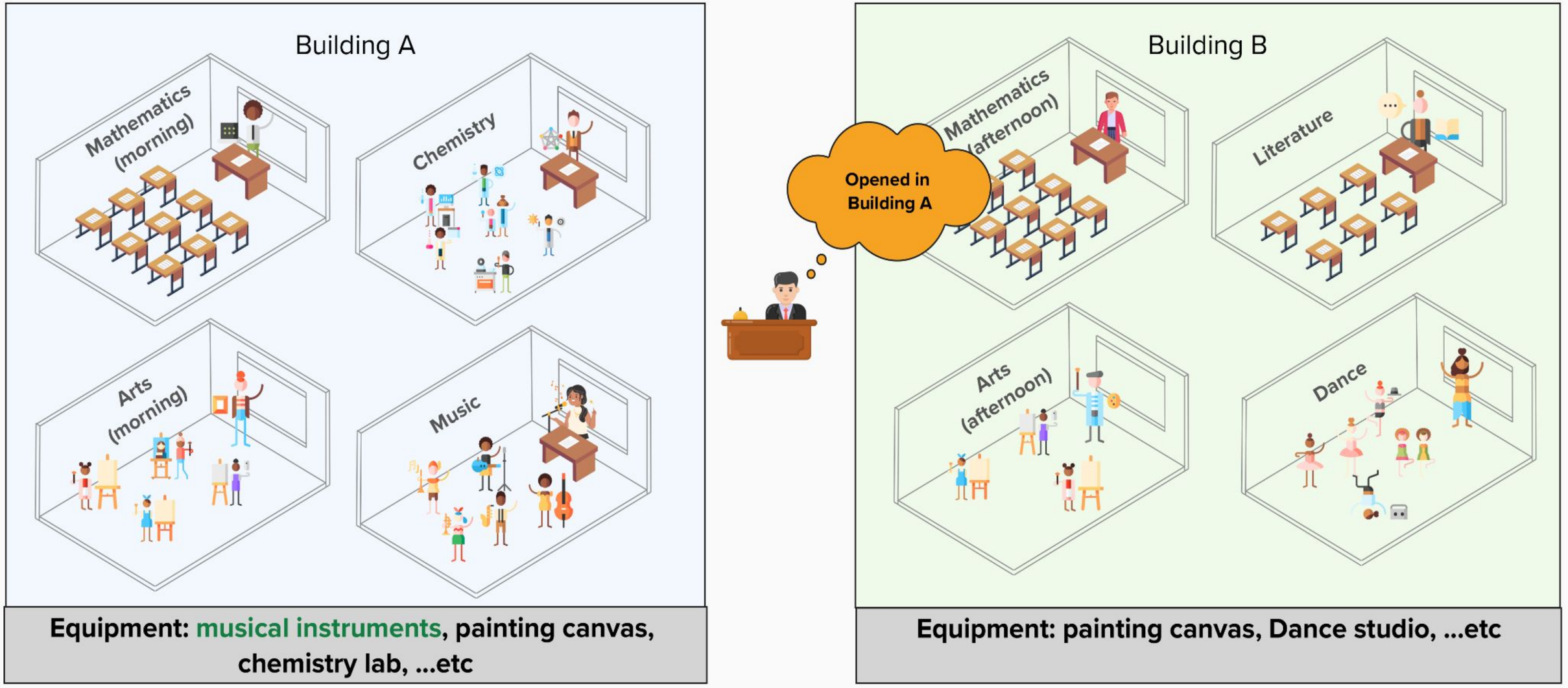
Equipment: instrument musical, painting canvas, chemistry lab, ...etc



Equipment: painting canvas, Dance studio, ...etc

IBM Red Hat

1. KUBERNETES

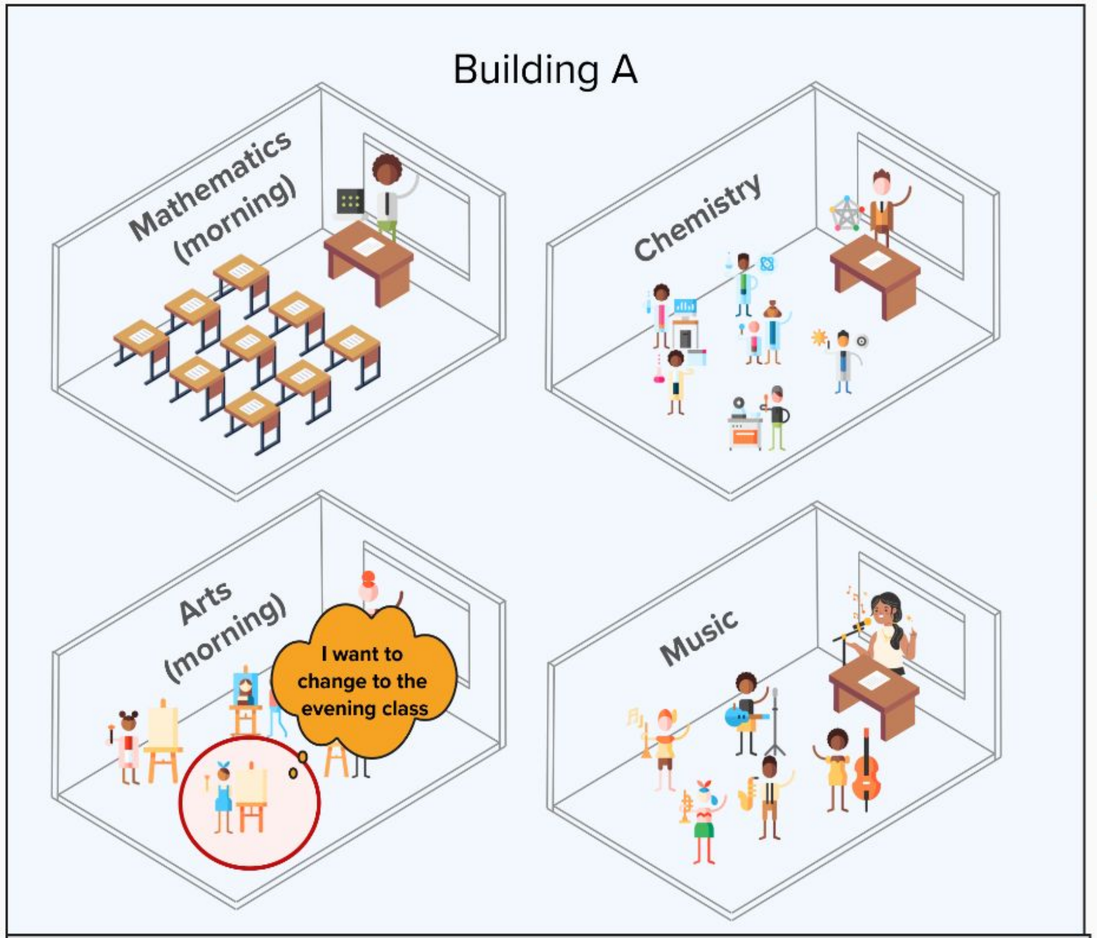


Equipment: musical instruments, painting canvas, chemistry lab, ...etc

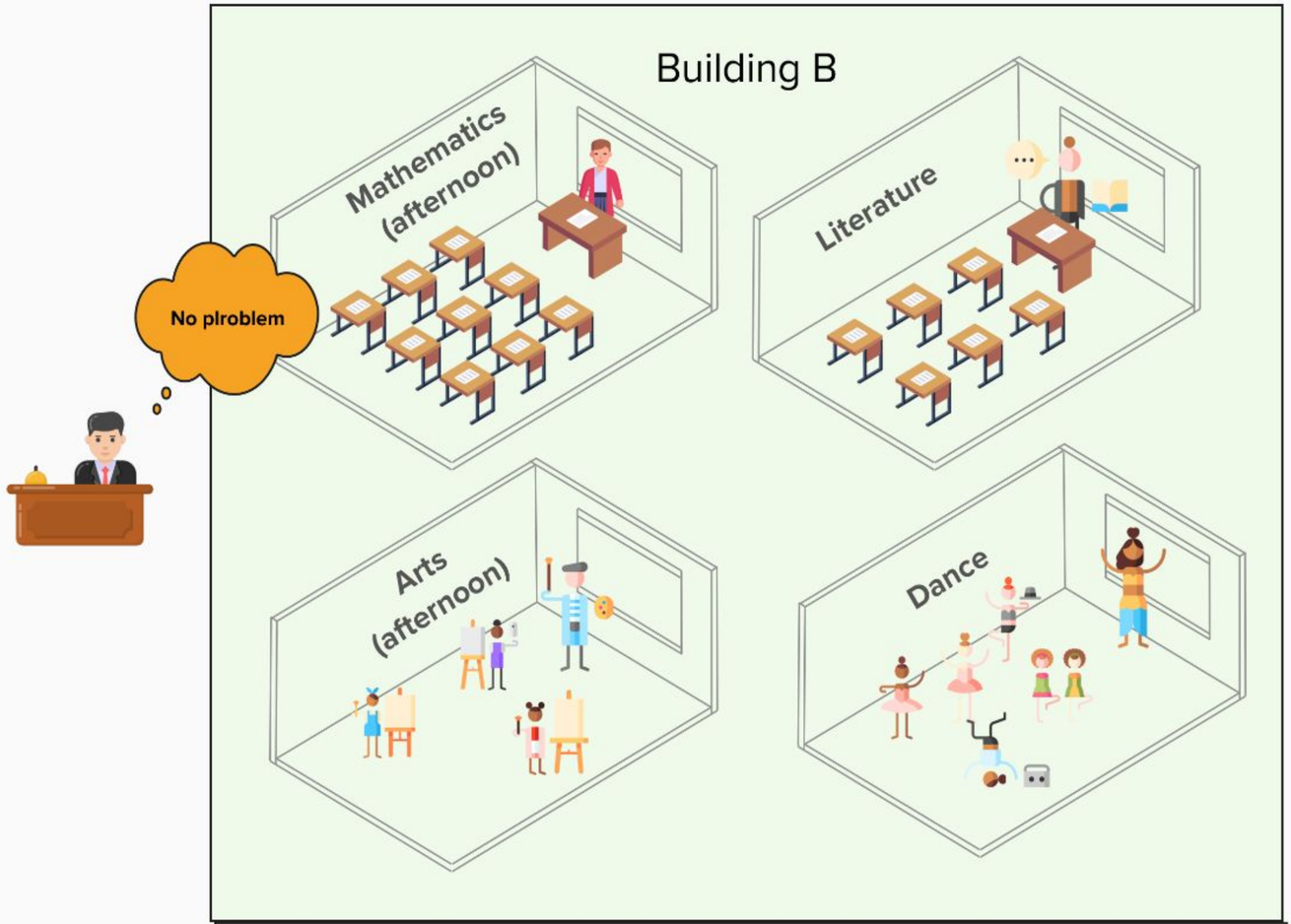
Equipment: painting canvas, Dance studio, ...etc

IBM Red Hat

1. KUBERNETES



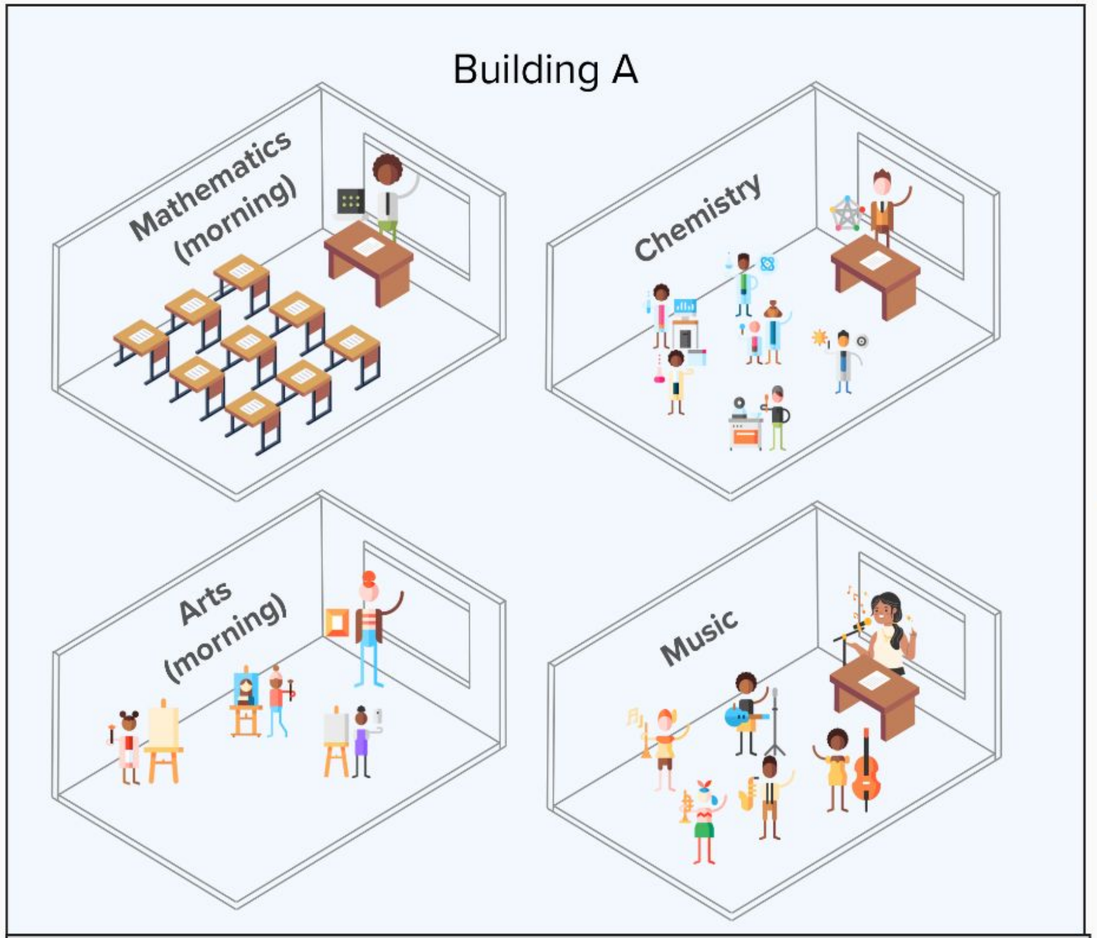
Equipment: instrument musical, painting canvas, chemistry lab, ...etc



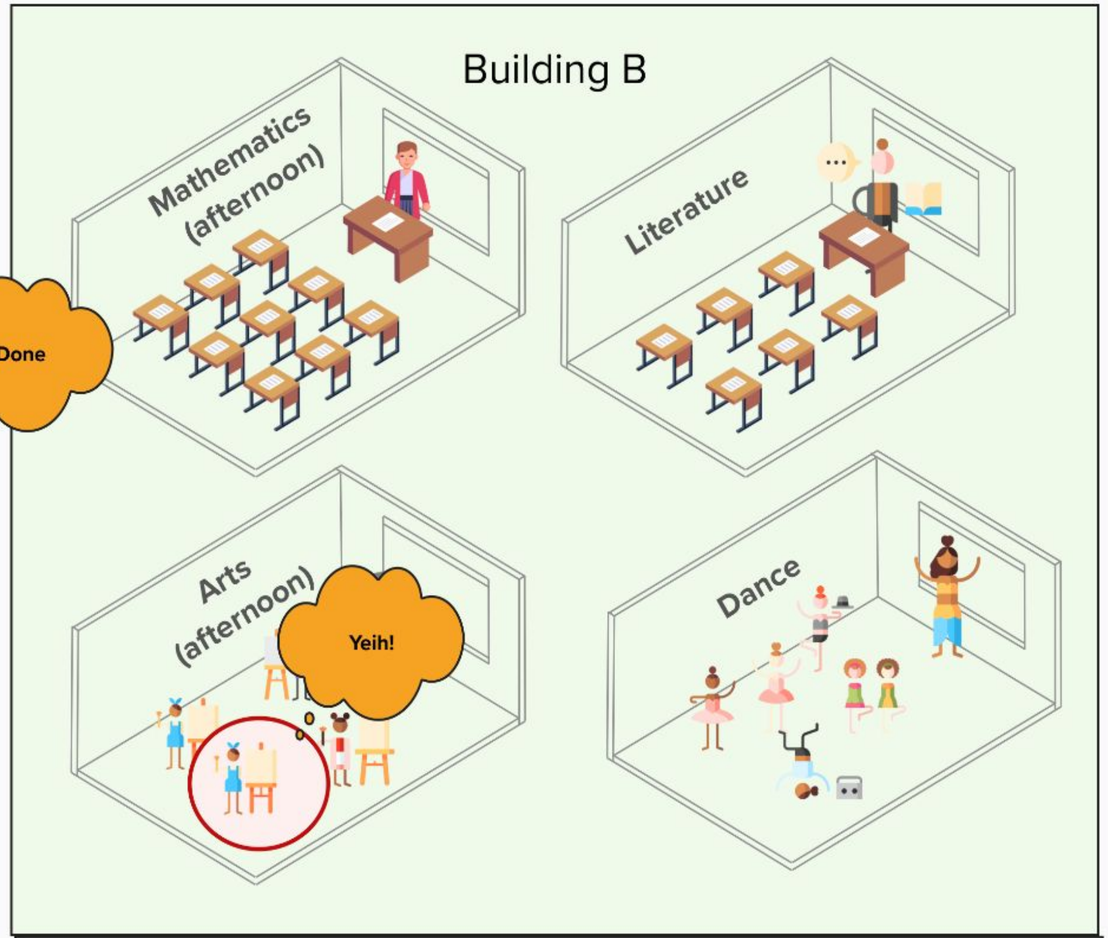
Equipment: painting canvas, Dance studio, ...etc

IBM Red Hat

1. KUBERNETES



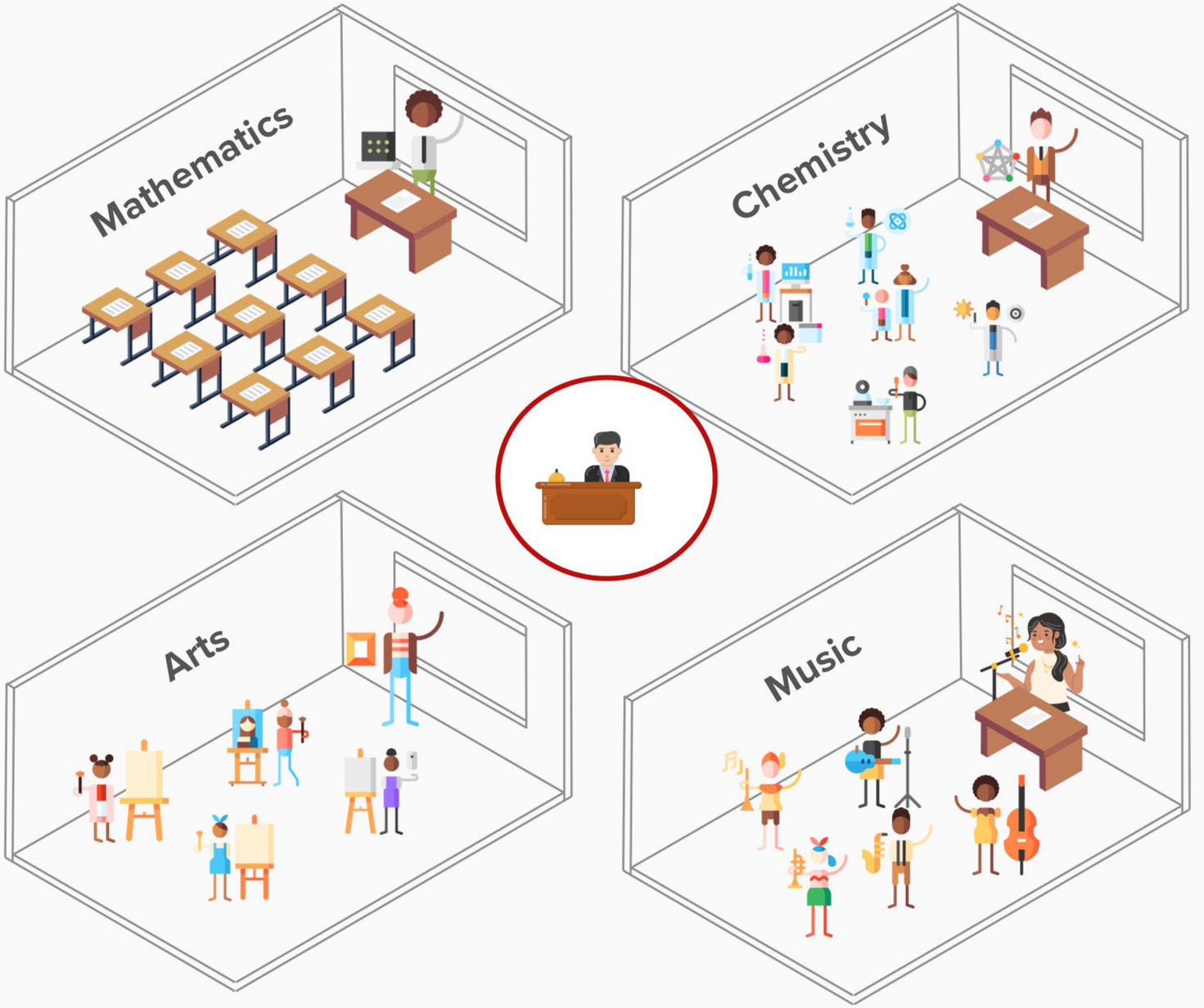
Equipment: instrument musical, painting canvas, chemistry lab, ...etc



Equipment: painting canvas, Dance studio, ...etc

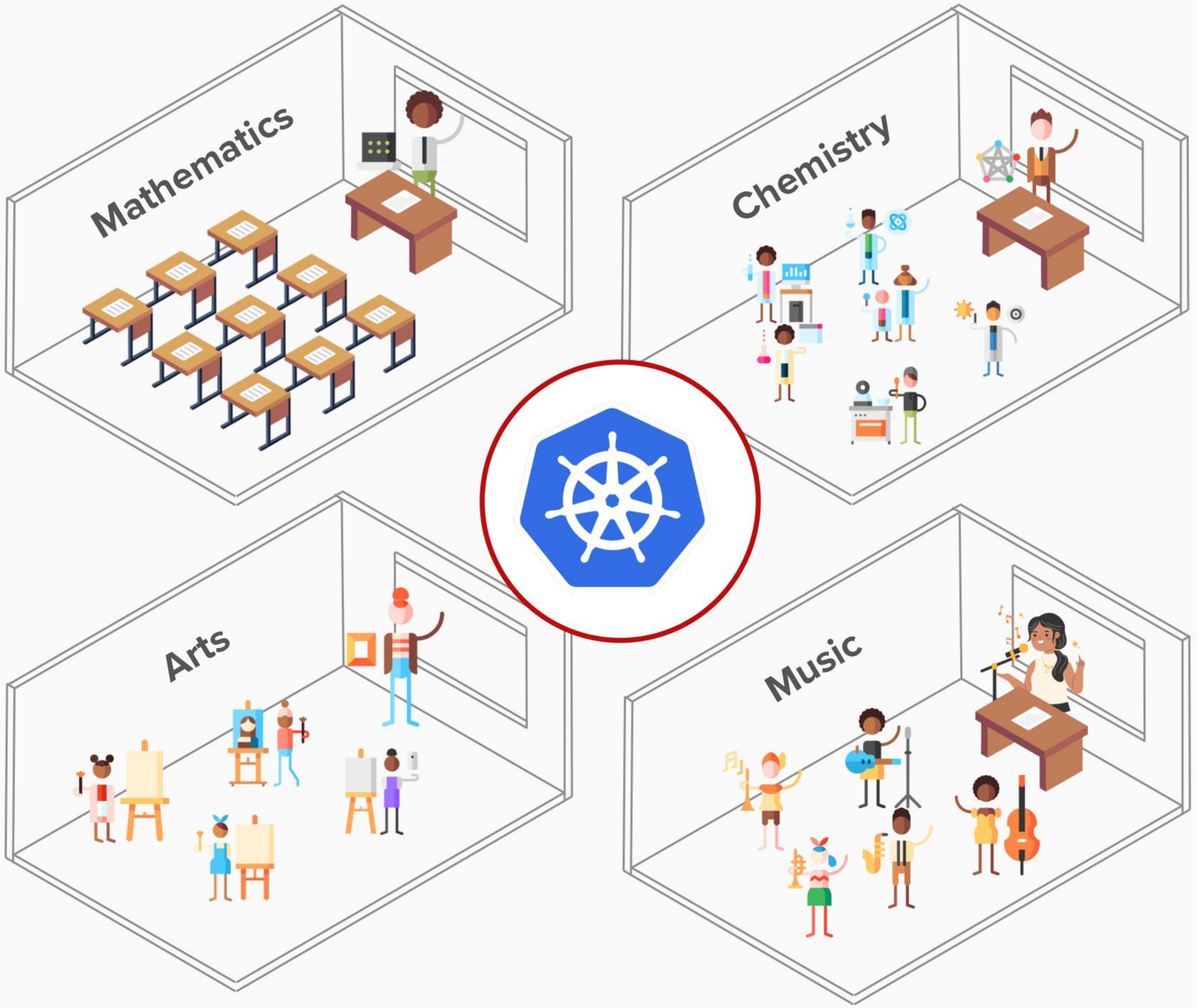
IBM Red Hat

1. KUBERNETES



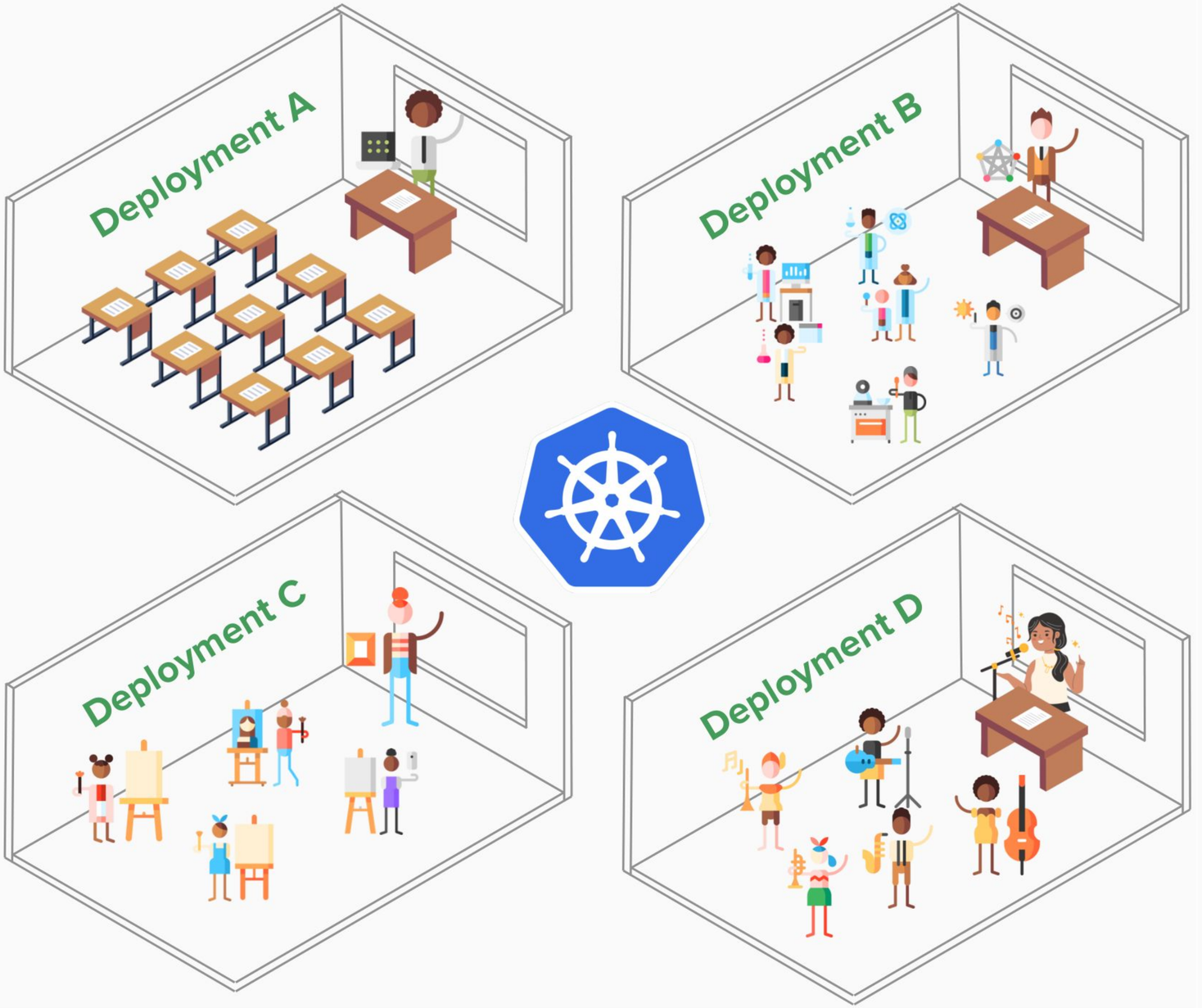
IBM Red Hat

1. KUBERNETES



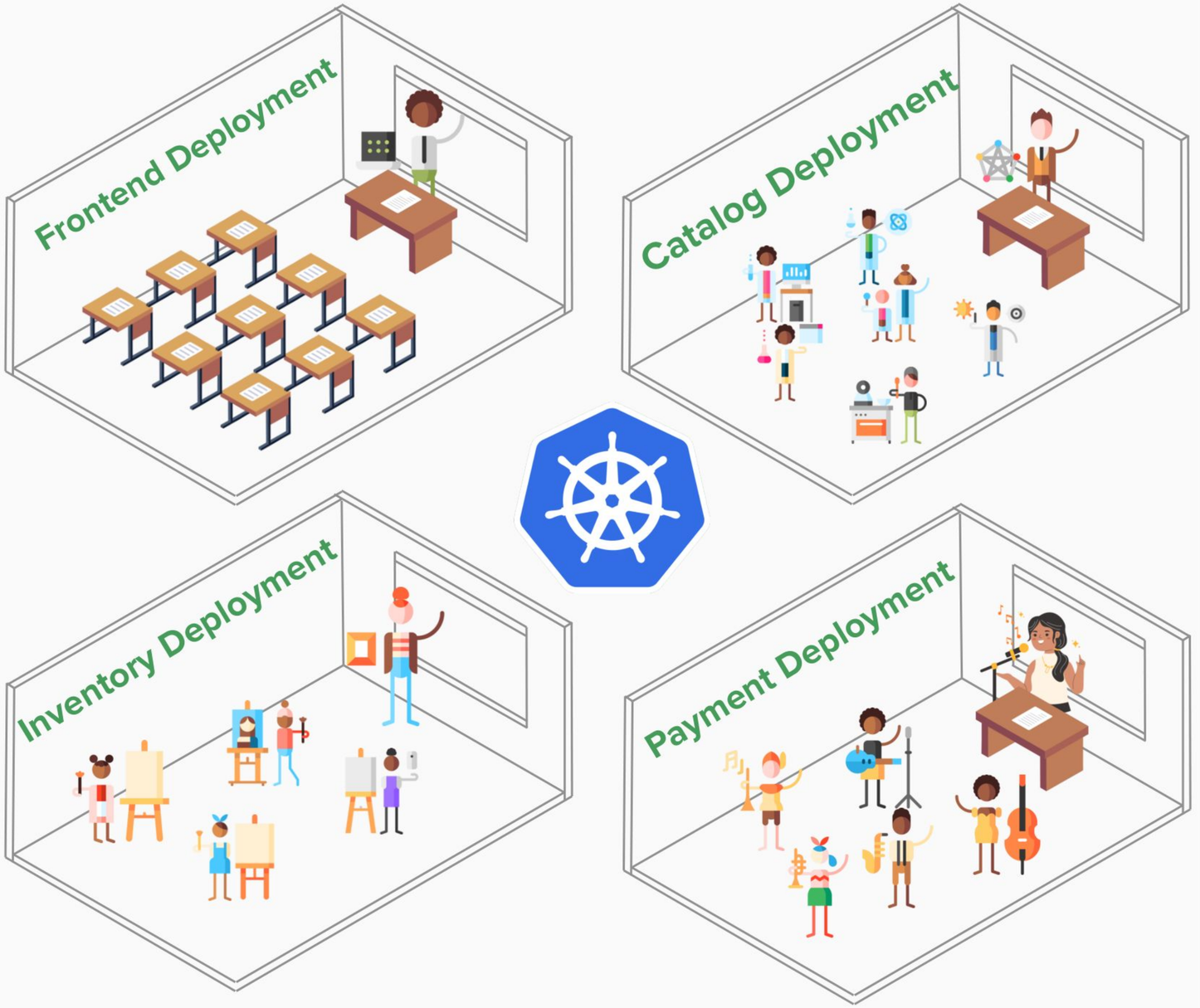
IBM Red Hat

1. KUBERNETES



IBM Red Hat

1. KUBERNETES

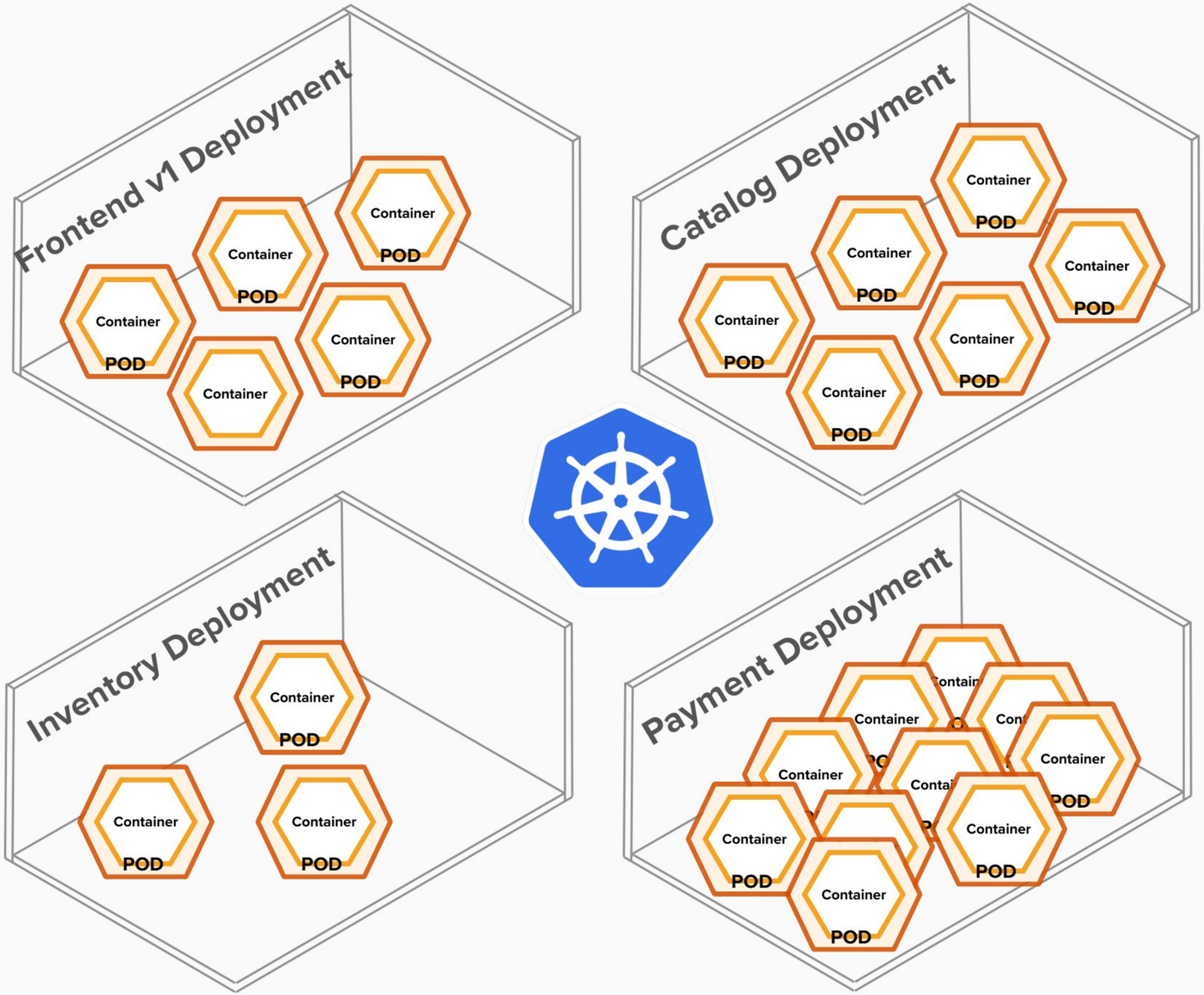


IBM Red Hat

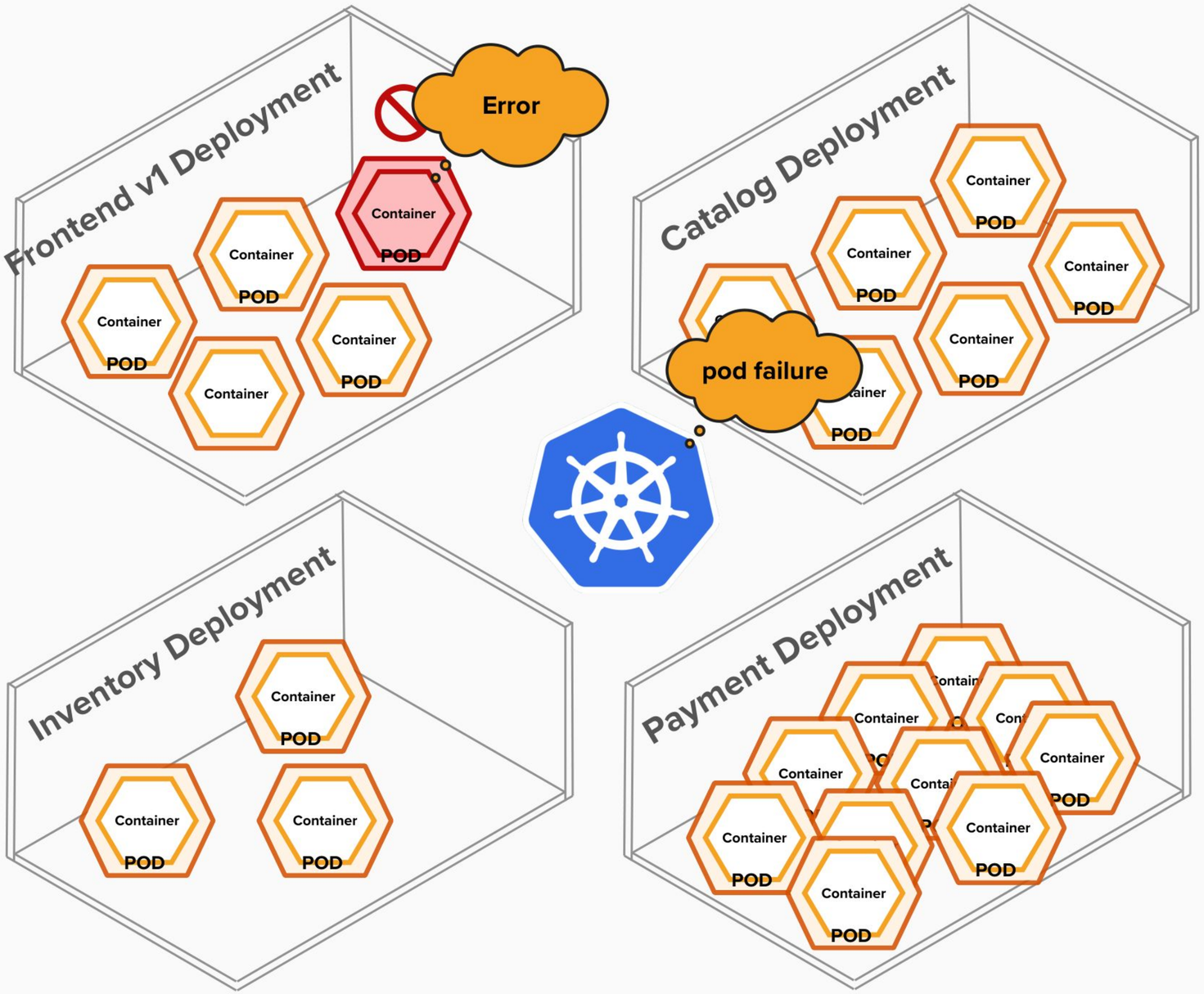
1. KUBERNETES



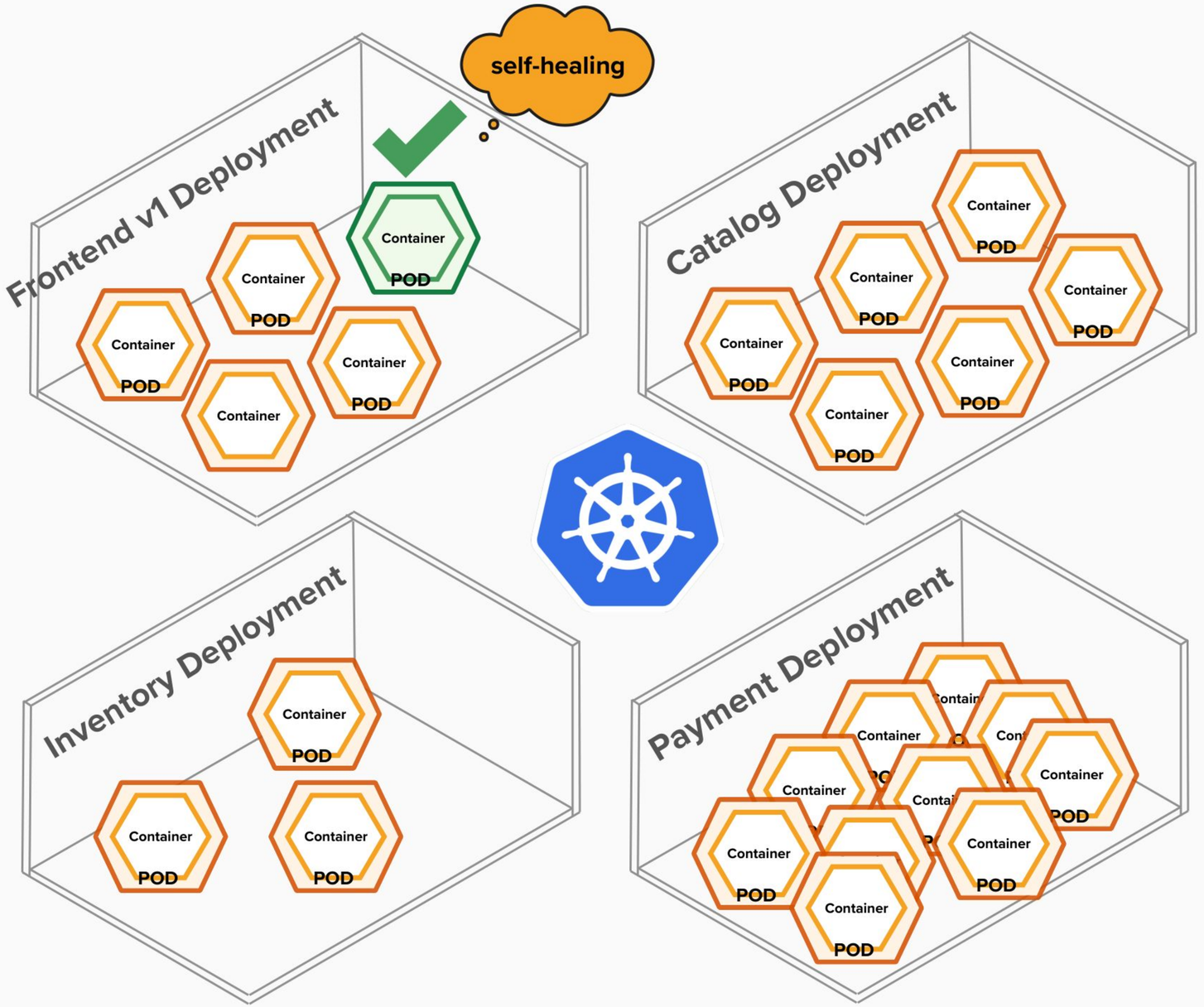
1. KUBERNETES



1. KUBERNETES

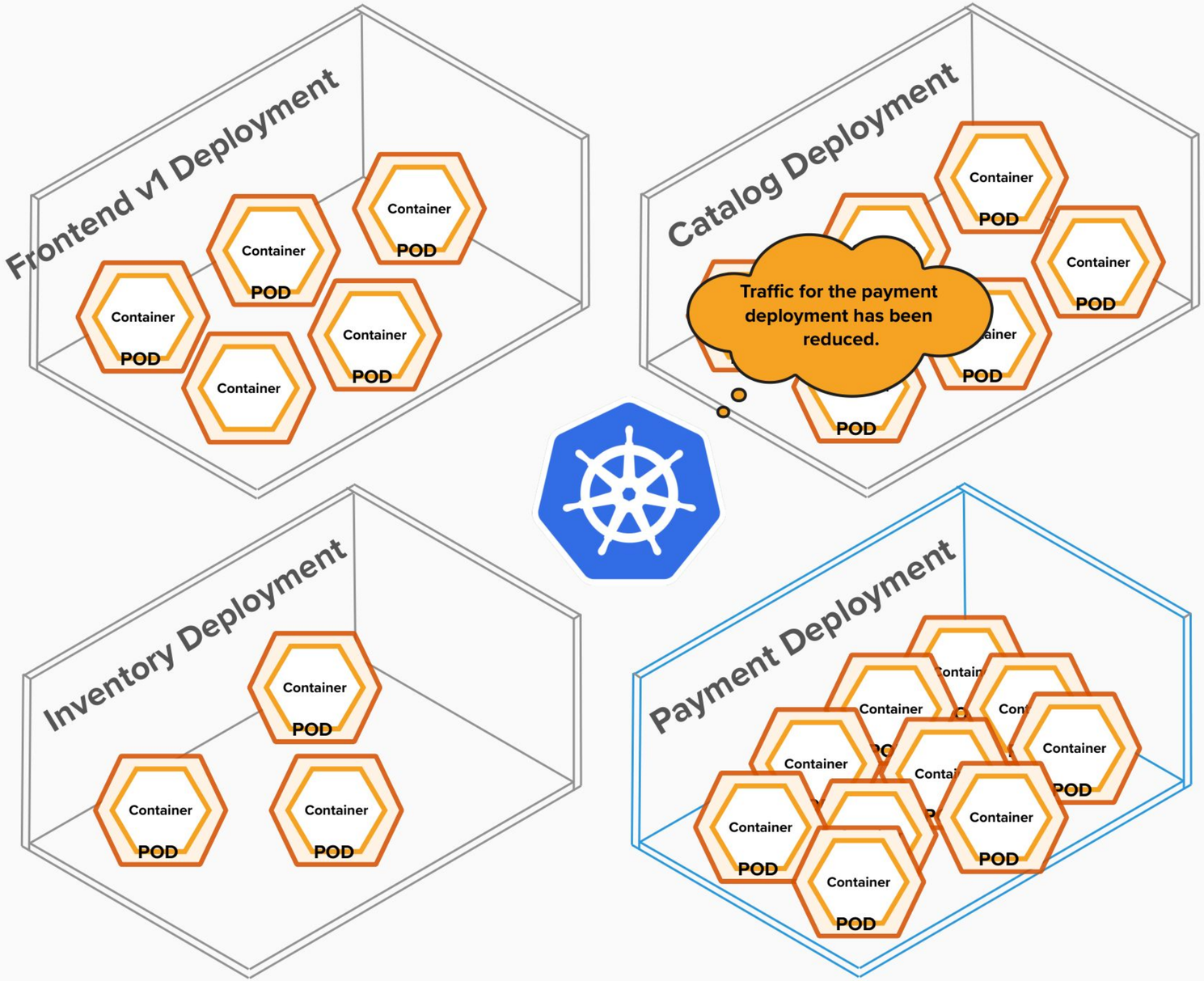


1. KUBERNETES

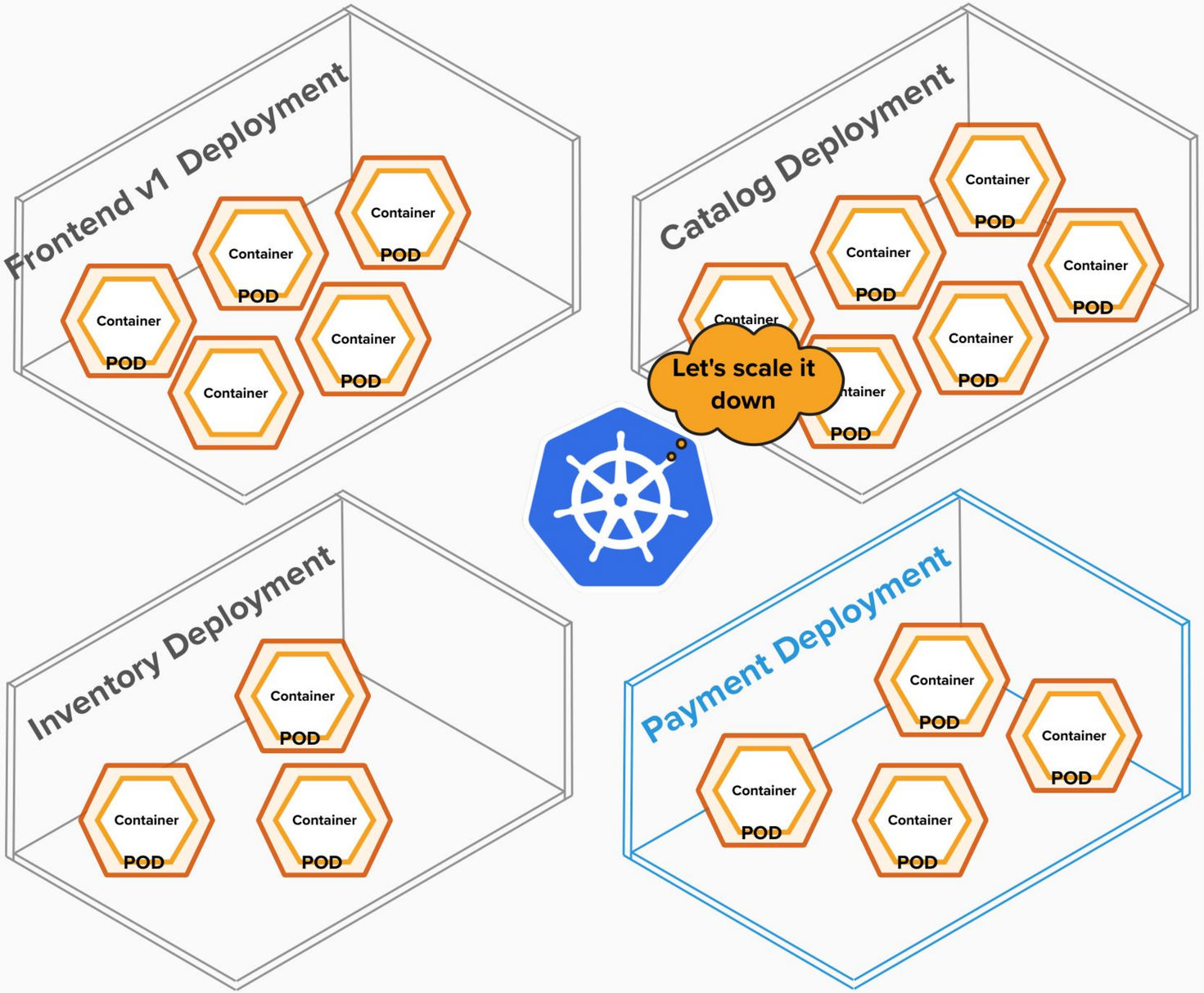


IBM Red Hat

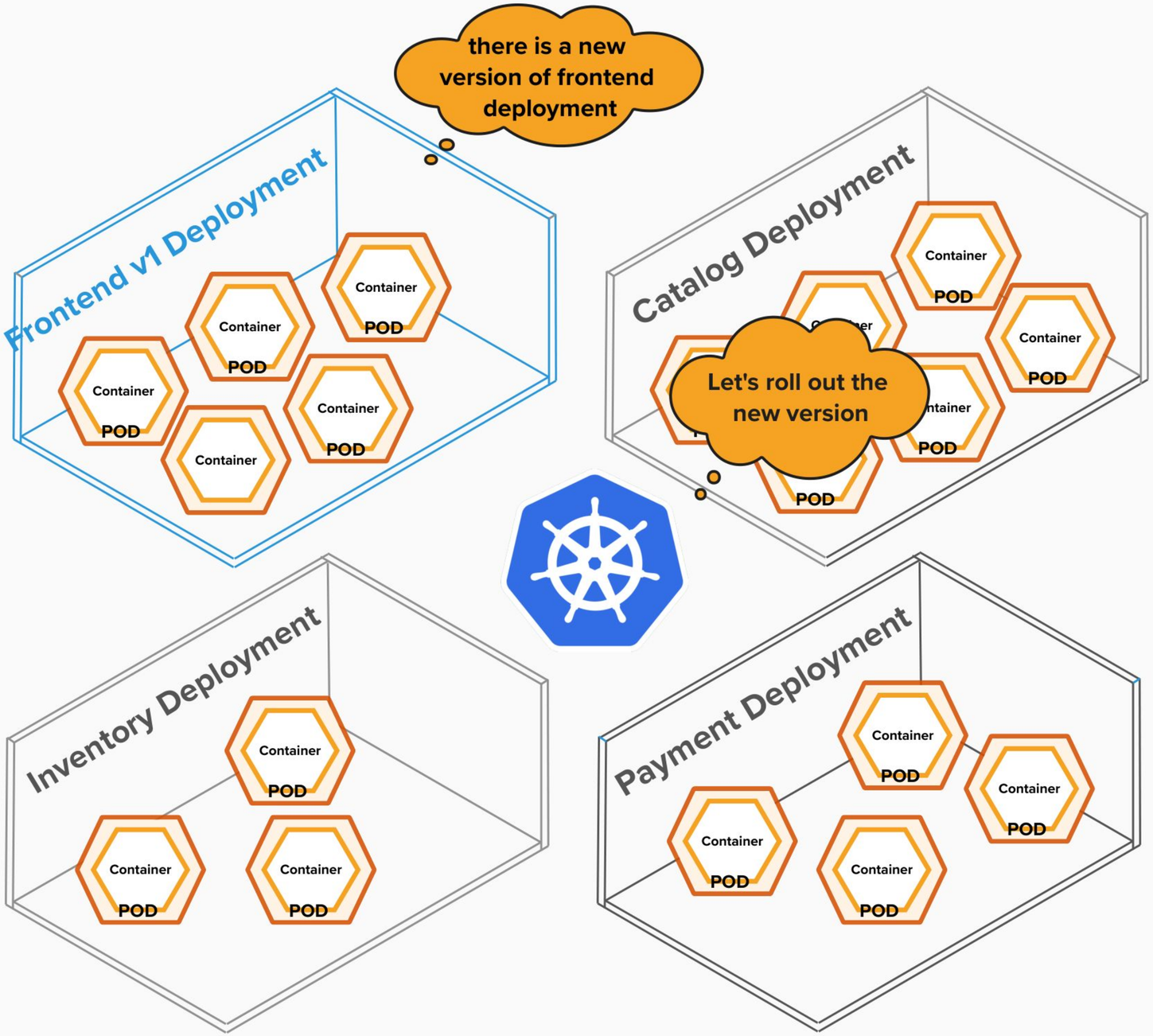
1. KUBERNETES



1. KUBERNETES

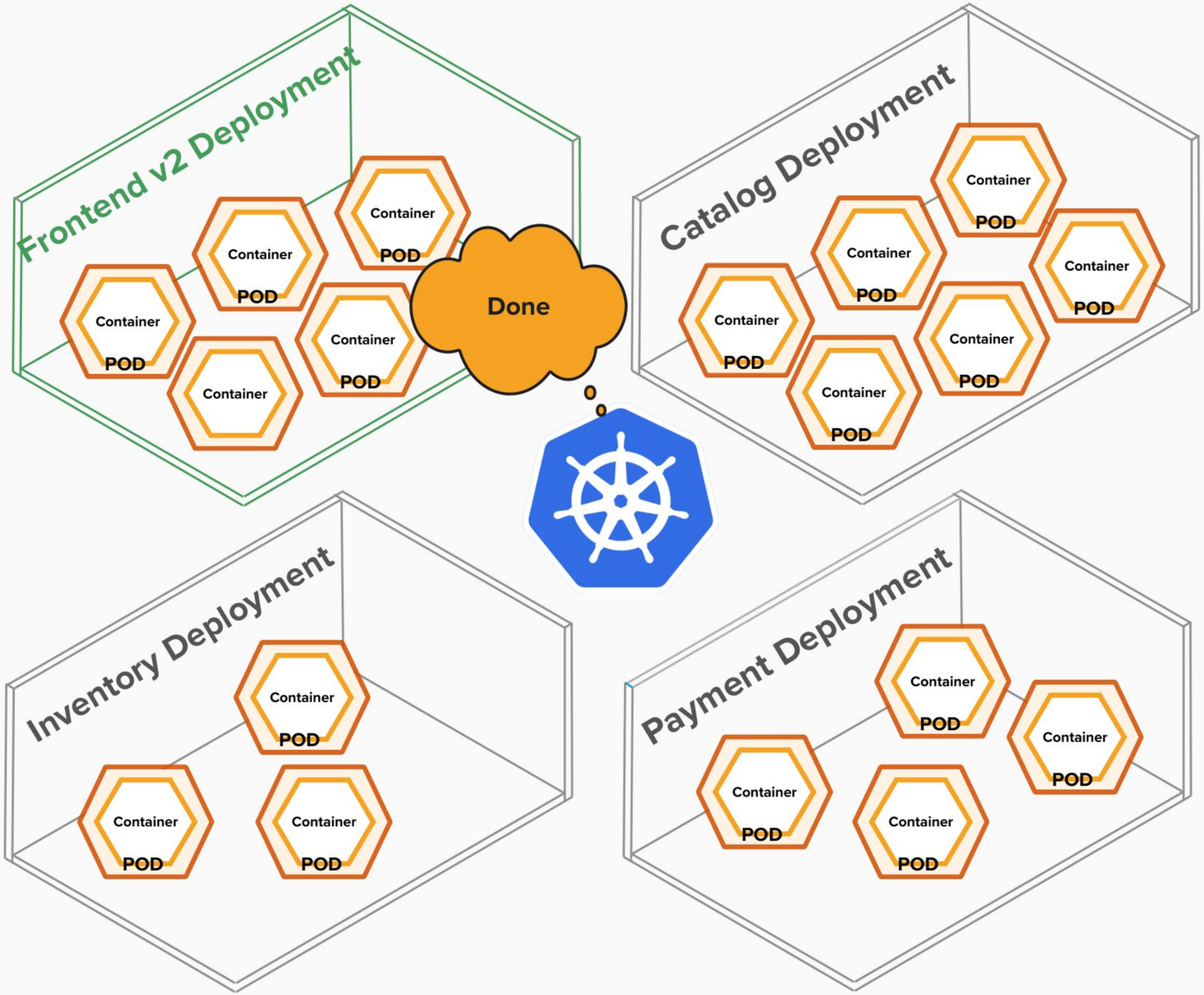


1. KUBERNETES

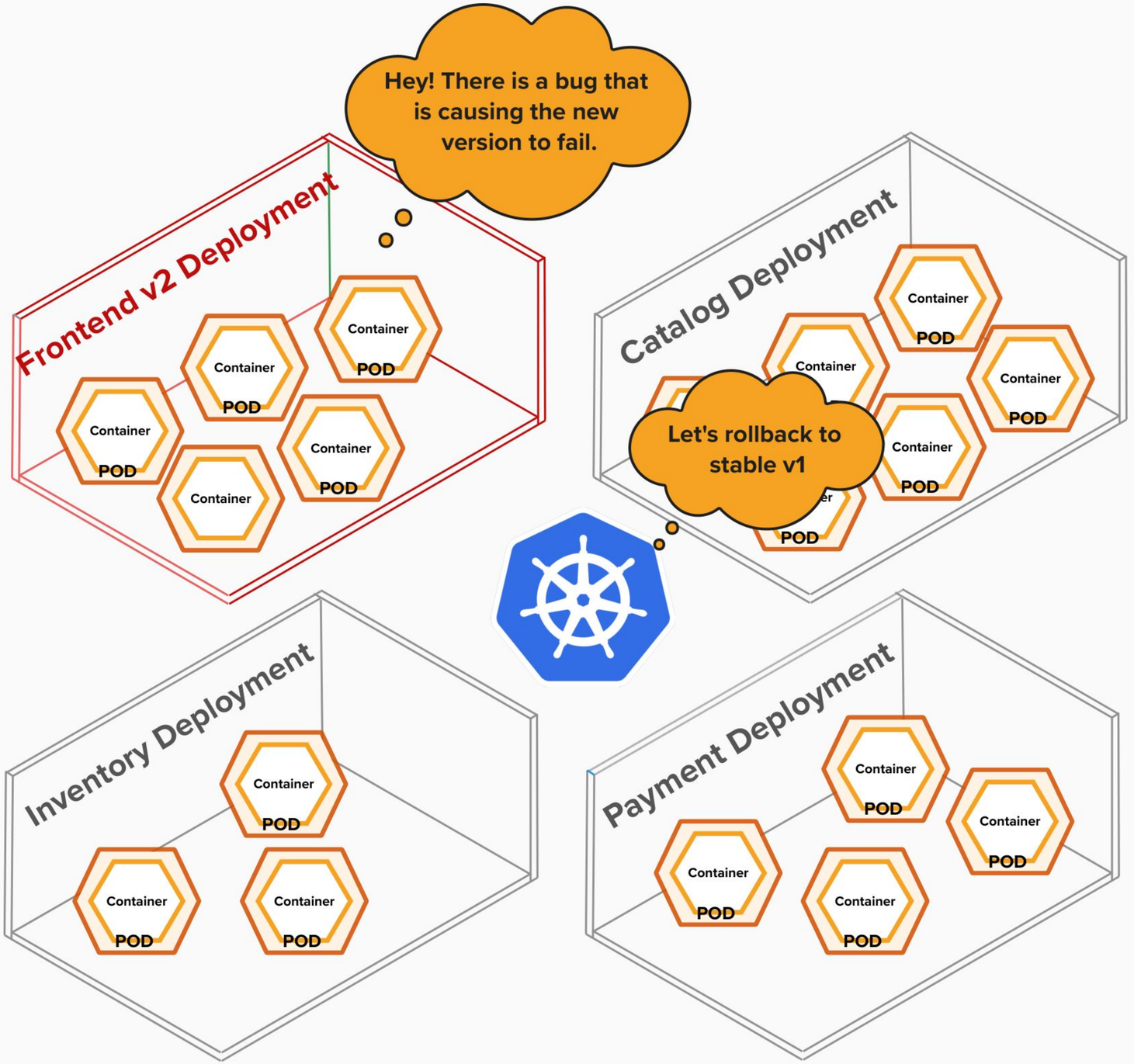


IBM Red Hat

1. KUBERNETES

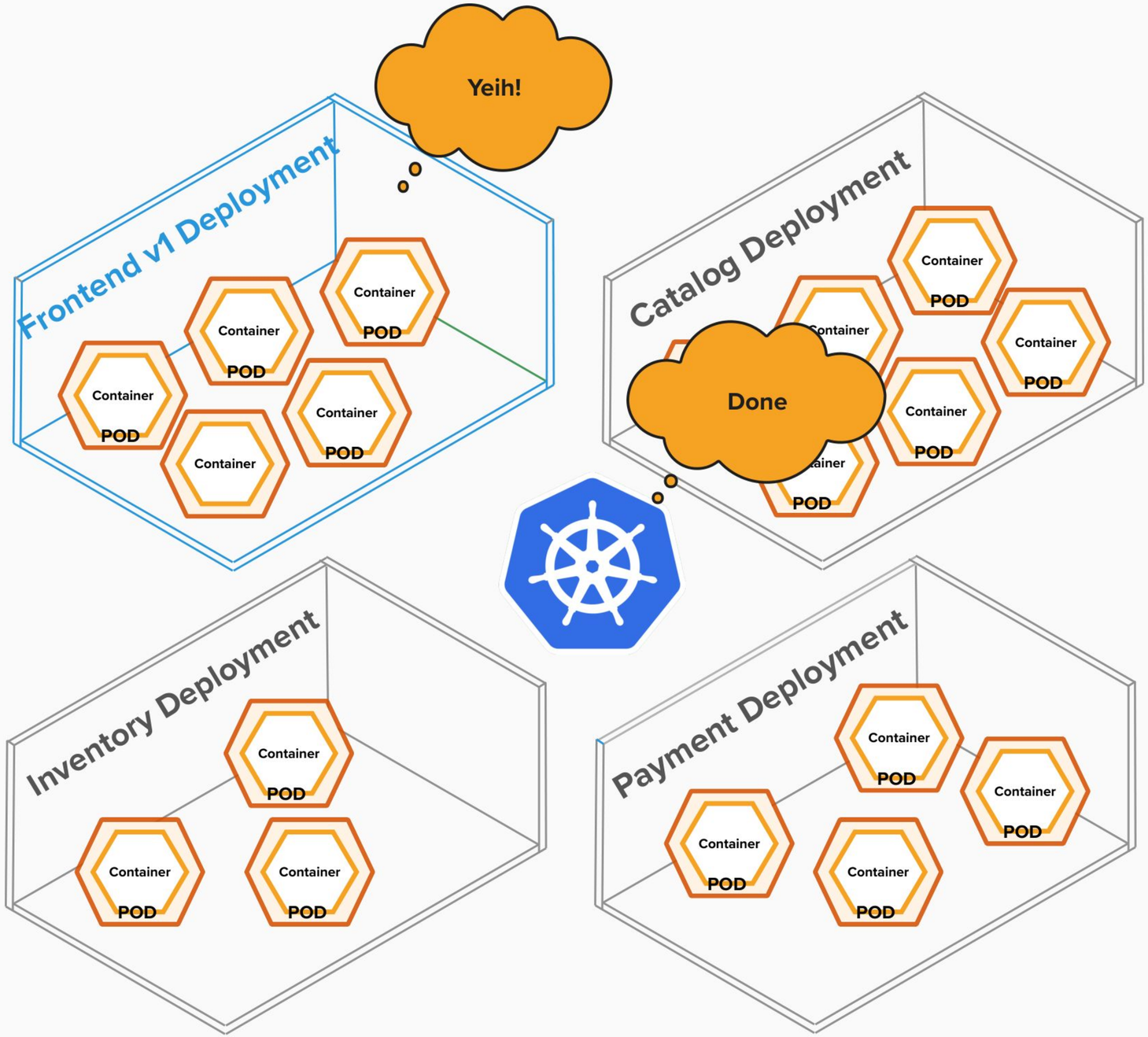


1. KUBERNETES

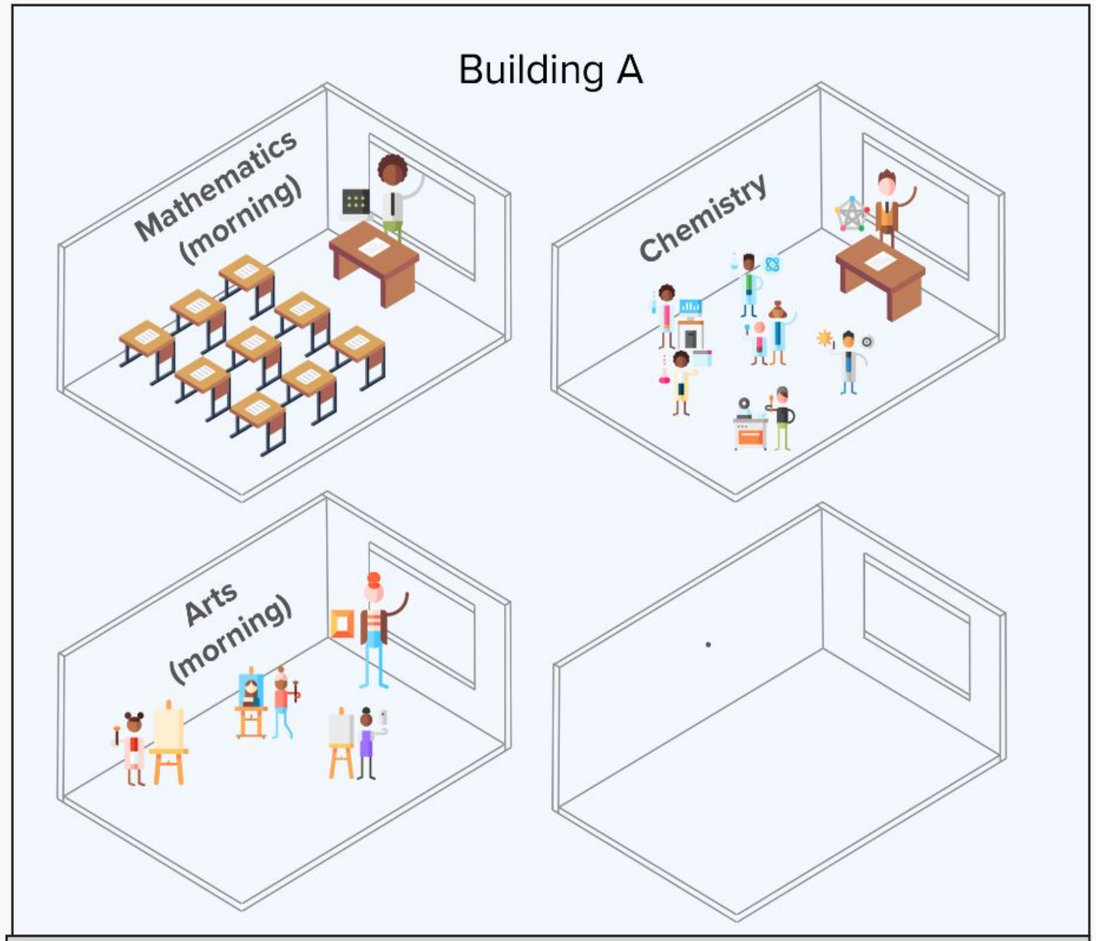


IBM Red Hat

1. KUBERNETES

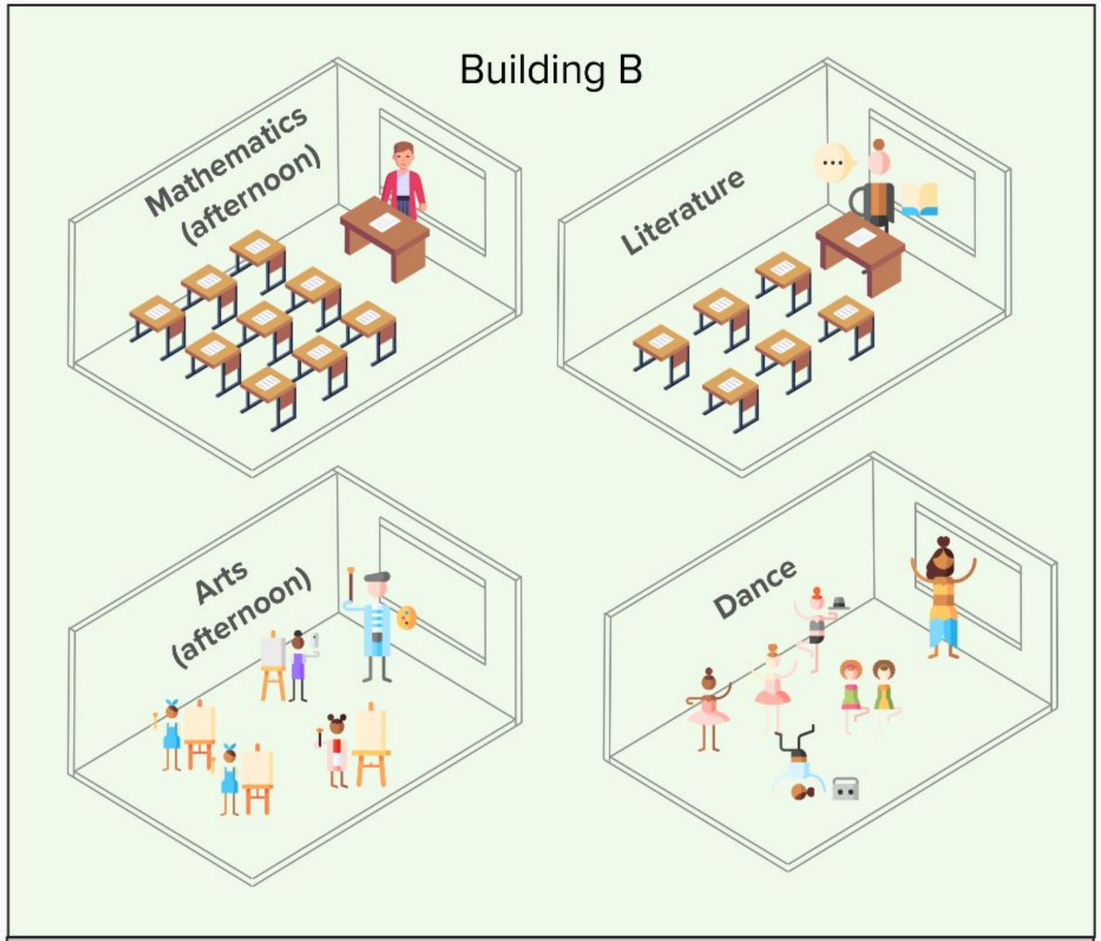


1. KUBERNETES



Equipment: instrument musical, painting canvas, chemistry lab, ...etc

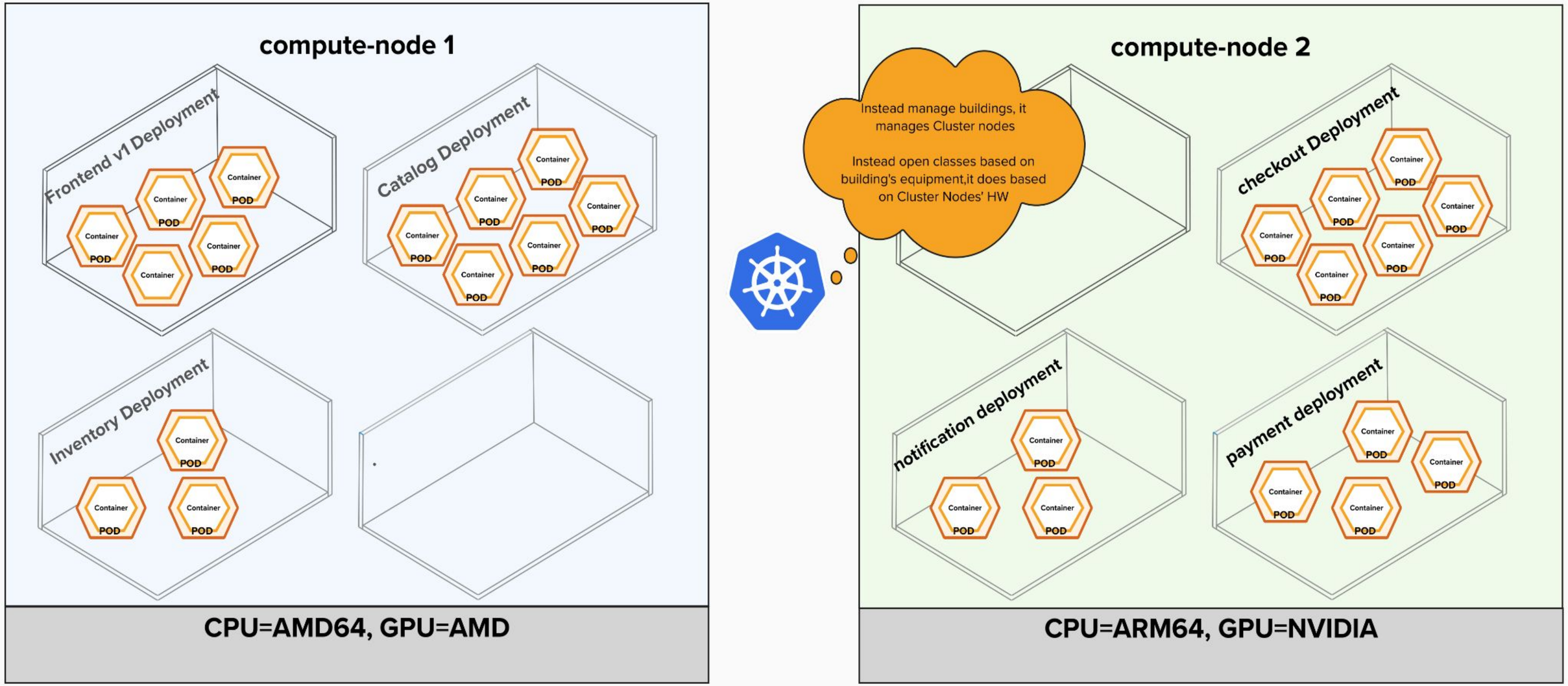




Equipment: painting canvas, Dance studio, ...etc

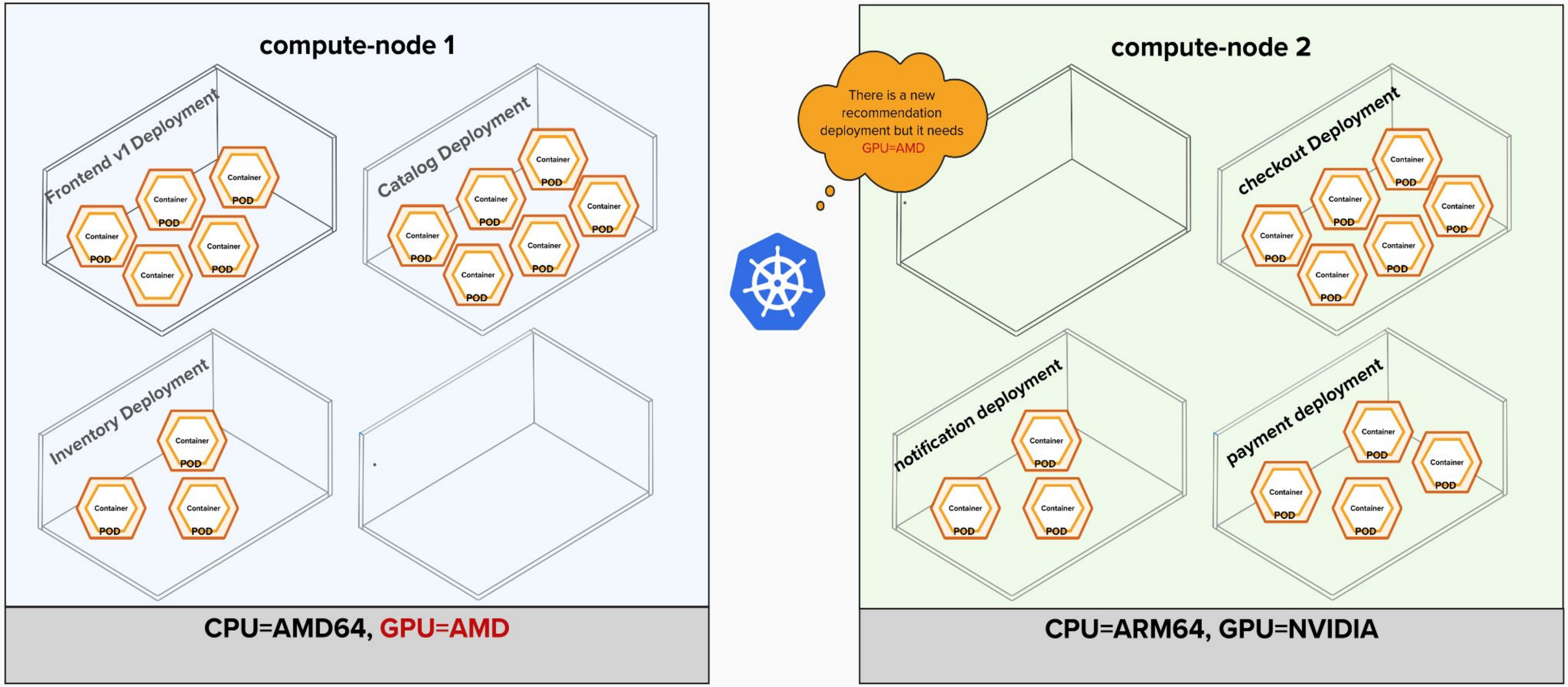
IBM Red Hat

1. KUBERNETES



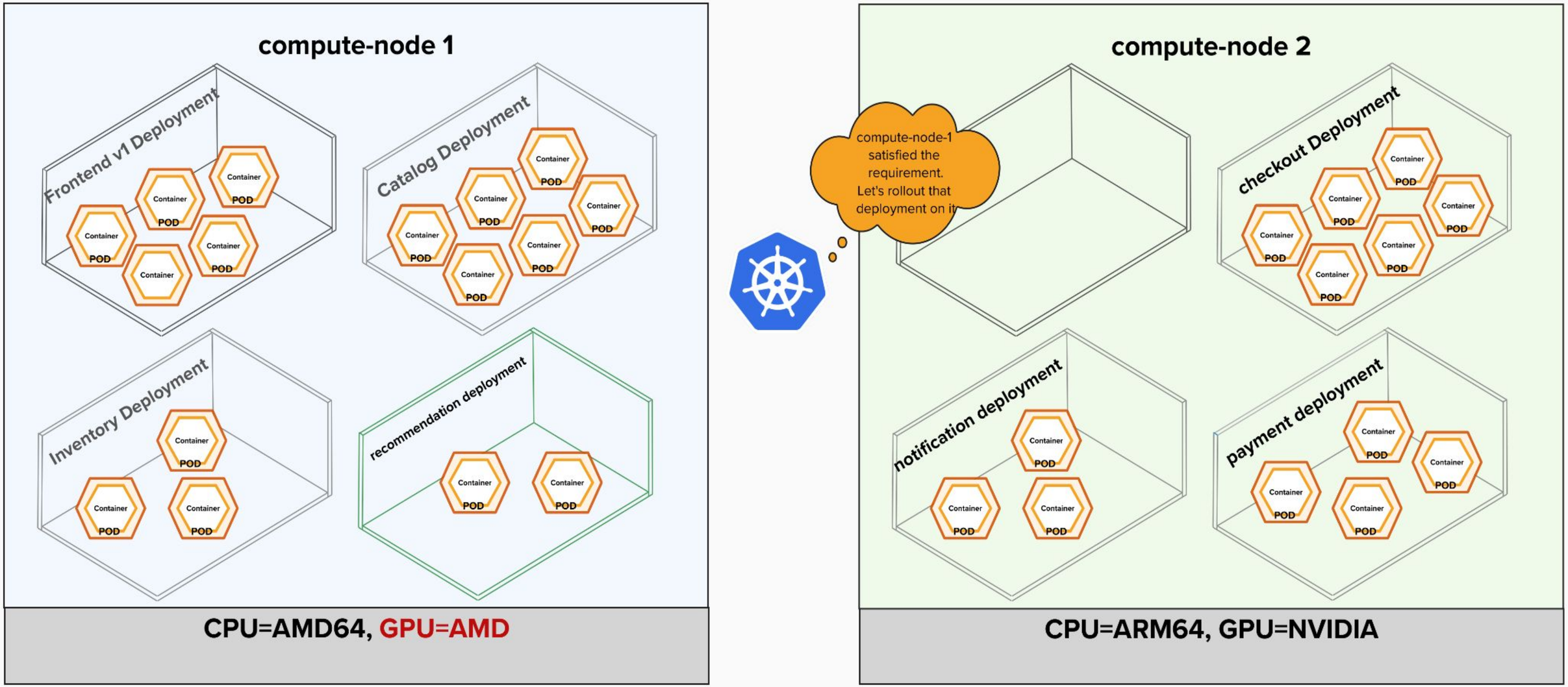
IBM Red Hat

1. KUBERNETES



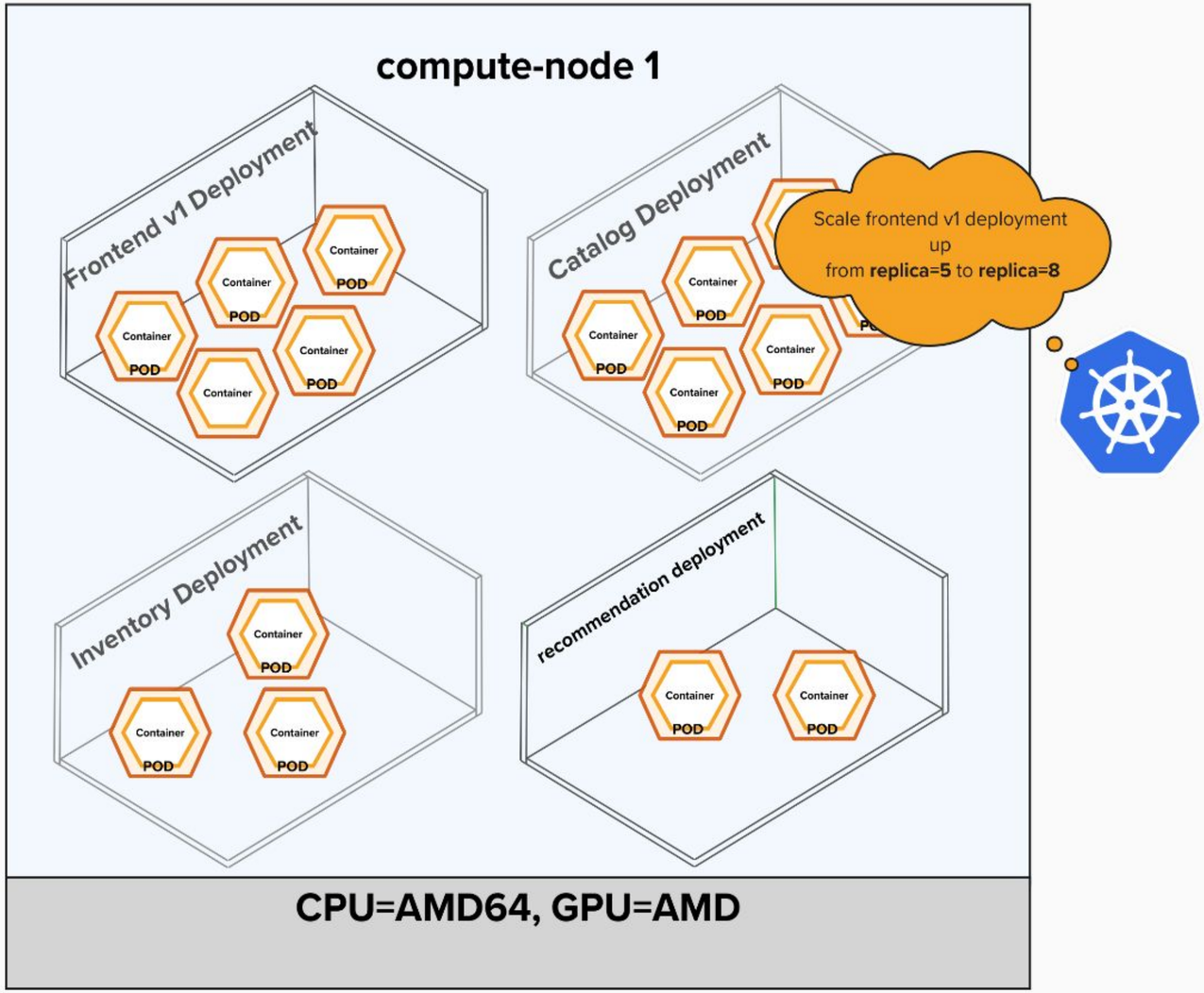
IBM Red Hat

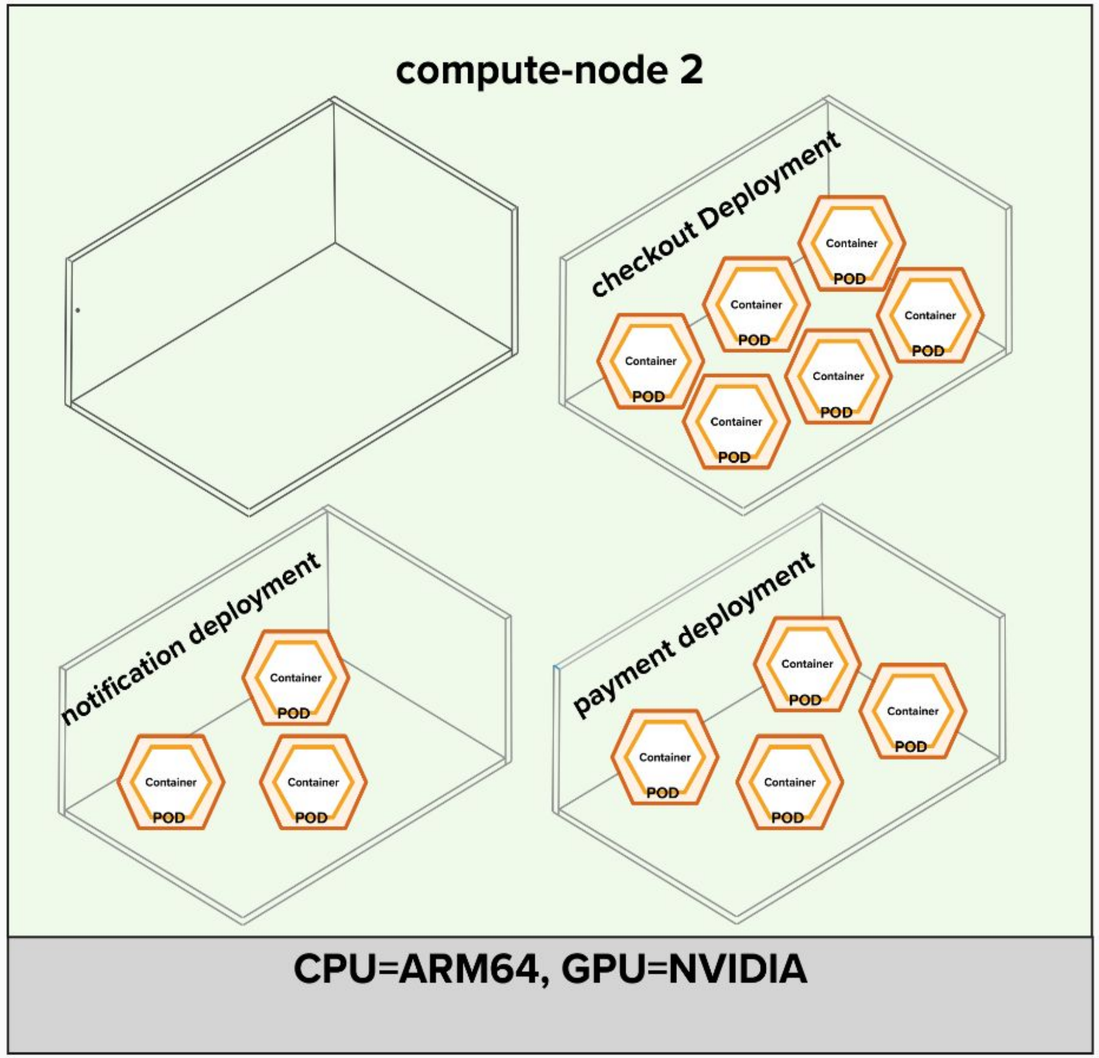
1. KUBERNETES



IBM Red Hat

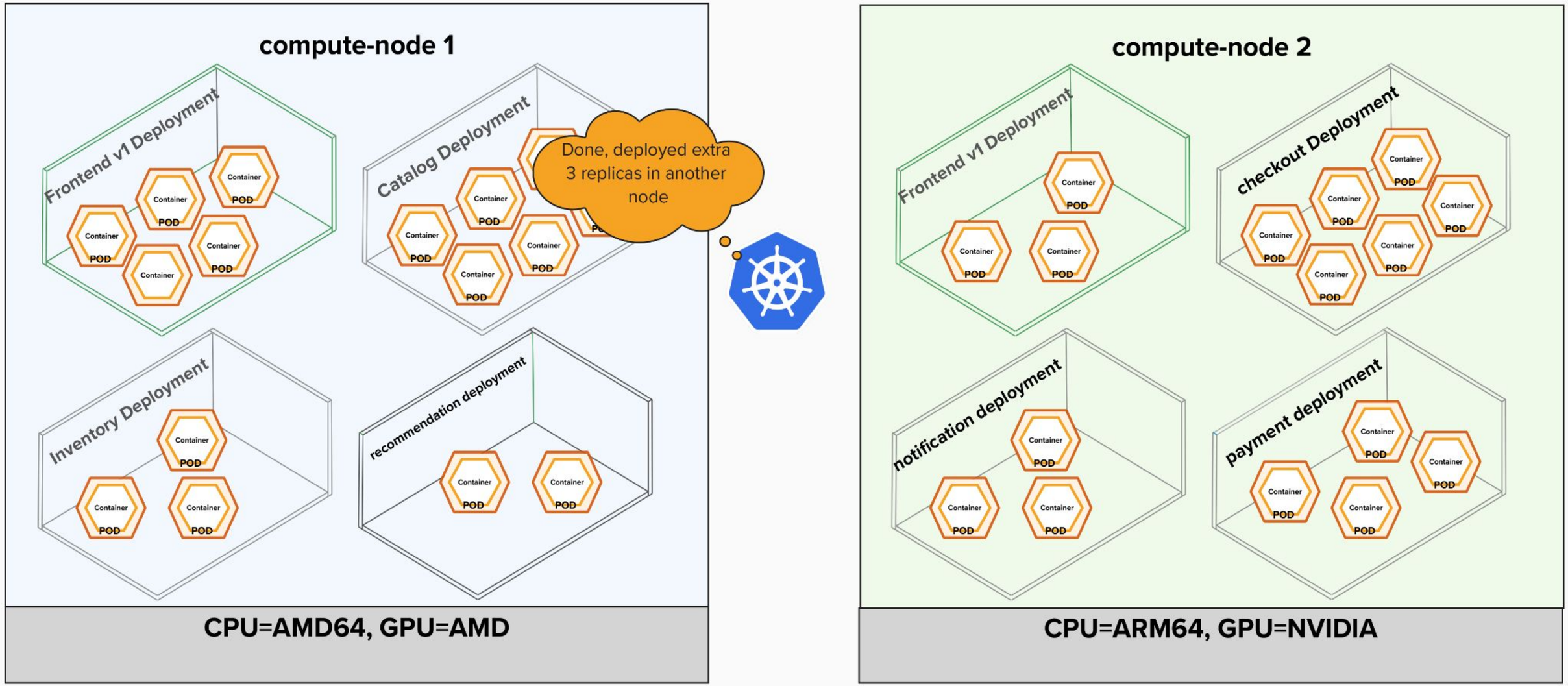
1. KUBERNETES





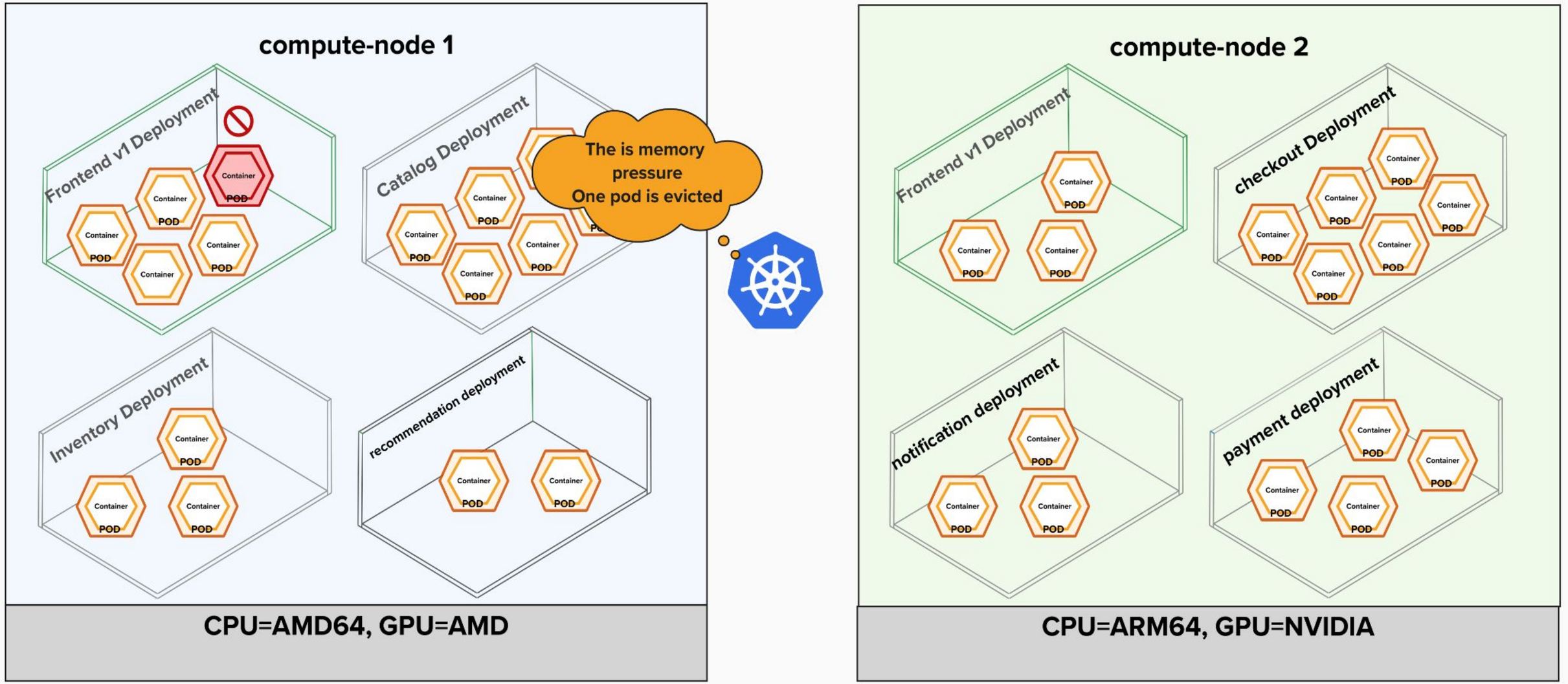
IBM Red Hat

1. KUBERNETES



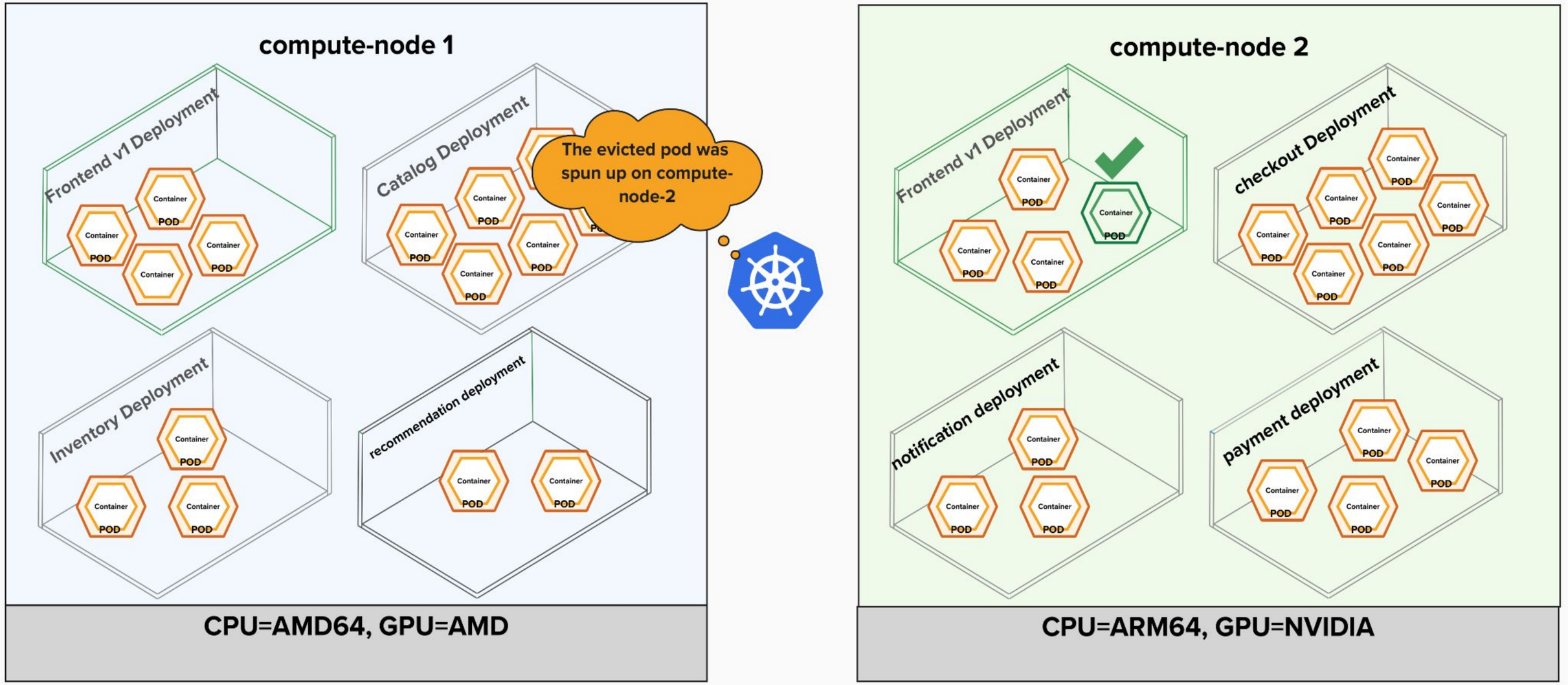
IBM Red Hat

1. KUBERNETES



IBM Red Hat

1. KUBERNETES



IBM Red Hat

WHAT IS KUBERNETES

- Kubernetes is a portable, extensible, open source platform for managing containerized workloads and services that facilitate both declarative configuration and automation. It has a large, rapidly growing ecosystem. Kubernetes services, support, and tools are widely available.
- Created by three Google employees initially during the summer of 2014; grew exponentially and became the first project to get donated to the CNCF.
- Hit the first production-grade version v1.0.1 in July 2015. Has continually released a new minor version every three months since v1.2.0 in March 2016. Lately v1.13.0 was released in December 2018.

Decouples Infrastructure and Scaling

- **All services** within Kubernetes are natively Load Balanced.
- Can scale up and down dynamically.
- Used both to enable self-healing and seamless upgrading or rollback of applications.

Self Healing

- Kubernetes will **ALWAYS** try and steer the cluster to its desired state.
- **Me:** “I want 3 healthy instances of redis to always be running.”
- **Kubernetes:** “Okay, I’ll ensure there are always 3 instances up and running.”
- **Kubernetes:** “Oh look, one has died. I’m going to attempt to spin up a new one.”

containers are created from container images

Architecture Overview

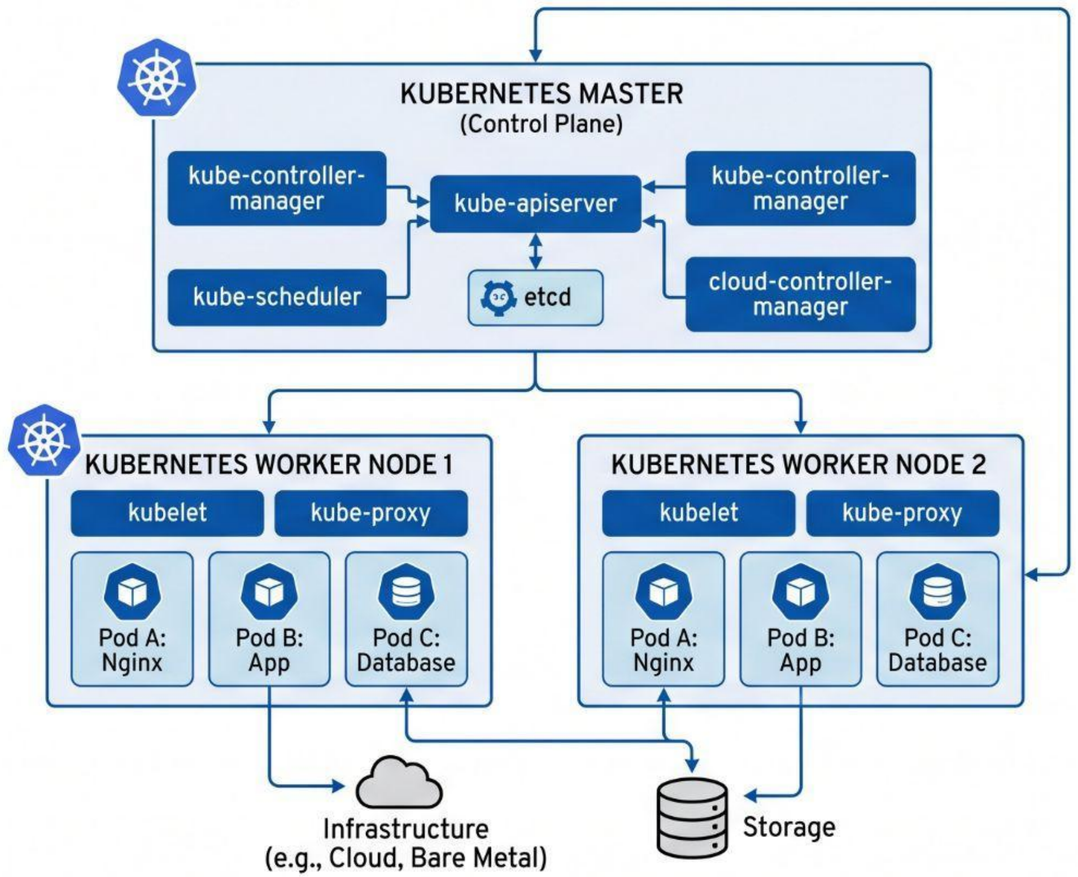


Red Hat

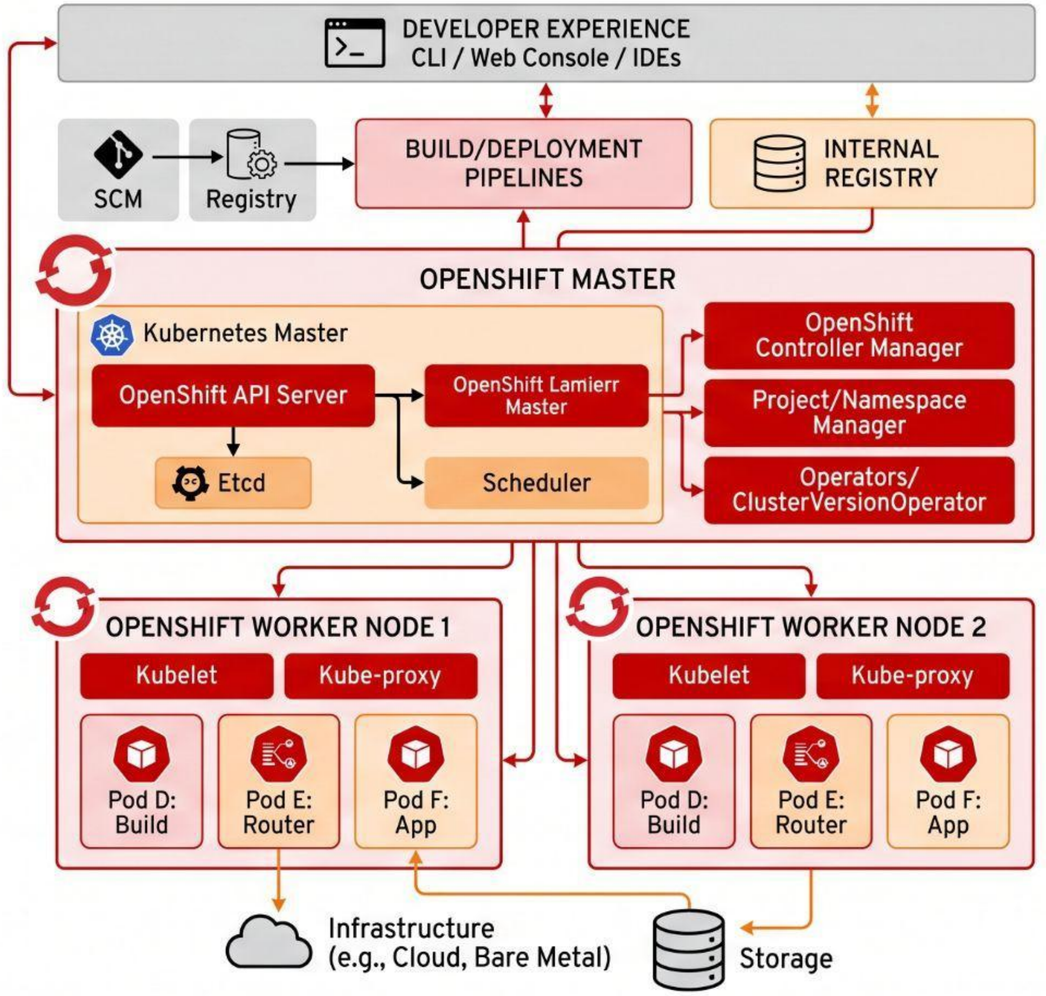


COMPARING KUBERNETES AND OPENSHIFT ARCHITECTURES

KUBERNETES ARCHITECTURE



OPENSHIFT ARCHITECTURE



IBM Red Hat

containers are created from container images

Core Objects and API

- Namespaces
- Pods
- Labels
- Selectors
- Services

Red Hat IBM

Namespaces

Namespaces are a logical cluster or environment, and are the primary method of partitioning a cluster or scoping access.

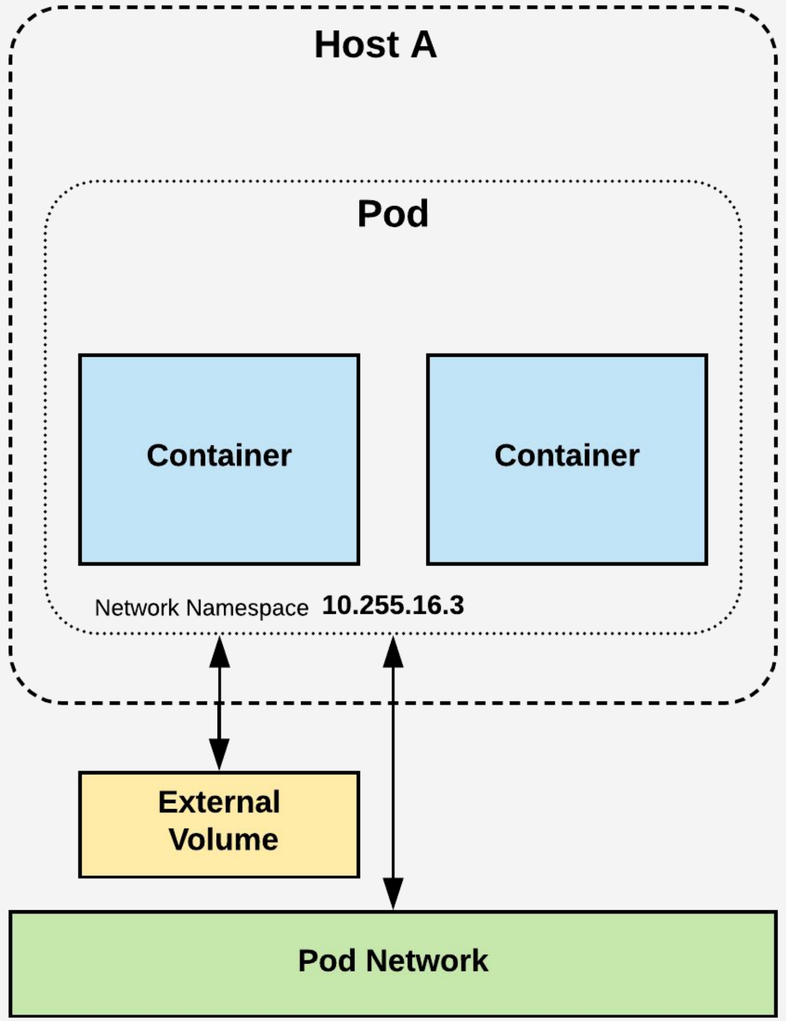
```
apiVersion: v1
kind: Namespace
metadata:
  name: prod
  labels:
    app: MyBigWebApp
```

```
$ kubectl get ns --show-labels
NAME          STATUS    AGE       LABELS
default       Active    11h       <none>
kube-public   Active    11h       <none>
kube-system   Active    11h       <none>
prod          Active    6s        app=MyBigWebApp
```

Red Hat IBM

Pods

- **Atomic unit** or smallest "*unit of work*"of Kubernetes.
- Pods are **one or MORE containers** that share volumes and namespace.
- **They are also ephemeral!**



Red Hat IBM

Pod Examples

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-example
spec:
  containers:
  - name: nginx
    image: nginx:stable-alpine
    ports:
    - containerPort: 80
```

Red Hat IBM

Key Pod Container Attributes

- `name` - The name of the container
- `image` - The container image
- `ports` - array of ports to expose. Can be granted a friendly name and protocol may be specified
- `env` - array of environment variables
- `command` – Entrypoint array (equiv to Docker `ENTRYPOINT`)
- `args` - Arguments to pass to the command (equiv to Docker `CMD`)

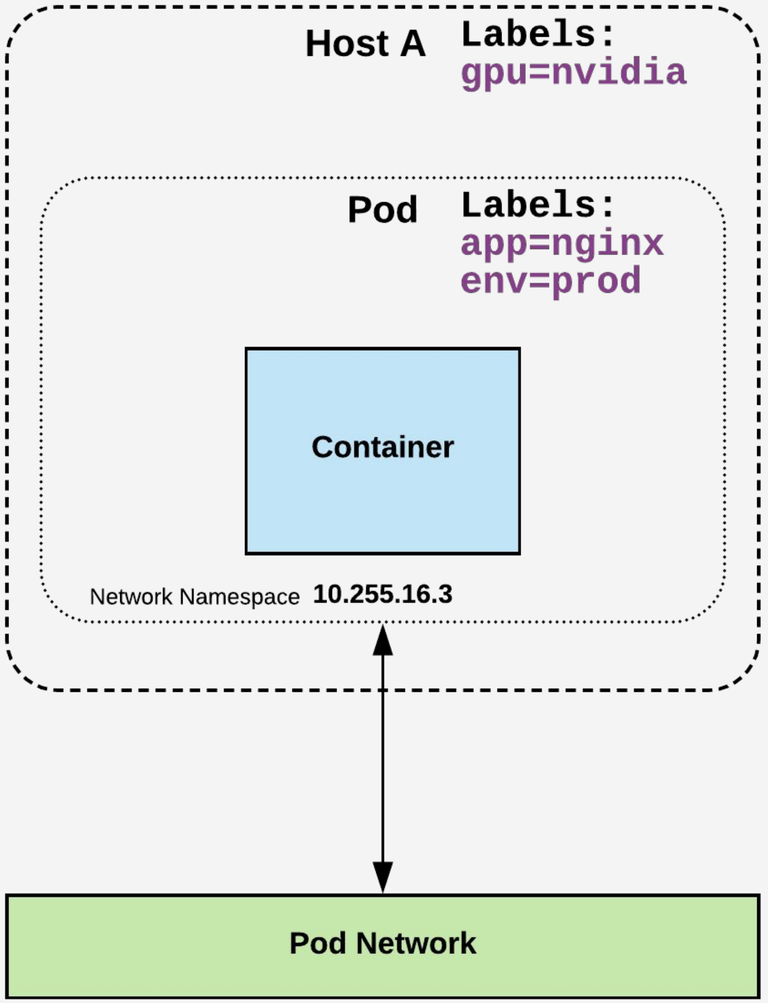
Container

```
name: nginx
image: nginx:stable-alpine
ports:
  - containerPort: 80
    name: http
    protocol: TCP
env:
  - name: MYVAR
    value: isAwesome
command: [“/bin/sh”, “-c”]
args: [“echo ${MYVAR}”]
```

Red Hat IBM

Labels

- key-value pairs that are used to identify, describe and group together related sets of objects or resources.
- **NOT** characteristic of uniqueness.
- Have a strict syntax with a slightly limited character set*.



Selectors

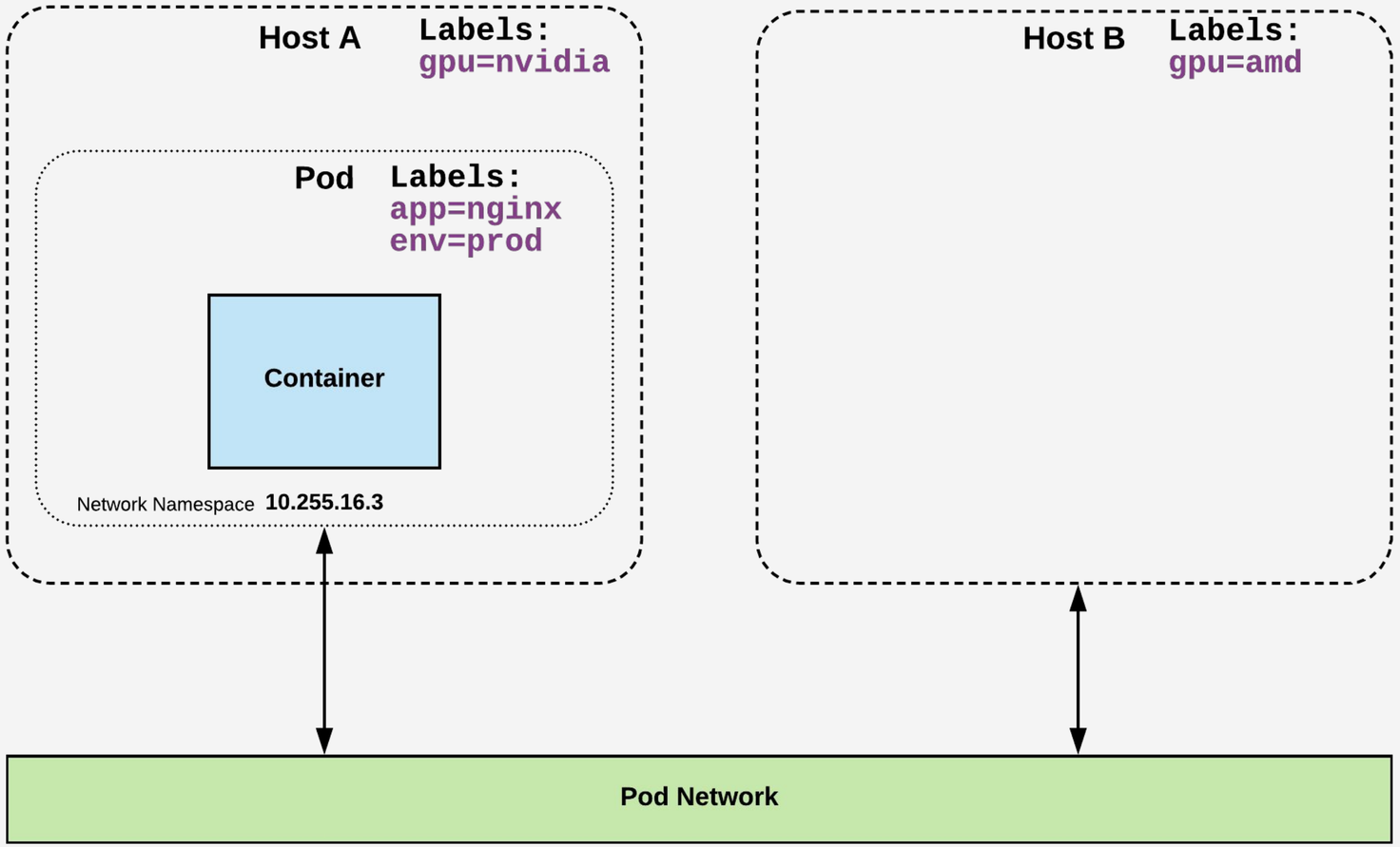
Selectors use labels to filter or select objects, and are used throughout Kubernetes.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-label-example
  labels:
    app: nginx
    env: prod
spec:
  containers:
  - name: nginx
    image: nginx:stable-alpine
    ports:
    - containerPort: 80
  nodeSelector:
    gpu: nvidia
```

Red Hat IBM

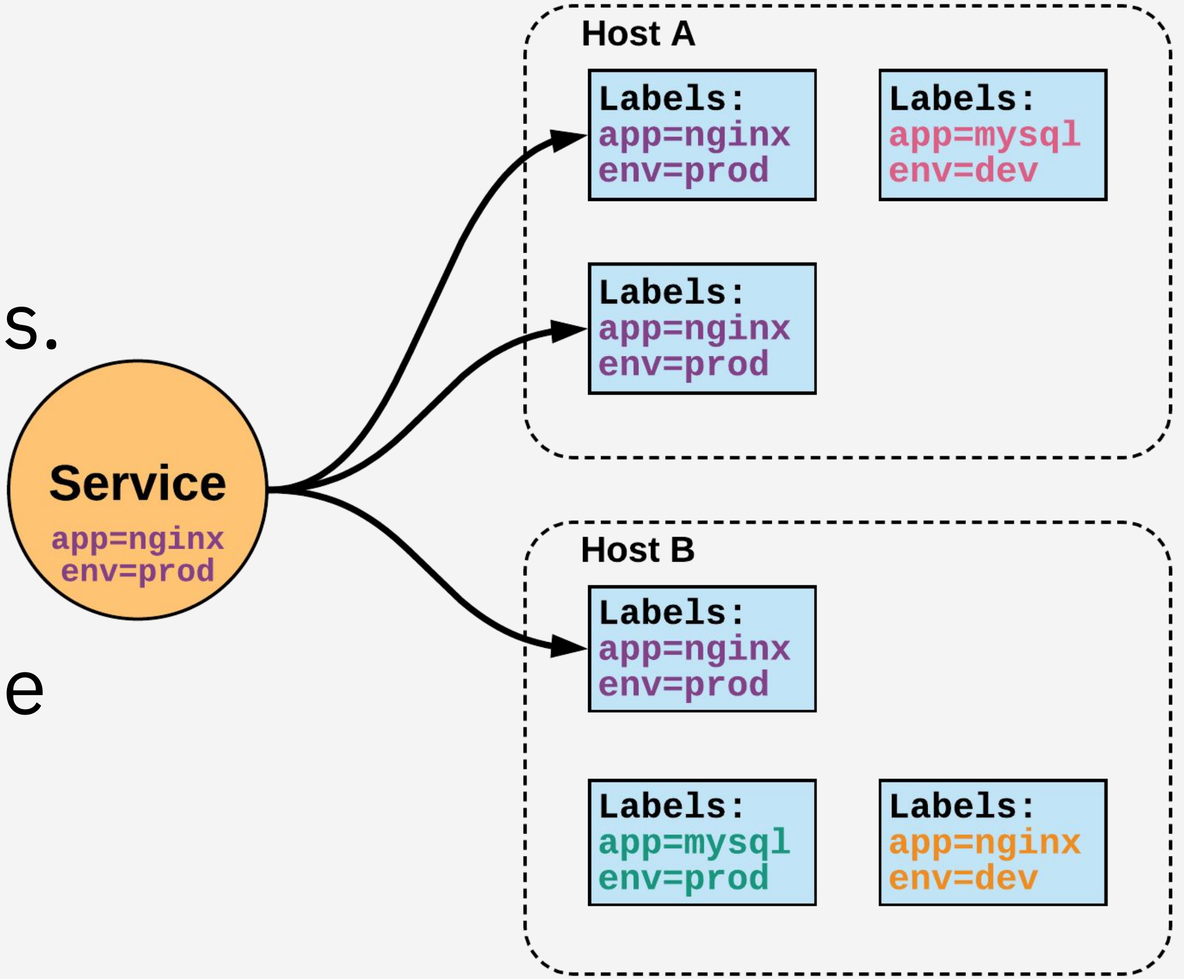
Selector Example

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-label-example
  labels:
    app: nginx
    env: prod
spec:
  containers:
  - name: nginx
    image: nginx:stable-alpine
    ports:
    - containerPort: 80
  nodeSelector:
    gpu: nvidia
```



Services

- **Unified method of accessing** the exposed workloads of Pods.
- **Durable resource**
  - static cluster IP
  - static namespaced DNS name




```
<service name>.<namespace>.svc.cluster.local
```

Red Hat IBM

Service Types

There are 4 major service types:

- ClusterIP (default)
- NodePort
- LoadBalancer
- ExternalName

Red Hat IBM

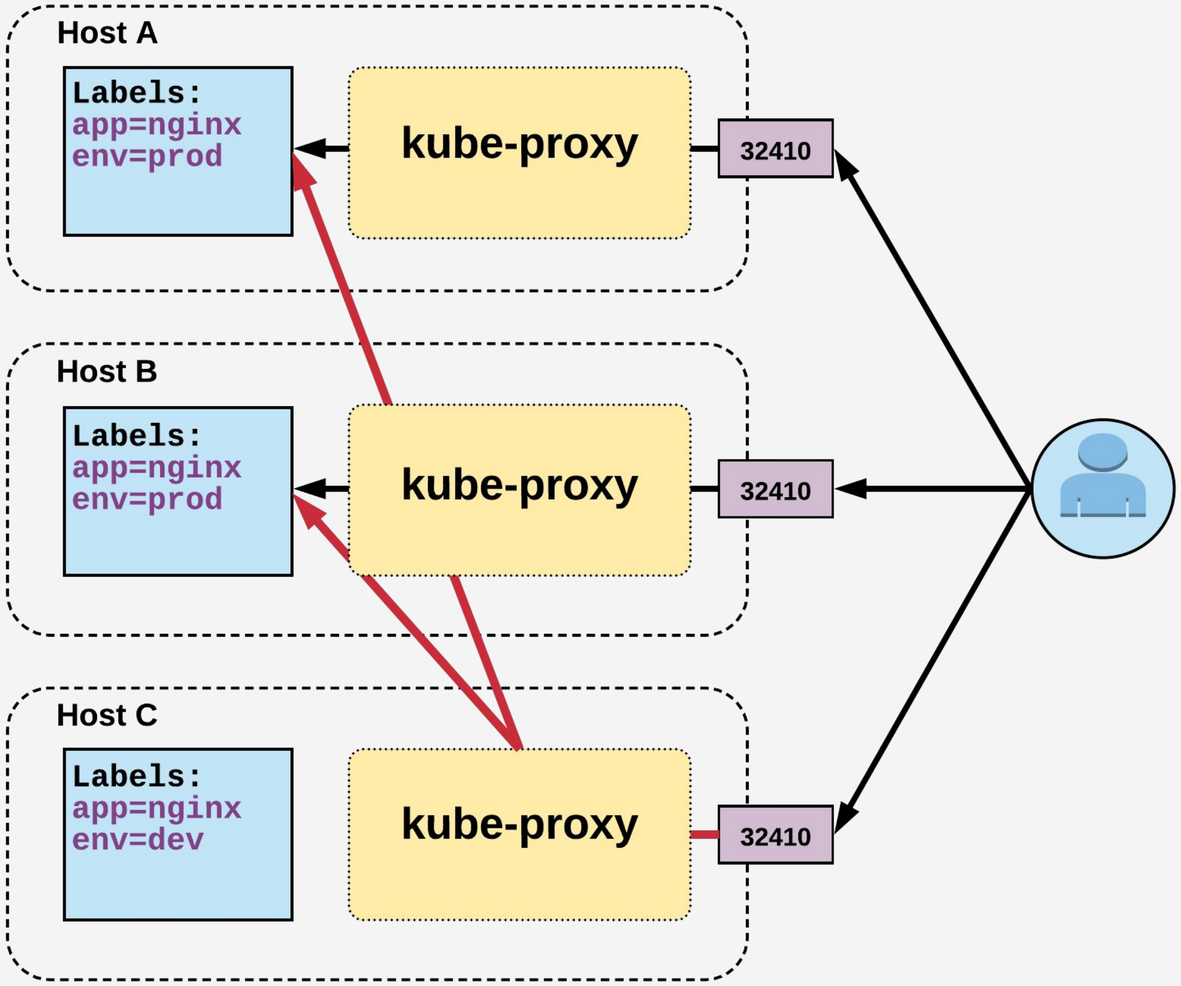
NodePort Service

- **NodePort** services extend the **ClusterIP** service.
- Exposes a port on every node's IP.
- Port can either be statically defined, or dynamically taken from a range between 30000-32767.

```yaml
apiVersion: v1
kind: Service
metadata:
  name: example-prod
spec:
  type: NodePort
  selector:
    app: nginx
    env: prod
  ports:
  - nodePort: 32410
    protocol: TCP
    port: 80
    targetPort: 80
```

Red Hat IBM

NodePort Service




```
Name:           example-prod
Selector:       app=nginx,env=prod
Type:           NodePort
IP:             10.96.28.176
Port:           <unset>  80/TCP
TargetPort:     80/TCP
NodePort:       <unset>  32410/TCP
Endpoints:      10.255.16.3:80,
                10.255.16.4:80
```

Red Hat IBM

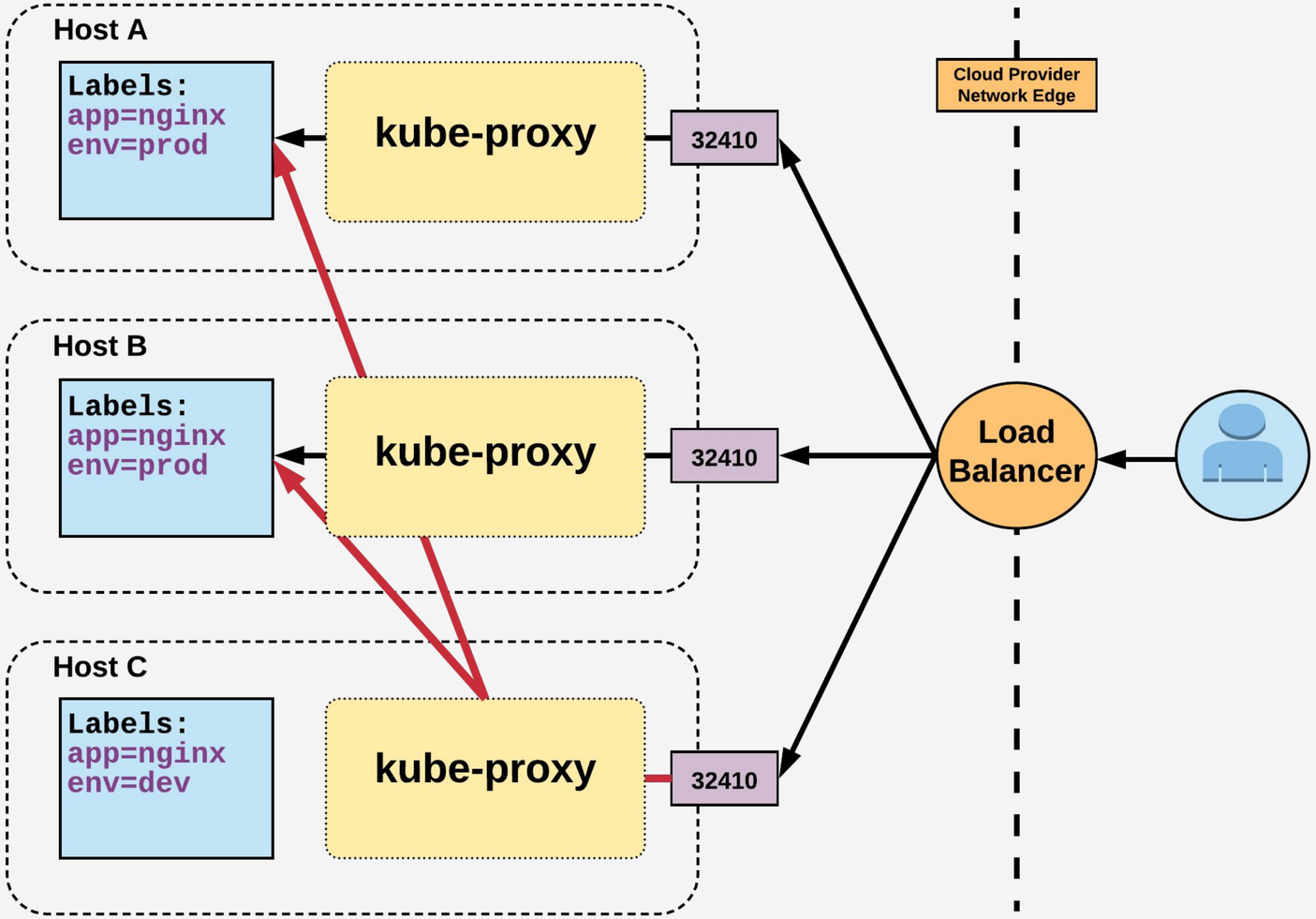
LoadBalancer Service

- **LoadBalancer** services extend **NodePort.**
- Works in conjunction with an external system to map a cluster external IP to the exposed service.

```
apiVersion: v1
kind: Service
metadata:
  name: example-prod
spec:
  type: LoadBalancer
  selector:
    app: nginx
    env: prod
  ports:
    protocol: TCP
    port: 80
    targetPort: 80
```

Red Hat IBM

LoadBalancer Service




```
Name:           example-prod
Selector:       app=nginx,env=prod
Type:           LoadBalancer
IP:             10.96.28.176
LoadBalancer
Ingress:        172.17.18.43
Port:           <unset>  80/TCP
TargetPort:     80/TCP
NodePort:       <unset>  32410/TCP
Endpoints:      10.255.16.3:80,
                10.255.16.4:80
```

Red Hat IBM

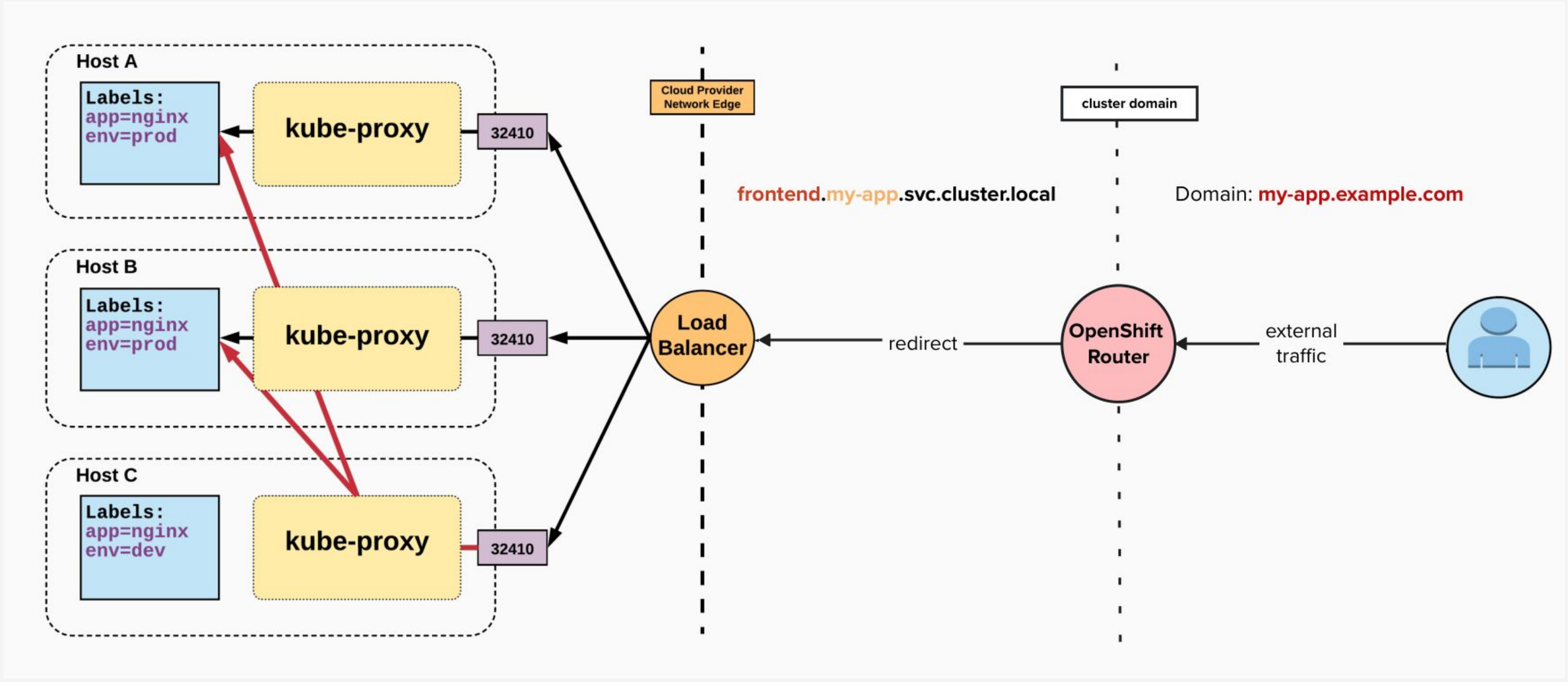
OpenShift Router

- An OpenShift component that manages external access to the services in a cluster
- Provides load balancing, SSL termination and host/path-based routing
- Gives services externally reachable URLs through Route objects

```yaml
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: simple-fanout-example
spec:
  host: foo.bar.com
  path:
  to:
    kind: Service
    name: service1
  port:
    targetPort: 8080
  tls:
    termination: edge
    insecureEdgeTerminationPolicy: Redirect
```

Red Hat IBM

OpenShift Router



containers are created from container images

Core workloads

- ReplicaSet
- Deployment
- StatefulSet

Red Hat IBM

ReplicaSet

- Primary method of managing pod replicas and their lifecycle.
- Includes their scheduling, scaling, and deletion.
- Their job is simple: **Always ensure the desired number of pods are running.**



ReplicaSet

- replicas: The desired number of instances of the Pod.
- selector:The label selector for the **ReplicaSet** will manage **ALL** Pod instances that it targets; whether it’s desired or not.

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: rs-example
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
      env: prod
  template:
    <pod template>
```

Red Hat IBM

ReplicaSet

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: rs-example
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
      env: prod
  template:
    metadata:
      labels:
        app: nginx
        env: prod
    spec:
      containers:
      - name: nginx
        image: nginx:stable-alpine
        ports:
        - containerPort: 80
```

```
$ kubectl get pods
NAME                READY     STATUS    RESTARTS   AGE
rs-example-9l4dt    1/1       Running   0          1h
rs-example-b7bcg    1/1       Running   0          1h
rs-example-mkll2    1/1       Running   0          1h
```

```
$ kubectl describe rs rs-example
Name:         rs-example
Namespace:    default
Selector:     app=nginx,env=prod
Labels:       app=nginx
              env=prod
Annotations:  <none>
Replicas:     3 current / 3 desired
Pods Status:  3 Running / 0 Waiting / 0 Succeeded / 0 Failed
Pod Template:
  Labels:  app=nginx
           env=prod
  Containers:
   nginx:
    Image:        nginx:stable-alpine
    Port:         80/TCP
    Environment:  <none>
    Mounts:       <none>
  Volumes:        <none>
Events:
  Type    Reason            Age   From                   Message
  ----    ------            ----  ----                   -------
  Normal  SuccessfulCreate  16s   replicaset-controller  Created pod: rs-example-mkll2
  Normal  SuccessfulCreate  16s   replicaset-controller  Created pod: rs-example-b7bcg
  Normal  SuccessfulCreate  16s   replicaset-controller  Created pod: rs-example-9l4dt
```

Red Hat IBM

Deployment

- Way of managing Pods via **ReplicaSets.**
- Provide rollback functionality and update control.
- Updates are managed through the **pod-template-hash** label.
- Each iteration creates a unique label that is assigned to both the **ReplicaSet** and subsequent Pods.

Deployment

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.7.9
        ports:
        - containerPort: 80
```

Red Hat IBM

Deployment

- revisionHistoryLimit: The number of previous iterations of the Deployment to retain.
- strategy: Describes the method of updating the Pods based on the type. Valid options are Recreate or RollingUpdate.
  - Recreate: All existing Pods are killed before the new ones are created.
  - RollingUpdate: Cycles through updating the Pods according to the parameters: maxSurge and maxUnavailable.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: deploy-example
spec:
  replicas: 3
  revisionHistoryLimit: 3
  selector:
    matchLabels:
      app: nginx
      env: prod
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    <pod template>
```

Red Hat IBM

Deployment

```
$ kubectl create deployment test --image=nginx
$ kubectl set image deployment test nginx=nginx:1.9.1 --record

$ kubectl rollout history deployment test
deployments "test"
REVISION  CHANGE-CAUSE
1         <none>
2         kubectl set image deployment test nginx=nginx:1.9.1 --record=true

$ kubectl annotate deployment test kubernetes.io/change-cause="image updated to 1.9.1"

$ kubectl rollout undo deployment test
$ kubectl rollout undo deployment test --to-revision=2

$ kubectl rollout history deployment test
deployments "test"
REVISION  CHANGE-CAUSE
2         kubectl set image deployment test nginx=nginx:1.9.1 --record=true
3         <none>

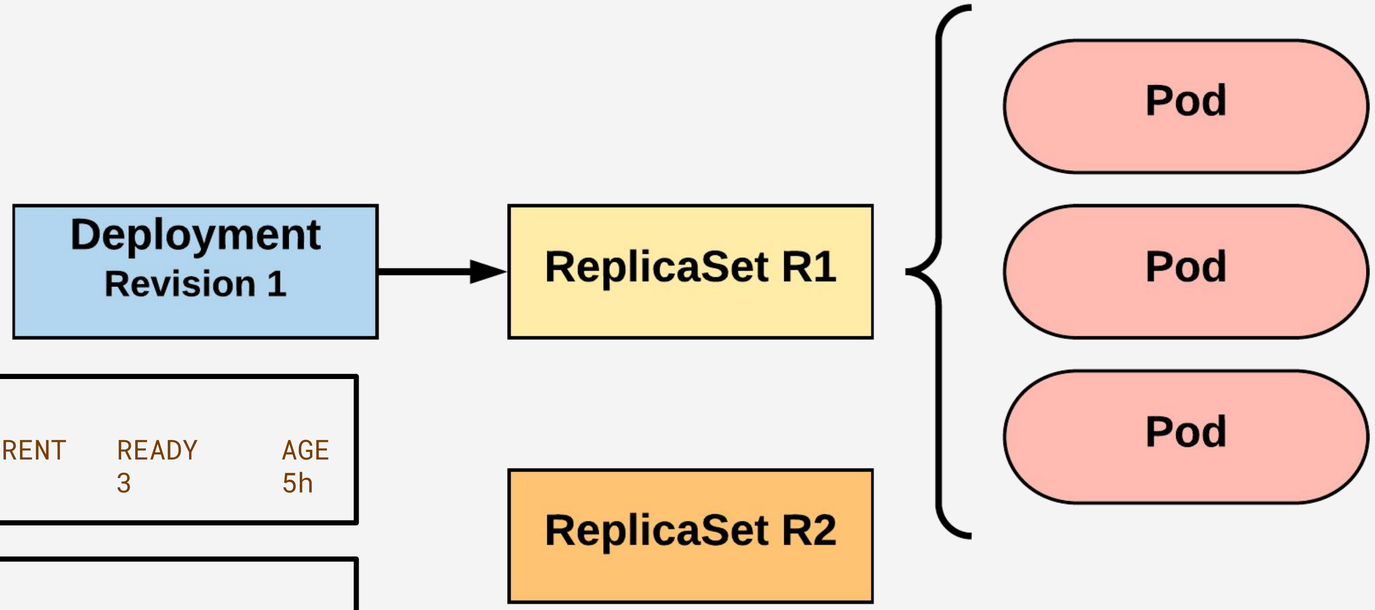
kubectl scale deployment test --replicas=10
kubectl rollout pause deployment test
kubectl rollout resume deployment test
```

Red Hat IBM

RollingUpdate Deployment

Updating pod template generates a new **ReplicaSet** revision.

R1 pod-template-hash:
676677fff
R2 pod-template-hash:
54f7ff7d6d




```
$ kubectl get replicaset
NAME                          DESIRED   CURRENT   READY     AGE
mydep-6766777fff              3         3         3         5h
```

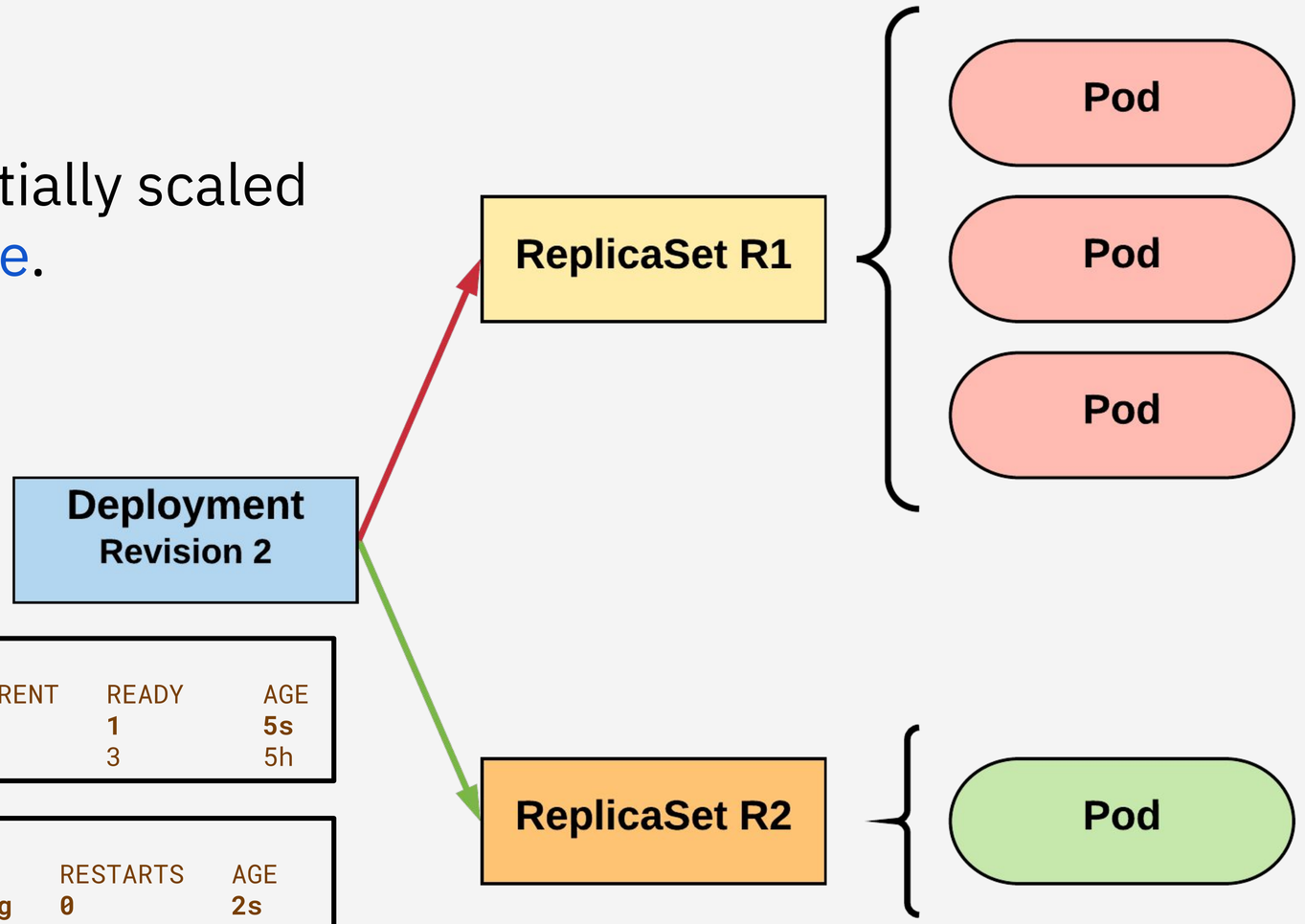
```
$ kubectl get pods
NAME                          READY     STATUS    RESTARTS   AGE
mydep-6766777fff-9r2zn        1/1       Running   0          5h
mydep-6766777fff-hsfz9        1/1       Running   0          5h
mydep-6766777fff-sjxhf        1/1       Running   0          5h
```

Red Hat IBM

RollingUpdate Deployment

New **ReplicaSet** is initially scaled up based on maxSurge.

R1 pod-template-hash:
676677fff
R2 pod-template-hash:
54f7ff7d6d




```
$ kubectl get replicaset
NAME                          DESIRED   CURRENT   READY     AGE
mydep-54f7ff7d6d              1         1         1         5s
mydep-6766777fff              2         3         3         5h
```

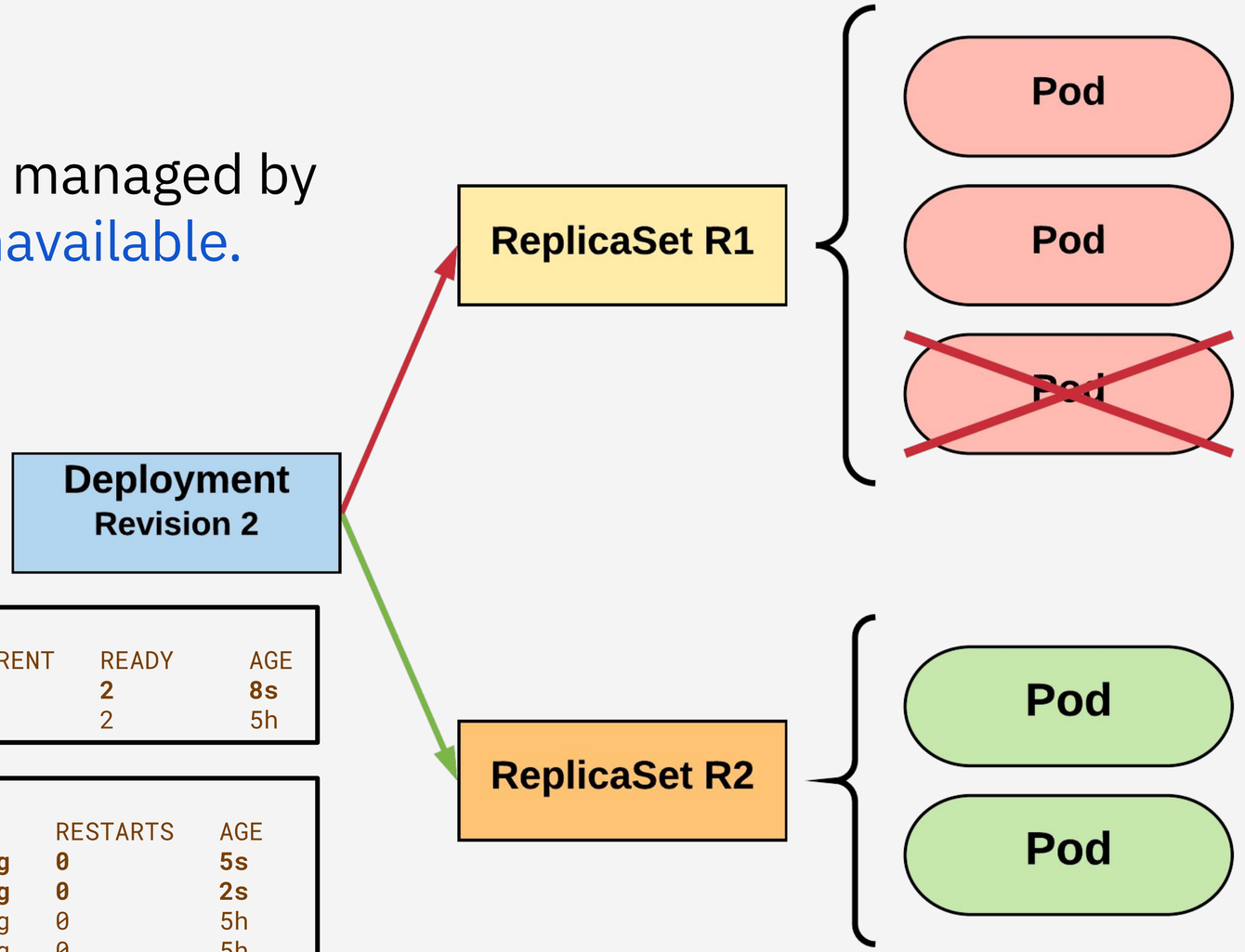
```
$ kubectl get pods
NAME                          READY     STATUS    RESTARTS   AGE
mydep-54f7ff7d6d-9gvll        1/1       Running   0          2s
mydep-6766777fff-9r2zn        1/1       Running   0          5h
mydep-6766777fff-hsfz9        1/1       Running   0          5h
mydep-6766777fff-sjxhf        1/1       Running   0          5h
```

Red Hat IBM

RollingUpdate Deployment

Phase out of old Pods managed by maxSurge and maxUnavailable.

R1 pod-template-hash:
676677fff
R2 pod-template-hash:
54f7ff7d6d




```
$ kubectl get replicaset
NAME                          DESIRED   CURRENT   READY     AGE
mydep-54f7ff7d6d              2         2         2         8s
mydep-6766777fff              2         2         2         5h
```

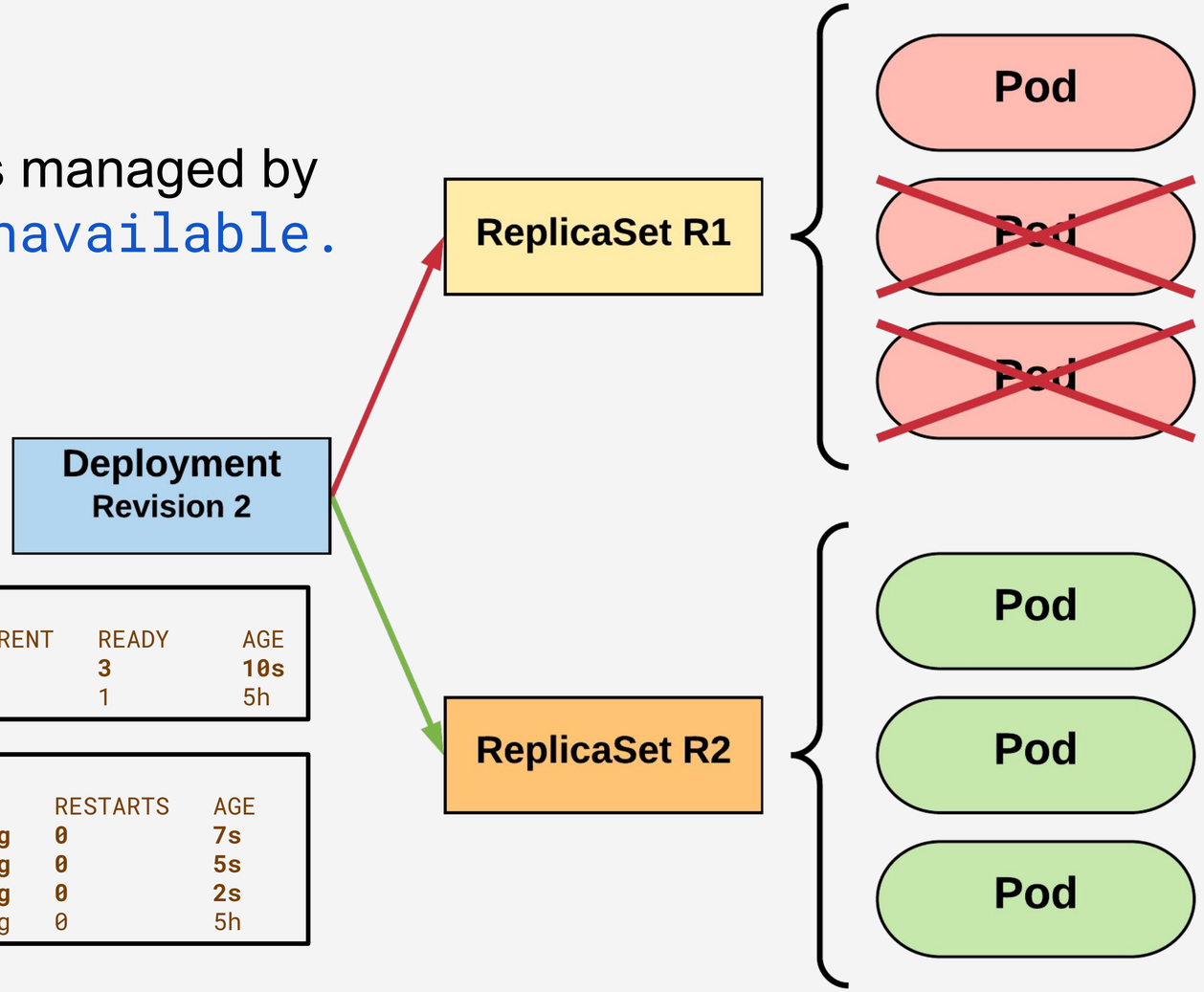
```
$ kubectl get pods
NAME                          READY     STATUS    RESTARTS   AGE
mydep-54f7ff7d6d-9gvll        1/1       Running   0          5s
mydep-54f7ff7d6d-cqvlq        1/1       Running   0          2s
mydep-6766777fff-9r2zn        1/1       Running   0          5h
mydep-6766777fff-hsfz9        1/1       Running   0          5h
```

Red Hat IBM

RollingUpdate Deployment

Phase out of old Pods managed by `maxSurge` and `maxUnavailable`.

R1 pod-template-hash:
`676677fff`
R2 pod-template-hash:
`54f7ff7d6d`




```
$ kubectl get replicaset
NAME                          DESIRED   CURRENT   READY     AGE
mydep-54f7ff7d6d              3         3         3         10s
mydep-6766777fff              0         1         1         5h
```

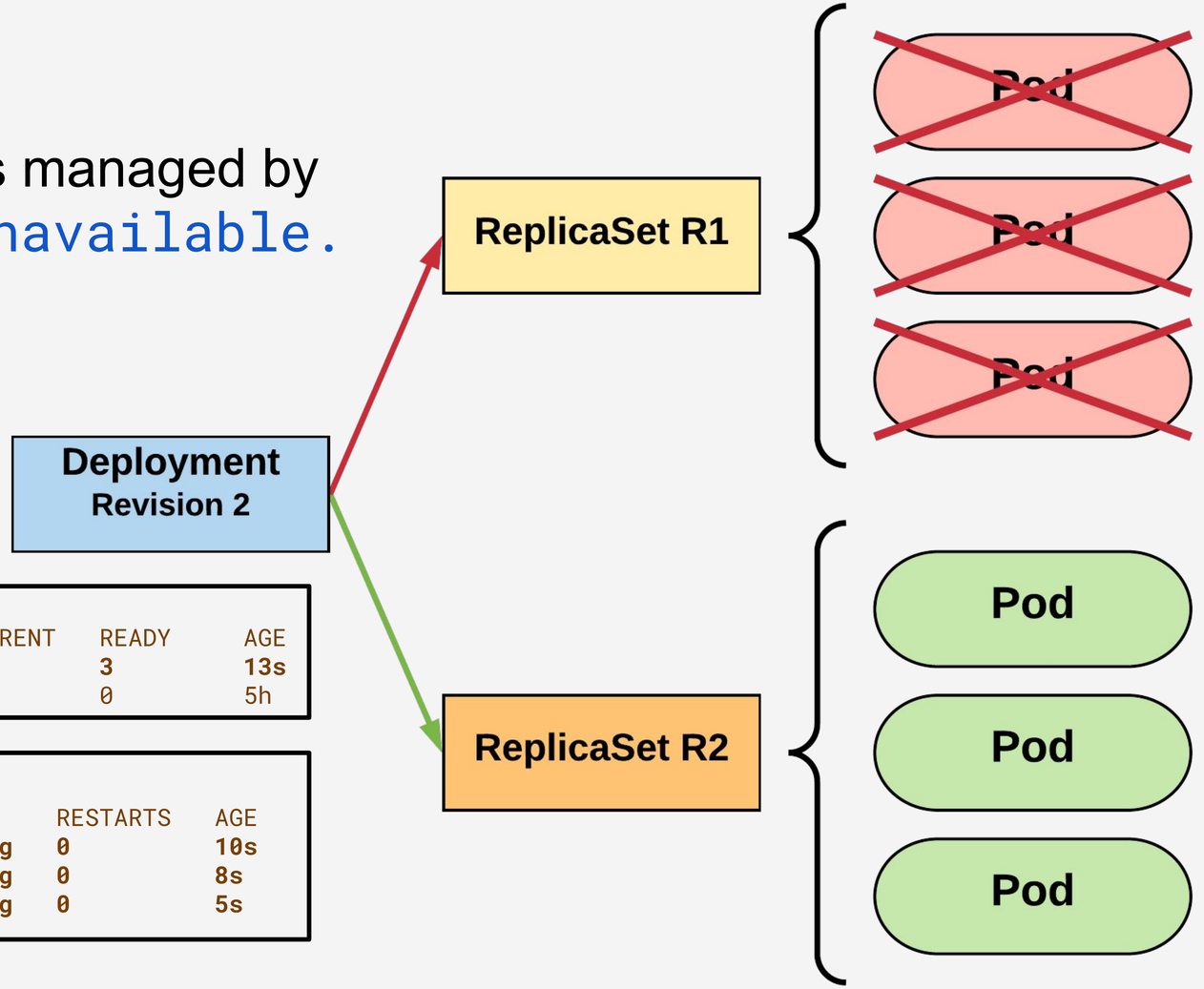
```
$ kubectl get pods
NAME                          READY     STATUS    RESTARTS   AGE
mydep-54f7ff7d6d-9gvll        1/1       Running   0          7s
mydep-54f7ff7d6d-cqvlq        1/1       Running   0          5s
mydep-54f7ff7d6d-gccr6        1/1       Running   0          2s
mydep-6766777fff-9r2zn        1/1       Running   0          5h
```

Red Hat IBM

RollingUpdate Deployment

Phase out of old Pods managed by `maxSurge` and `maxUnavailable`.

R1 pod-template-hash:
676677fff
R2 pod-template-hash:
54f7ff7d6d




```
$ kubectl get replicaset
NAME                          DESIRED   CURRENT   READY     AGE
mydep-54f7ff7d6d              3         3         3         13s
mydep-6766777fff              0         0         0         5h
```

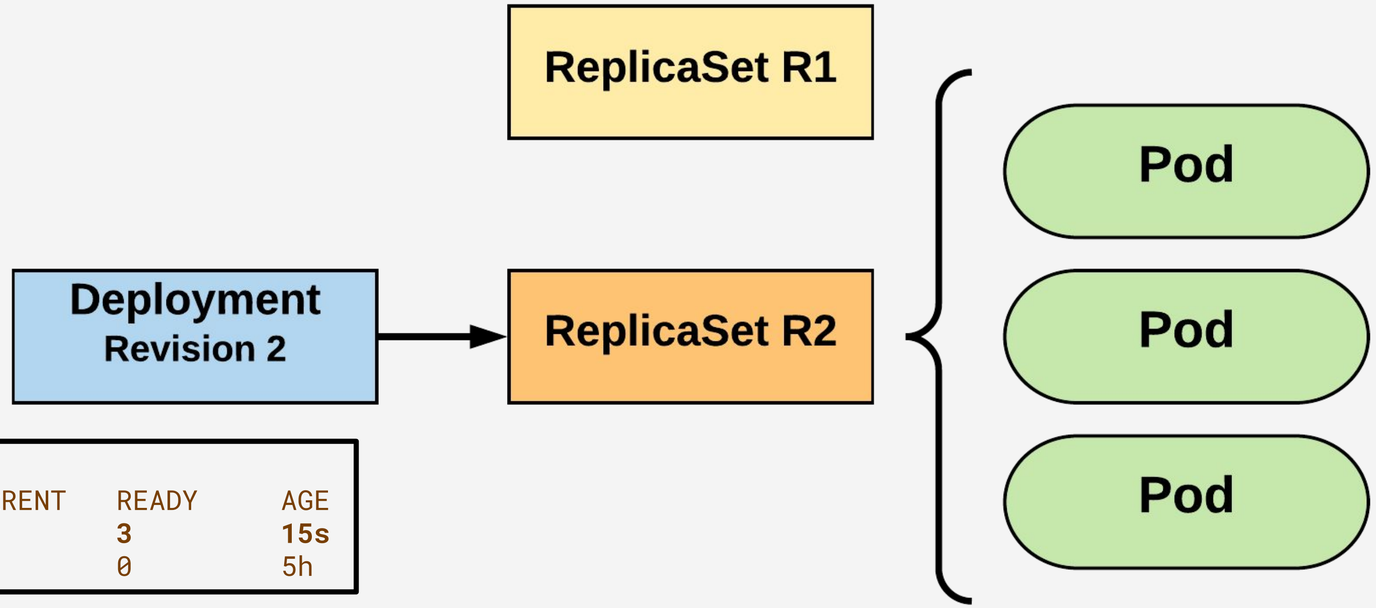
```
$ kubectl get pods
NAME                          READY     STATUS    RESTARTS   AGE
mydep-54f7ff7d6d-9gvll        1/1       Running   0          10s
mydep-54f7ff7d6d-cqvlq        1/1       Running   0          8s
mydep-54f7ff7d6d-gccr6        1/1       Running   0          5s
```

Red Hat IBM

RollingUpdate Deployment

Updated to new deployment revision completed.

R1 pod-template-hash:
`676677fff`
R2 pod-template-hash:
`54f7ff7d6d`




```
$ kubectl get replicaset
NAME                          DESIRED   CURRENT   READY     AGE
mydep-54f7ff7d6d              3         3         3         15s
mydep-6766777fff              0         0         0         5h
```

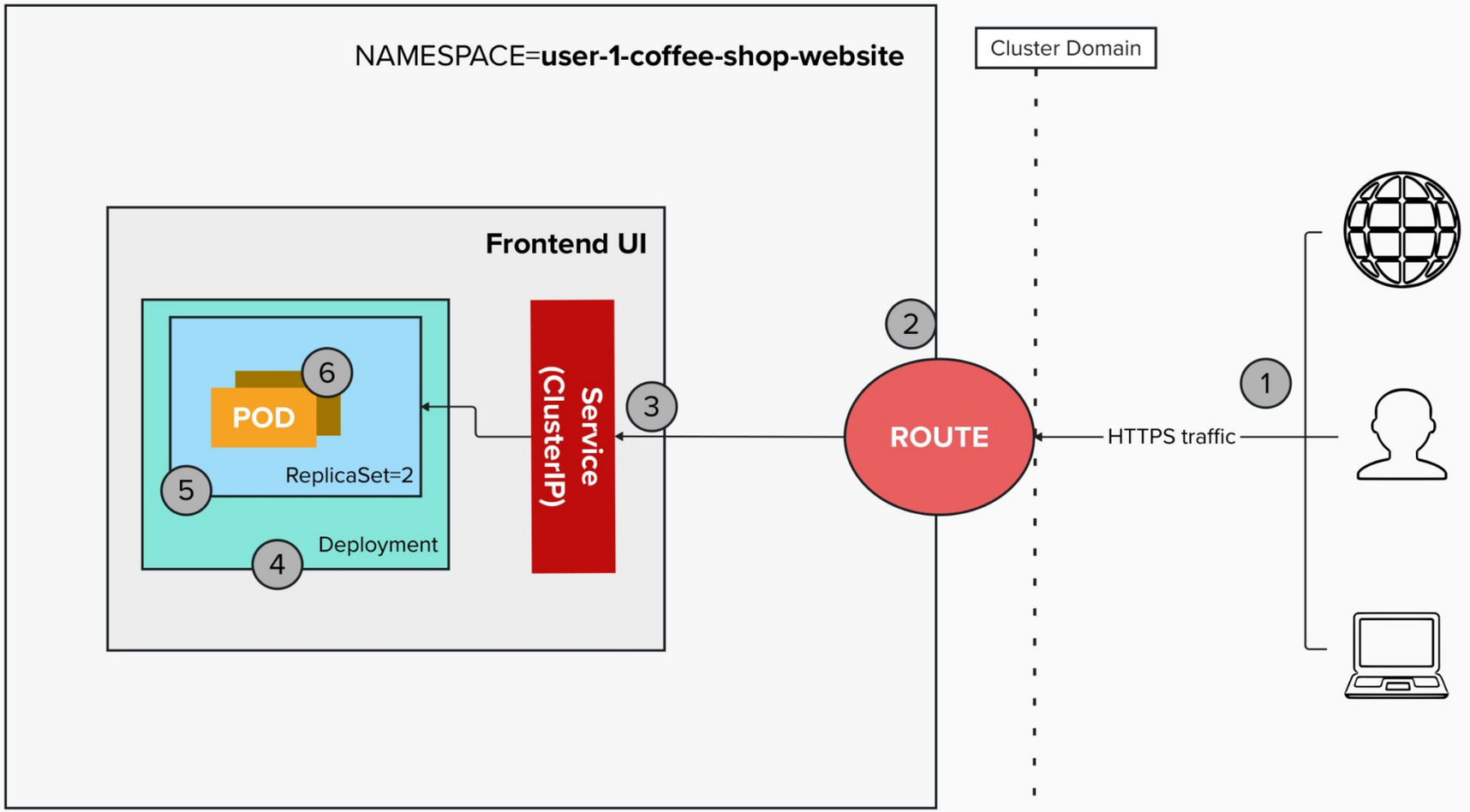
```
$ kubectl get pods
NAME                          READY     STATUS    RESTARTS   AGE
mydep-54f7ff7d6d-9gvll        1/1       Running   0          12s
mydep-54f7ff7d6d-cqvlq        1/1       Running   0          10s
mydep-54f7ff7d6d-gccr6        1/1       Running   0          7s
```

Red Hat IBM

Let’s deploy an application!

Concepts and resources

coffee shop website



Red Hat IBM

StatefulSet

- Tailored to managing Pods that must persist or maintain state.
- Pod lifecycle will be ordered and follow consistent patterns.
- Assigned a unique ordinal name following the convention of '*<statefulset name>-<ordinal index>*'.



Red Hat IBM

StatefulSet

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: sts-example
spec:
  replicas: 2
  revisionHistoryLimit: 3
  selector:
    matchLabels:
      app: stateful
  serviceName: app
  updateStrategy:
   type: RollingUpdate
   rollingUpdate:
     partition: 0
  template:
    metadata:
      labels:
        app: stateful
```

<continued>

<continued>

```yaml
    spec:
      containers:
      - name: nginx
        image: nginx:stable-alpine
        ports:
        - containerPort: 80
        volumeMounts:
        - name: www
          mountPath: /usr/share/nginx/html
  volumeClaimTemplates:
  - metadata:
      name: www
    spec:
      accessModes: [ "ReadWriteOnce" ]
      storageClassName: standard
      resources:
        requests:
          storage: 1Gi
```

Red Hat IBM

StatefulSet

- revisionHistoryLimit: The number of previous iterations of the StatefulSet to retain.
- serviceName: The name of the associated headless service; or a service without a **ClusterIP**.

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: sts-example
spec:
  replicas: 2
  revisionHistoryLimit: 3
  selector:
    matchLabels:
      app: stateful
  serviceName: app
  updateStrategy:
   type: RollingUpdate
   rollingUpdate:
     partition: 0
  template:
    <pod template>
```

Red Hat IBM

containers are created from container images

Storage

- Volumes
- Persistent Volumes
- Persistent Volume Claims
- StorageClass

Red Hat IBM

Storage

Pods by themselves are useful, but many workloads require exchanging data between containers, or persisting some form of data.

For this we have **Volumes**, **PersistentVolumes**, **PersistentVolumeClaims**, and **StorageClasses**.

StorageClass

- Storage classes are an abstraction on top of an external storage resource (PV)
- Work hand-in-hand with the external storage system to enable **dynamic provisioning** of storage by eliminating the need for the cluster admin to pre-provision a PV

StorageClass

- provisioner: Defines the '*driver*' to be used for provisioning of the external storage.
- parameters: A hash of the various configuration parameters for the provisioner.
- reclaimPolicy: The behaviour for the backing storage when the PVC is deleted.
  - Retain - manual clean-up
  - Delete - storage asset deleted by provider

```
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: standard
provisioner: kubernetes.io/gce-pd
parameters:
  type: pd-standard
  zones: us-central1-a, us-central1-b
reclaimPolicy: Delete
```

Red Hat IBM

Volumes

- Storage that is tied to the **Pod’s Lifecycle**.
- A pod can have one or more types of volumes attached to it.
- Can be consumed by any of the containers within the pod.
- Survive Pod restarts; however their durability beyond that is dependent on the Volume Type.

Volumes

- volumes: A list of volume objects to be attached to the Pod. Every object within the list must have it's own unique name.
- volumeMounts: A container specific list referencing the Pod volumes by name, along with their desired mountPath.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: volume-example
spec:
  containers:
  - name: nginx
    image: nginx:stable-alpine
    volumeMounts:
    - name: html
      mountPath: /usr/share/nginx/html
      ReadOnly: true
  - name: content
    image: alpine:latest
    command: ["/bin/sh", "-c"]
    args:
      - while true; do
          date >> /html/index.html;
          sleep 5;
        done
    volumeMounts:
    - name: html
      mountPath: /html

  volumes:
  - name: html
    emptyDir: {}
```

Red Hat IBM

Persistent Volumes

- A **PersistentVolume** (PV) represents a storage resource.
- PVs are a **cluster wide resource** linked to a backing storage provider: NFS, GCEPersistentDisk, RBD etc.
- Generally provisioned by an administrator.
- Their lifecycle is handled independently from a pod
- **CANNOT** be attached to a Pod directly. Relies on a **PersistentVolumeClaim**

Red Hat IBM

PersistentVolume

- `capacity.storage:` The total amount of available storage.
- `volumeMode:` The type of volume, this can be either `Filesystem` or `Block`.
- `accessModes:` A list of the supported methods of accessing the volume. Options include:
  - `ReadWriteOnce`
  - `ReadOnlyMany`
  - `ReadWriteMany`

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: nfsserver
spec:
  capacity:
    storage: 50Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Delete
  storageClassName: slow
  mountOptions:
    - hard
    - nfsvers=4.1
  nfs:
    path: /exports
    server: 172.22.0.42
```

Red Hat IBM

PersistentVolume

- persistentVolumeReclaimPolicy: The behaviour for PVC's that have been deleted. Options include:
  - Retain - manual clean-up
  - Delete - storage asset deleted by provider.
- storageClassName: Optional name of the storage class that PVC's can reference. If provided, **ONLY** PVC's referencing the name consume use it.
- mountOptions: Optional mount options for the PV.

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: nfsserver
spec:
  capacity:
    storage: 50Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Delete
  storageClassName: slow
  mountOptions:
    - hard
    - nfsvers=4.1
  nfs:
    path: /exports
    server: 172.22.0.42
```

Red Hat IBM

PersistentVolumeClaims

- A **PersistentVolumeClaim** (PVC) is a **namespaced** request for storage.
- Satisfies a set of requirements instead of mapping to a storage resource directly.
- Ensures that an application's '*claim*' for storage is portable across numerous backends or providers.

PersistentVolumeClaim

- accessModes: The selected method of accessing the storage. This **MUST** be a subset of what is defined on the target PV or Storage Class.
  - ReadWriteOnce
  - ReadOnlyMany
  - ReadWriteMany
- resources.requests.storage: The desired amount of storage for the claim
- storageClassName: The name of the desired Storage Class

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: pvc-sc-example
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: slow
```

Red Hat IBM

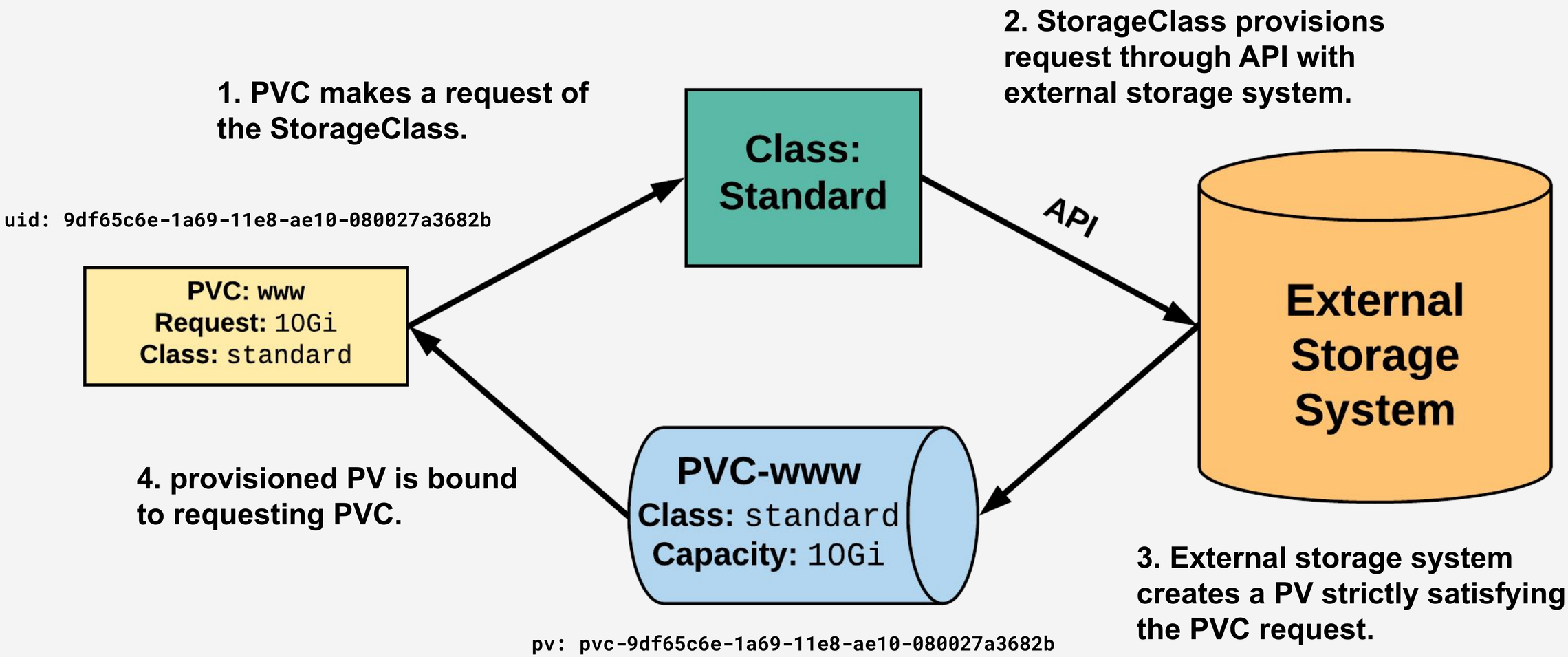
PVs and PVCs with Selectors

```yaml
kind: PersistentVolume
apiVersion: v1
metadata:
  name: pv-selector-example
  labels:
    type: hostpath
spec:
  capacity:
    storage: 2Gi
  accessModes:
    - ReadWriteMany
  hostPath:
    path: "/mnt/data"
```

```yaml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: pvc-selector-example
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 1Gi
  selector:
    matchLabels:
      type: hostpath
```

Red Hat IBM

StorageClass



Red Hat IBM

containers are created from container images

Configurations

- ConfigMap
- Secret

Red Hat IBM

Configuration

Kubernetes has an integrated pattern for decoupling configuration from application or container.

This pattern makes use of two Kubernetes components: **ConfigMaps** and **Secrets.**

ConfigMap

- Externalized data stored within kubernetes.
- Can be referenced through several different means:
  - environment variable
  - a command line argument (via env var)
  - injected as a file into a volume mount
- Can be created from a manifest, literals, directories, or files directly.

ConfigMap

data: Contains key-value pairs of ConfigMap contents.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: manifest-example
data:
  state: Michigan
  city: Ann Arbor
  content: |
    Look at this,
    its multiline!
```

Red Hat IBM

Secret

- Functionally identical to a ConfigMap.
- Stored as **base64 encoded content.**
- Encrypted at rest within etcd (**if configured!**).
- Stored on each worker node in tmpfs directory.
- Ideal for username/passwords, certificates or other sensitive information that should not be stored in a container.

Secret

- type: There are three different types of secrets within Kubernetes:
  - docker-registry - credentials used to authenticate to a container registry
  - generic/Opaque - literal values from different sources
  - tls - a certificate based secret
- data: Contains key-value pairs of base64 encoded content.

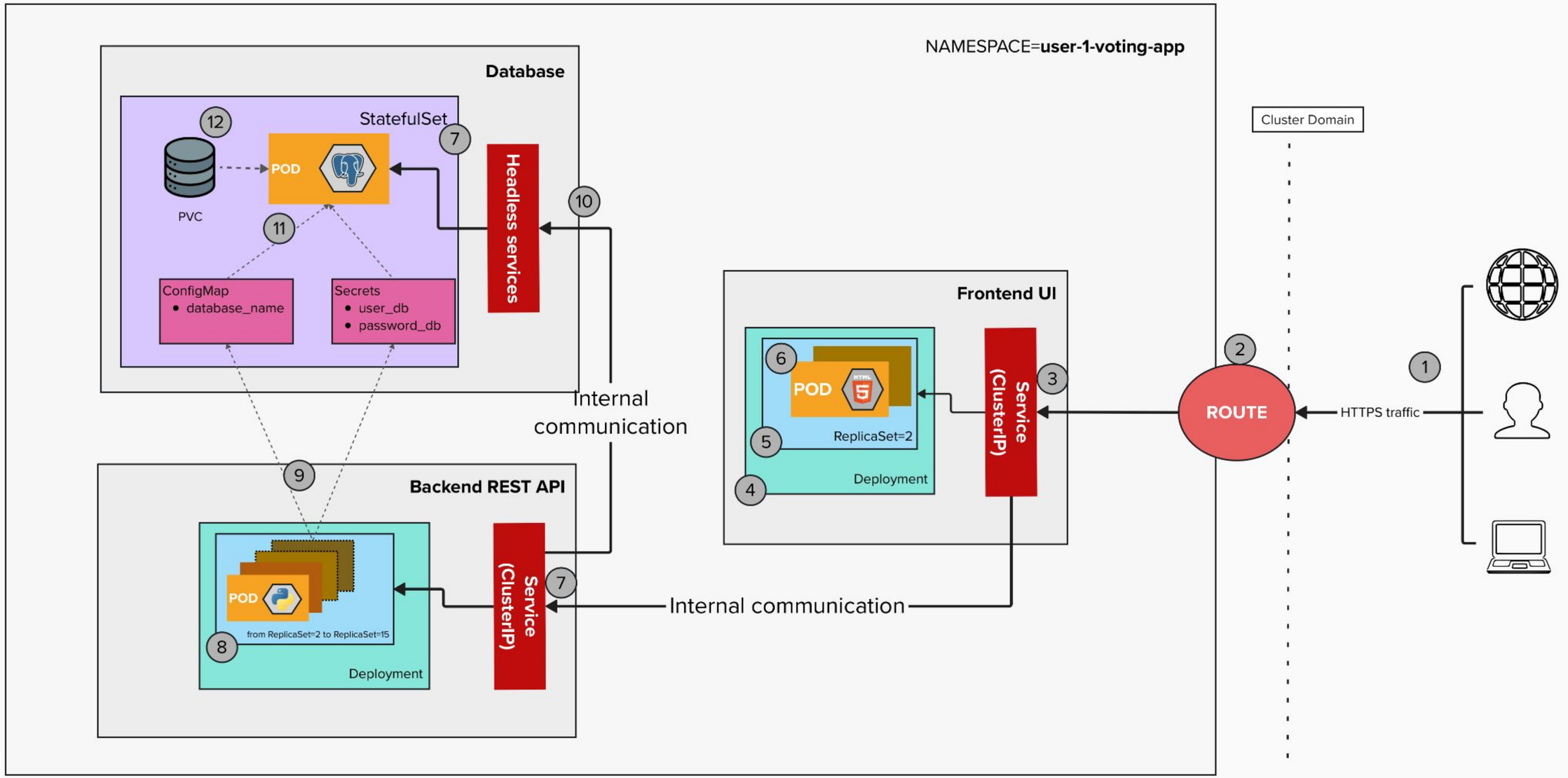
```
apiVersion: v1
kind: Secret
metadata:
  name: manifest-secret
type: Opaque
data:
  username: ZXhhbXBsZQ==
  password: bXlwYXNzd29yZA==
```

Red Hat IBM

Let’s deploy another application!

Red Hat IBM

Voting App



Red Hat IBM

It is time you do it by yourself

E-commerce app

Red Hat IBM

Questions?

Adapted from slides by Joe Beda (Gluecon 2017).

Licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0).

Changes were made.